visual_odom

Generated by Doxygen 1.9.8

1 Visual Odometry SLAM Package 1
1.1 visual_odom . . . 1
1.1.1 Authors . . . 1
1.2 visual_odom . . . 1
1.2.1 1. Overview . . . 1
1.2.1.1 UML Diagram . . . 1
1.2.1.2 System Pipeline . . . 1
1.2.1.3 TF Tree . . . 2
1.2.1.4 ROS Topics . . . 2
1.2.1.5 Dependencies . . . 2
1.2.2 2. Package Structure . . . 2
1.2.2.1 Module Descriptions . . . 3
1.2.2.2 Configuration (param.yaml) . . . 7
1.2.3 3. Build and Run . . . 7
1.2.3.1 Build . . . 7
1.2.3.2 Launch (recommended) . . . 7
1.2.3.3 Node only (without launch file) . . . 7
1.2.4 4. Documentation . . . 7
1.2.4.1 Generate HTML docs . . . 7
1.2.4.2 Generate PDF from LaTeX output . . . 7

2 Namespace Index 9
2.1 Package List . . . 9

3 Hierarchical Index 11
3.1 Class Hierarchy . . . 11

4 Class Index 13
4.1 Class List . . . 13

5 File Index 15
5.1 File List . . . 15

6 Namespace Documentation 17
6.1 metadata Namespace Reference . . . 17
6.1.1 Variable Documentation . . . 17
6.1.1.1 avoid_ros_namespace_conventions . . . 17
6.1.1.2 compression_format . . . 18
6.1.1.3 compression_mode . . . 18
6.1.1.4 custom_data . . . 18
6.1.1.5 deadline . . . 18
6.1.1.6 depth . . . 18
6.1.1.7 durability . . . 18
6.1.1.8 duration . . . 18

6.1.1.9 files . . . 18
6.1.1.10 lifespan . . . 18
6.1.1.11 liveliness . . . 18
6.1.1.12 liveliness_lease_duration . . . 18
6.1.1.13 message_count . . . 18
6.1.1.14 name . . . 18
6.1.1.15 nanoseconds . . . 18
6.1.1.16 nanoseconds_since_epoch . . . 18
6.1.1.17 nsec . . . 19
6.1.1.18 offered_qos_profiles . . . 19
6.1.1.19 relative_file_paths . . . 19
6.1.1.20 reliability . . . 19
6.1.1.21 ros_distro . . . 19
6.1.1.22 rosbag2_bagfile_information . . . 19
6.1.1.23 sec . . . 19
6.1.1.24 serialization_format . . . 19
6.1.1.25 starting_time . . . 19
6.1.1.26 storage_identifier . . . 19
6.1.1.27 topics_with_message_count . . . 19
6.1.1.28 type . . . 19
6.1.1.29 type_description_hash . . . 19
6.1.1.30 version . . . 19
6.2 param Namespace Reference . . . 20
6.2.1 Variable Documentation . . . 20
6.2.1.1 evaluation_tolerance . . . 20
6.2.1.2 iterations . . . 20
6.2.1.3 matches_for_new_landmarks . . . 20
6.2.1.4 max_depth . . . 20
6.2.1.5 max_rotation_angle_deg . . . 21
6.2.1.6 min_inlier_ratio . . . 21
6.2.1.7 min_landmark_trust . . . 21
6.2.1.8 min_matches_for_ransac . . . 21
6.2.1.9 n_robot_samples . . . 21
6.2.1.10 pixel_tolerance . . . 21
6.2.1.11 ransac . . . 21
6.2.1.12 rgb_depth_sync_tolerance_sec . . . 21
6.2.1.13 ros__parameters . . . 21
6.2.1.14 rosbag_path . . . 21
6.2.1.15 sample_size . . . 21
6.2.1.16 visual_odom_node . . . 21
6.3 setup Namespace Reference . . . 22
6.3.1 Variable Documentation . . . 22

6.3.1.1 data_files . . . 22
6.3.1.2 description . . . 22
6.3.1.3 entry_points . . . 22
6.3.1.4 extras_require . . . 22
6.3.1.5 install_requires . . . 22
6.3.1.6 license . . . 22
6.3.1.7 maintainer . . . 22
6.3.1.8 maintainer_email . . . 22
6.3.1.9 name . . . 22
6.3.1.10 package_name . . . 22
6.3.1.11 packages . . . 23
6.3.1.12 version . . . 23
6.3.1.13 zip_safe . . . 23
6.4 visual_odom Namespace Reference . . . 23
6.5 visual_odom.config Namespace Reference . . . 23
6.5.1 Detailed Description . . . 23
6.6 visual_odom.constants Namespace Reference . . . 23
6.6.1 Detailed Description . . . 26
6.6.2 Function Documentation . . . 27
6.6.2.1 normalize_angle() . . . 27
6.6.3 Variable Documentation . . . 27
6.6.3.1 ACCUMULATED_POINTCLOUD_FRAME_TOPIC . . . 27
6.6.3.2 BASE_LINK_FRAME_ID . . . 27
6.6.3.3 CAMERA_ANGLE_HOR_RAD . . . 27
6.6.3.4 CAMERA_ANGLE_VER_RAD . . . 27
6.6.3.5 CAMERA_POS_IN_BASELINK . . . 27
6.6.3.6 CU . . . 27
6.6.3.7 CV . . . 27
6.6.3.8 DECREASE_TRUST_FACTOR . . . 27
6.6.3.9 DEPTH_IMAGE_TOPIC . . . 28
6.6.3.10 ERROR_MIN_DEPTH . . . 28
6.6.3.11 ERROR_QUADRATIC_DEPTH . . . 28
6.6.3.12 F . . . 28
6.6.3.13 FILTERED_ODOMETRY_TOPIC . . . 28
6.6.3.14 GAUSS_NOISE_THETA_SIGMA . . . 28
6.6.3.15 GAUSS_NOISE_X_SIGMA . . . 28
6.6.3.16 GAUSS_NOISE_Y_SIGMA . . . 28
6.6.3.17 IMU_ODOMETRY_TOPIC . . . 28
6.6.3.18 INCREASE_TRUST_VALUE . . . 28
6.6.3.19 INITIAL_LANDMARK_COUNT . . . 28
6.6.3.20 KEYPOINT_POINTCLOUD_FRAME_TOPIC . . . 28
6.6.3.21 KINECT_FRAME_ID . . . 29

6.6.3.22 KP_IMAGE_TOPIC . . . 29
6.6.3.23 MATCHES_FOR_NEW_LANDMARKS . . . 29
6.6.3.24 MAX_ADDITIONAL_NOISE_RANSAC_FAILURE . . . 29
6.6.3.25 MAX_ALTITUDE . . . 29
6.6.3.26 MAX_AZIMUTH . . . 29
6.6.3.27 MAX_DEPTH . . . 29
6.6.3.28 MAX_ROTATION_ANGLE_DEG . . . 29
6.6.3.29 MAX_U . . . 29
6.6.3.30 MAX_V . . . 29
6.6.3.31 MIN_DEPTH . . . 29
6.6.3.32 MIN_DET_VALUE . . . 29
6.6.3.33 MIN_LANDMARK_TRUST . . . 30
6.6.3.34 MIN_MATCHES_FOR_RANSAC . . . 30
6.6.3.35 N_ROBOT_SAMPLES . . . 30
6.6.3.36 NOISE_INCREMENT_RANSAC_FAILURE . . . 30
6.6.3.37 NOISE_ROTATION_THRESHOLD . . . 30
6.6.3.38 NOISE_TRANSLATION_THRESHOLD . . . 30
6.6.3.39 parameters . . . 30
6.6.3.40 PERCENTAGE_OF_PLAY_SPEED . . . 30
6.6.3.41 PIXEL_TOLERANCE . . . 30
6.6.3.42 POINTCLOUD_FRAME_TOPIC . . . 30
6.6.3.43 RANSAC_EVALUATION_TOLERANCE . . . 30
6.6.3.44 RANSAC_ITERATION . . . 30
6.6.3.45 RANSAC_MIN_INLIER_RATIO . . . 31
6.6.3.46 RANSAC_SAMPLE_SIZE . . . 31
6.6.3.47 RESAMPLE_POS_TOLERANCE . . . 31
6.6.3.48 RESAMPLE_THETA_TOLERANCE . . . 31
6.6.3.49 RESAMPLE_TIME_TOLERANCE . . . 31
6.6.3.50 RESOLUTION_CONE_H . . . 31
6.6.3.51 RESOLUTION_CONE_V . . . 31
6.6.3.52 RGB_DEPTH_SYNC_TOLERANCE_SEC . . . 31
6.6.3.53 RGB_IMAGE_TOPIC . . . 31
6.6.3.54 ROT_BK . . . 31
6.6.3.55 SIGMA_PIXEL . . . 31
6.6.3.56 SIGMA_THETA_ODOM_WHEEL_Q . . . 32
6.6.3.57 SIGMA_X_ODOM_WHEEL_Q . . . 32
6.6.3.58 SIGMA_Y_ODOM_WHEEL_Q . . . 32
6.6.3.59 TF_STATIC_TOPIC . . . 32
6.6.3.60 VISION_CONE_TOPIC . . . 32
6.6.3.61 VISUAL_ODOM_FRAME_ID . . . 32
6.6.3.62 VISUAL_ODOM_MSG_TOPIC . . . 32
6.6.3.63 VISUAL_ODOM_PATH_TOPIC . . . 32

6.6.3.64 WHEEL_ODOMETRY_TOPIC . . . 32
6.7 visual_odom.csv_creator_node Namespace Reference . . . 32
6.7.1 Detailed Description . . . 33
6.7.2 Function Documentation . . . 33
6.7.2.1 main() . . . 33
6.8 visual_odom.ekf_landmark Namespace Reference . . . 33
6.8.1 Detailed Description . . . 33
6.9 visual_odom.ekf_robot Namespace Reference . . . 33
6.9.1 Detailed Description . . . 33
6.10 visual_odom.landmark Namespace Reference . . . 33
6.10.1 Detailed Description . . . 33
6.10.2 Function Documentation . . . 34
6.10.2.1 kabsch() . . . 34
6.11 visual_odom.odom_buffer Namespace Reference . . . 34
6.12 visual_odom.robot Namespace Reference . . . 34
6.13 visual_odom.tf_methods Namespace Reference . . . 34
6.13.1 Detailed Description . . . 35
6.13.2 Function Documentation . . . 35
6.13.2.1 calculate_tf() . . . 35
6.13.2.2 kinect_depth_to_baselink() . . . 35
6.13.2.3 kinect_depth_to_odom() . . . 35
6.13.2.4 odom_to_baselink() . . . 36
6.13.2.5 pixel_to_kinect() . . . 36
6.13.2.6 pixel_to_kinect_() . . . 36
6.14 visual_odom.visual_odom_map Namespace Reference . . . 37
6.14.1 Detailed Description . . . 37
6.15 visual_odom.visual_odom_node Namespace Reference . . . 37
6.15.1 Detailed Description . . . 37
6.15.2 Function Documentation . . . 37
6.15.2.1 create_particle_path_markers() . . . 37
6.15.2.2 main() . . . 38
6.16 visual_odom_launch Namespace Reference . . . 38
6.16.1 Function Documentation . . . 38
6.16.1.1 generate_launch_description() . . . 38

7 Class Documentation **39**

7.1 visual_odom.constants.Coordinate Class Reference . . . 39
7.1.1 Detailed Description . . . 39
7.1.2 Member Function Documentation . . . 39
7.1.2.1 __add__() . . . 39
7.1.2.2 __mul__() . . . 40
7.1.2.3 __sub__() . . . 40

7.1.2.4 __truediv__() . . . 40
7.1.3 Member Data Documentation . . . 40
7.1.3.1 x . . . 40
7.1.3.2 y . . . 41
7.1.3.3 z . . . 41
7.2 visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark Class Reference . . . 41
7.2.1 Detailed Description . . . 42
7.2.2 Constructor & Destructor Documentation . . . 42
7.2.2.1 __init__() . . . 42
7.2.3 Member Function Documentation . . . 42
7.2.3.1 calculate_jacobian_F() . . . 42
7.2.3.2 calculate_jacobian_H() . . . 42
7.2.3.3 calculate_Q_matrix() . . . 43
7.2.3.4 calculate_R_sigma_matrix() . . . 43
7.2.3.5 computeKalmanGain() . . . 43
7.2.3.6 get_R() . . . 43
7.2.3.7 landmark_kalman_iteration() . . . 44
7.2.3.8 meas_func() . . . 44
7.2.3.9 prediction() . . . 44
7.2.3.10 predictMeasurement() . . . 45
7.2.3.11 set_jacobian_F() . . . 45
7.2.3.12 set_jacobian_H() . . . 45
7.2.3.13 set_Q() . . . 45
7.2.3.14 set_R() . . . 45
7.2.3.15 sigma_R_approximation() . . . 46
7.2.3.16 state_func() . . . 46
7.2.3.17 update() . . . 46
7.2.4 Member Data Documentation . . . 47
7.2.4.1 JF . . . 47
7.2.4.2 JH . . . 47
7.2.4.3 P . . . 47
7.2.4.4 Q . . . 47
7.2.4.5 R . . . 47
7.2.4.6 x . . . 47
7.3 visual_odom.ekf_robot.ExtendedKalmanFilterRobot Class Reference . . . 47
7.3.1 Detailed Description . . . 48
7.3.2 Constructor & Destructor Documentation . . . 48
7.3.2.1 __init__() . . . 48
7.3.3 Member Function Documentation . . . 48
7.3.3.1 add_noise_to_R() . . . 48
7.3.3.2 computeKalmanGain() . . . 48
7.3.3.3 get_covariance_p() . . . 49

7.3.3.4 get_state() . . . 49
7.3.3.5 prediction() . . . 49
7.3.3.6 set_Q() . . . 49
7.3.3.7 set_R() . . . 49
7.3.3.8 set_state() . . . 49
7.3.3.9 update() . . . 50
7.3.4 Member Data Documentation . . . 50
7.3.4.1 additional_noise . . . 50
7.3.4.2 F . . . 50
7.3.4.3 H . . . 50
7.3.4.4 P . . . 50
7.3.4.5 Q . . . 50
7.3.4.6 R . . . 50
7.3.4.7 x . . . 50
7.4 visual_odom.landmark.Landmark Class Reference . . . 50
7.4.1 Detailed Description . . . 52
7.4.2 Constructor & Destructor Documentation . . . 52
7.4.2.1 __init__() . . . 52
7.4.3 Member Function Documentation . . . 52
7.4.3.1 calculate_likelihood() . . . 52
7.4.3.2 decrease_trust() . . . 53
7.4.3.3 get_color() . . . 53
7.4.3.4 get_depth() . . . 53
7.4.3.5 get_descriptor() . . . 53
7.4.3.6 get_kinect_coordinates() . . . 53
7.4.3.7 get_odom_coordinates() . . . 53
7.4.3.8 get_P() . . . 54
7.4.3.9 get_trust() . . . 54
7.4.3.10 get_u() . . . 54
7.4.3.11 get_v() . . . 54
7.4.3.12 increase_trust() . . . 54
7.4.3.13 is_visible() . . . 54
7.4.3.14 landmark_kalman_iteration() . . . 55
7.4.3.15 reset_trust() . . . 55
7.4.3.16 set_odom_coordinates() . . . 55
7.4.4 Member Data Documentation . . . 55
7.4.4.1 color . . . 55
7.4.4.2 des . . . 55
7.4.4.3 kinect_coordinates . . . 55
7.4.4.4 landmark_ekf . . . 55
7.4.4.5 odom_coordinates . . . 56
7.4.4.6 P . . . 56

7.4.4.7 pixel_coor . . . 56
7.4.4.8 trust . . . 56
7.5 visual_odom.odom_buffer.odom_message Class Reference . . . 56
7.5.1 Member Data Documentation . . . 56
7.5.1.1 msg . . . 56
7.5.1.2 pose . . . 56
7.5.1.3 stamp . . . 57
7.6 visual_odom.odom_buffer.OdomBuffer Class Reference . . . 57
7.6.1 Constructor & Destructor Documentation . . . 57
7.6.1.1 __init__() . . . 57
7.6.2 Member Function Documentation . . . 57
7.6.2.1 add_odom_message() . . . 57
7.6.2.2 get_interpolated() . . . 57
7.6.2.3 get_latest() . . . 57
7.6.3 Member Data Documentation . . . 57
7.6.3.1 Odom_Buffer . . . 57
7.7 visual_odom.csv_creator_node.OdomImuCsvWriter Class Reference . . . 58
7.7.1 Detailed Description . . . 59
7.7.2 Constructor & Destructor Documentation . . . 59
7.7.2.1 __init__() . . . 59
7.7.3 Member Function Documentation . . . 59
7.7.3.1 destroy_node() . . . 59
7.7.3.2 imu_callback() . . . 59
7.7.3.3 normalize_angle() . . . 60
7.7.3.4 odom_callback() . . . 60
7.7.3.5 quaternion_to_yaw() . . . 60
7.7.3.6 visual_odom_callback() . . . 60
7.7.4 Member Data Documentation . . . 61
7.7.4.1 csv_file . . . 61
7.7.4.2 csv_path . . . 61
7.7.4.3 imu_callback . . . 61
7.7.4.4 imu_theta . . . 61
7.7.4.5 odom_callback . . . 61
7.7.4.6 odom_visual_theta . . . 61
7.7.4.7 odom_visual_x . . . 61
7.7.4.8 odom_visual_y . . . 61
7.7.4.9 visual_odom_callback . . . 61
7.7.4.10 wheel_x . . . 61
7.7.4.11 wheel_y . . . 62
7.8 visual_odom.constants.Parameters Class Reference . . . 62
7.8.1 Detailed Description . . . 62
7.8.2 Member Data Documentation . . . 62

7.8.2.1 matches_for_new_landmarks . . . 62
7.8.2.2 max_depth . . . 62
7.8.2.3 max_rotation_angle_deg . . . 63
7.8.2.4 min_landmark_trust . . . 63
7.8.2.5 min_matches_for_ransac . . . 63
7.8.2.6 n_robot_samples . . . 63
7.8.2.7 pixel_tolerance . . . 63
7.8.2.8 ransac_evaluation_tolerance . . . 63
7.8.2.9 ransac_iteration . . . 63
7.8.2.10 ransac_min_inlier_ratio . . . 63
7.8.2.11 ransac_sample_size . . . 63
7.8.2.12 rgb_depth_sync_tolerance_sec . . . 63
7.8.2.13 topic_visual_odometry_msg . . . 63
7.9 visual_odom.constants.PixelCoordinate Class Reference . . . 63
7.9.1 Detailed Description . . . 64
7.9.2 Member Data Documentation . . . 64
7.9.2.1 u . . . 64
7.9.2.2 v . . . 64
7.9.2.3 z . . . 64
7.10 visual_odom.constants.State Class Reference . . . 64
7.10.1 Detailed Description . . . 65
7.10.2 Member Function Documentation . . . 65
7.10.2.1 __add__() . . . 65
7.10.2.2 __mul__() . . . 65
7.10.2.3 __sub__() . . . 65
7.10.2.4 __truediv__() . . . 65
7.10.3 Member Data Documentation . . . 66
7.10.3.1 theta . . . 66
7.10.3.2 x . . . 66
7.10.3.3 y . . . 66
7.11 visual_odom.visual_odom_node.VisualOdom Class Reference . . . 66
7.11.1 Detailed Description . . . 68
7.11.2 Constructor & Destructor Documentation . . . 68
7.11.2.1 __init__() . . . 68
7.11.3 Member Function Documentation . . . 69
7.11.3.1 _stamp_to_sec() . . . 69
7.11.3.2 img_to_kp_des_filtered() . . . 69
7.11.3.3 iteration() . . . 69
7.11.3.4 listener_depth_callback() . . . 70
7.11.3.5 listener_rgb_callback() . . . 70
7.11.3.6 listener_wheel_odom_callback() . . . 70
7.11.3.7 parameter_callback() . . . 70

7.11.3.8 parameter_initialization() . . . 70
7.11.3.9 possible_resample() . . . 71
7.11.3.10 ransac_calculation() . . . 71
7.11.3.11 ransac_frame_to_frame() . . . 71
7.11.3.12 ransac_frame_to_map() . . . 72
7.11.4 Member Data Documentation . . . 72
7.11.4.1 best_robot . . . 72
7.11.4.2 best_robot_idx . . . 72
7.11.4.3 best_weight . . . 72
7.11.4.4 bf . . . 72
7.11.4.5 bridge . . . 72
7.11.4.6 covariance_P . . . 73
7.11.4.7 first_des . . . 73
7.11.4.8 first_frame_stamp . . . 73
7.11.4.9 first_iteration . . . 73
7.11.4.10 first_kp . . . 73
7.11.4.11 first_kp_depth . . . 73
7.11.4.12 first_pos . . . 73
7.11.4.13 first_theta . . . 73
7.11.4.14 frame_depth . . . 73
7.11.4.15 frame_depth_stamp . . . 73
7.11.4.16 frame_rgb . . . 73
7.11.4.17 frame_stamp . . . 73
7.11.4.18 orb . . . 73
7.11.4.19 parameter_callback . . . 73
7.11.4.20 pos_baselink . . . 73
7.11.4.21 position_initialized . . . 74
7.11.4.22 publisher_accumulated_pixels . . . 74
7.11.4.23 publisher_cone . . . 74
7.11.4.24 publisher_image . . . 74
7.11.4.25 publisher_keypoints_3d . . . 74
7.11.4.26 publisher_visual_odometry_msg . . . 74
7.11.4.27 robots . . . 74
7.11.4.28 subscription_depth . . . 74
7.11.4.29 subscription_rgb . . . 74
7.11.4.30 subscription_wheel_odom . . . 74
7.11.4.31 tf_broadcaster . . . 74
7.11.4.32 tf_buffer . . . 74
7.11.4.33 tf_listener . . . 74
7.11.4.34 theta . . . 74
7.11.4.35 total_publisher . . . 74
7.11.4.36 valid_des . . . 75

7.11.4.37 valid_des_last . . . 75
7.11.4.38 valid_kp . . . 75
7.11.4.39 valid_kp_depth . . . 75
7.11.4.40 valid_kp_depth_last . . . 75
7.11.4.41 valid_kp_last . . . 75
7.11.4.42 visible_landmarks . . . 75
7.11.4.43 visual_odom_path . . . 75
7.12 visual_odom.visual_odom_map.VisualOdomMap Class Reference . . . 75
7.12.1 Detailed Description . . . 77
7.12.2 Constructor & Destructor Documentation . . . 78
7.12.2.1 __init__() . . . 78
7.12.3 Member Function Documentation . . . 78
7.12.3.1 add_landmark() . . . 78
7.12.3.2 add_landmarks_from_kps() . . . 78
7.12.3.3 calculate_log_weight() . . . 78
7.12.3.4 cleanup_old_landmarks() . . . 79
7.12.3.5 get_all_landmarks() . . . 79
7.12.3.6 get_colors() . . . 79
7.12.3.7 get_depth() . . . 79
7.12.3.8 get_descriptors() . . . 80
7.12.3.9 get_kinect_coordinates() . . . 80
7.12.3.10 get_landmark() . . . 80
7.12.3.11 get_odom_coordinates() . . . 80
7.12.3.12 get_size() . . . 80
7.12.3.13 get_u_coordinates() . . . 81
7.12.3.14 get_v_coordinates() . . . 81
7.12.3.15 get_visible_landmarks() . . . 81
7.12.3.16 landmark_kalman_iteration() . . . 81
7.12.3.17 publish_pointcloud_map() . . . 82
7.12.4 Member Data Documentation . . . 82
7.12.4.1 descriptors . . . 82
7.13 visual_odom.robot.VisualRobotSample Class Reference . . . 82
7.13.1 Constructor & Destructor Documentation . . . 83
7.13.1.1 __init__() . . . 83
7.13.2 Member Function Documentation . . . 84
7.13.2.1 accumulate_pixels() . . . 84
7.13.2.2 get_covariance_P() . . . 84
7.13.2.3 get_drawn_keypoints() . . . 84
7.13.2.4 get_log_weight() . . . 84
7.13.2.5 get_position() . . . 84
7.13.2.6 get_theta() . . . 84
7.13.2.7 get_weight() . . . 85

7.13.2.8 init_camera_cone() . . . 85
7.13.2.9 predict_with_wheel_odom() . . . 85
7.13.2.10 publish_accumulated_pixels() . . . 85
7.13.2.11 publish_odometry_msg() . . . 85
7.13.2.12 publish_pixels() . . . 86
7.13.2.13 publish_pointcloud_map() . . . 86
7.13.2.14 publish_vision_cone() . . . 86
7.13.2.15 publish_yourself() . . . 86
7.13.2.16 ransac_failed() . . . 87
7.13.2.17 robot_iteration() . . . 87
7.13.2.18 set_log_weight() . . . 87
7.13.2.19 set_state() . . . 87
7.13.2.20 set_weight() . . . 88
7.13.3 Member Data Documentation . . . 88
7.13.3.1 acc_ransac_noise_delta . . . 88
7.13.3.2 accumulated_pixels . . . 88
7.13.3.3 bf . . . 88
7.13.3.4 cone_publisher . . . 88
7.13.3.5 covariance_P . . . 88
7.13.3.6 delta_ransac_state . . . 88
7.13.3.7 delta_wheel_odom_ransac . . . 88
7.13.3.8 ekf . . . 88
7.13.3.9 first_iteration . . . 88
7.13.3.10 frame_depth . . . 88
7.13.3.11 frame_rgb . . . 88
7.13.3.12 index . . . 88
7.13.3.13 last_wheel_odom . . . 89
7.13.3.14 log_weight . . . 89
7.13.3.15 map_publisher . . . 89
7.13.3.16 noise_Q . . . 89
7.13.3.17 odometry_msg_publisher . . . 89
7.13.3.18 path . . . 89
7.13.3.19 path_color . . . 89
7.13.3.20 pos_baselink . . . 89
7.13.3.21 pos_visual_odom . . . 89
7.13.3.22 ransac_delta_p . . . 89
7.13.3.23 ransac_delta_theta . . . 89
7.13.3.24 ransac_draw_keypoints . . . 89
7.13.3.25 ransac_inlier_ratio . . . 89
7.13.3.26 theta . . . 89
7.13.3.27 total_publisher . . . 89
7.13.3.28 visible_landmarks . . . 90

7.13.3.29 visual_odom_map 90

8 File Documentation **91**

8.1 bagfiles/20260428_Bag_Without_TF/metadata.yaml File Reference 91

8.2 bagfiles/20260519_TransXY_bag/metadata.yaml File Reference 92

8.3 bagfiles/20260616_Around_The_Desks_bag/metadata.yaml File Reference 92

8.4 README.md File Reference 92

8.5 ros2_ws/src/visual_odom/config/param.yaml File Reference 92

8.6 ros2_ws/src/visual_odom/launch/visual_odom_launch.py File Reference 93

8.7 ros2_ws/src/visual_odom/setup.py File Reference 93

8.8 ros2_ws/src/visual_odom/visual_odom/__init__.py File Reference 93

8.9 ros2_ws/src/visual_odom/visual_odom/constants.py File Reference 93

8.10 ros2_ws/src/visual_odom/visual_odom/csv_creator_node.py File Reference 96

8.11 ros2_ws/src/visual_odom/visual_odom/ekf_landmark.py File Reference 97

8.12 ros2_ws/src/visual_odom/visual_odom/ekf_robot.py File Reference 97

8.13 ros2_ws/src/visual_odom/visual_odom/landmark.py File Reference 97

8.14 ros2_ws/src/visual_odom/visual_odom/odom_buffer.py File Reference 97

8.15 ros2_ws/src/visual_odom/visual_odom/robot.py File Reference 98

8.16 ros2_ws/src/visual_odom/visual_odom/tf_methods.py File Reference 98

8.17 ros2_ws/src/visual_odom/visual_odom/visual_odom_map.py File Reference 98

8.18 ros2_ws/src/visual_odom/visual_odom/visual_odom_node.py File Reference 99

Index **101**

Chapter 1

Visual Odometry SLAM Package

1.1 visual_odom

1.1.1 Authors

Lukas Grob
Paul Schellenberg

This project was developed by the authors listed above as part of a visual odometry and SLAM research/development project.
A ROS 2 package implementing a visual odometry pipeline for a mobile robot equipped with a Kinect RGB-D camera. ...

1.2 visual_odom

A ROS 2 package implementing a visual odometry pipeline for a mobile robot equipped with a Kinect RGB-D camera. The system detects ORB keypoints in incoming RGB frames, lifts them into 3D using registered depth data, and estimates frame-to-frame motion via RANSAC Kabsch alignment. Pose estimation is embedded in a Rao-Blackwellized particle filter: each particle maintains its own landmark map and an EKF per tracked landmark. The best particle is selected by log-likelihood weight. The resulting pose is broadcast as a TF transform and published as a `nav_msgs/Odometry` message.

1.2.1 1. Overview

1.2.1.1 UML Diagram

1.2.1.2 System Pipeline

```
RGB + Depth frames
      |
      v
ORB keypoint detection + depth filter  [MIN_DEPTH .. MAX_DEPTH]
      |
      v
Frame-to-frame RANSAC + Kabsch  -->  delta pose (translation + rotation)
      |
      v
Per-particle update  (N = n_robot_samples particles)
  |-- Wheel odometry EKF prediction (parralel to that)
  |-- Visible landmark query (FOV check)
  |-- BFMatcher: current frame vs last frame
  |-- Apply delta pose to particle position
  |-- BFMatcher: current frame vs visible landmarks
  |-- Landmark EKF update for matched landmarks
  |-- Log-weight calculation
  |
  v
Best particle selection (argmax normalized weight)
      |
      v
TF broadcast (odom -> base_link) + nav_msgs/Odometry
```

1.2.1.3 TF Tree

```
odom
  +-- base_link
        +-- kinect_depth   (from /tf_static in rosbag)
```

The `odom -> base_link` transform is updated on every processed RGB frame. The `base_link -> kinect_depth` static transform is provided by the rosbag.

1.2.1.4 ROS Topics

Topic	Type	Direction
RGB_IMAGE_TOPIC	`sensor_msgs/Image`	Subscribed
DEPTH_IMAGE_TOPIC	`sensor_msgs/Image`	Subscribed
WHEEL_ODOMETRY_TOPIC	`nav_msgs/Odometry`	Subscribed
VISUAL_ODOM_MSG_TOPIC	`nav_msgs/Odometry`	Published
KEYPOINT_POINTCLOUD_↩ FRAME_TOPIC	`sensor_msgs/PointCloud2`	Published – landmark map
POINTCLOUD_FRAME_TOPIC	`sensor_msgs/PointCloud2`	Published – full depth frame (not used for the main time)
VISION_CONE_TOPIC	`visualization_msgs/↩ Marker`	Published
KP_IMAGE_TOPIC	`sensor_msgs/Image`	Published
VISUAL_ODOM_PATH_TOPIC	`visualization_msgs/↩ MarkerArray`	Published

1.2.1.5 Dependencies

Package	Purpose
`rclpy`	ROS 2 Python client library – node lifecycle, publishers, subscriptions, parameters
`rcl_interfaces`	`SetParametersResult` for dynamic parameter callbacks
`builtin_interfaces`	ROS 2 built-in message types (timestamps, duration)
`std_msgs`	`Header` for PointCloud2 and TF stamped messages
`sensor_msgs`	`Image`, `PointCloud2`, `PointField`, `Imu`
`sensor_msgs_py`	PointCloud2 creation utilities (`point_cloud2.create_cloud`)
`nav_msgs`	`Odometry` – wheel, IMU and visual odometry messages
`geometry_msgs`	`TransformStamped`, `Point`, `Pose2D`, `Quaternion`
`visualization_msgs`	`Marker`, `MarkerArray` – particle path and vision cone visualisation
`tf2_ros`	TF broadcaster, listener and buffer for coordinate frame transforms
`cv_bridge`	ROS `Image` <-> OpenCV `Mat` conversion
`python3-opencv`	ORB detection, descriptor computation, BFMatcher, image I/O
`python3-numpy`	Numerical arrays, linear algebra (EKF matrices, RANSAC, Kabsch)
`python3-scipy`	Quaternion/rotation conversions (`scipy.spatial.transform.↩ Rotation`), KDTree

1.2.2 2. Package Structure

```
visual_odom/
+-- visual_odom/
|     +-- constants.py          # Global dataclasses, configuration constants, helper functions
|     +-- landmark.py           # Landmark dataclass, EKF wrapper, and Kabsch estimator
|     +-- ekf_landmark.py       # Extended Kalman Filter for individual landmark position
|     +-- ekf_robot.py          # Extended Kalman Filter for robot pose (wheel odom based)
|     +-- robot.py              # VisualRobotSample: one particle in the particle filter
|     +-- visual_odom_map.py    # VisualOdomMap: persistent landmark map per particle
|     +-- tf_methods.py         # Coordinate frame transformation utilities
|     +-- visual_odom_node.py   # Main ROS 2 node: orchestrates particle filter
+-- launch/
|     +-- visual_odom_launch.py # Launch file: node + rosbag playback + RViz2
+-- config/
```

```
      +-- param.yaml            # Runtime parameter configuration
+-- rviz/
      +-- rviz2_config.rviz     # Rviz2 configuration for Autostart
```

1.2.2.1 Module Descriptions

Dataclasses

Class	Purpose
`Coordinate`	3D position container (x, y, z) in meters. Supports arithmetic operators.
`PixelCoordinate`	2D image pixel (u, v) paired with its raw depth value z.
`State`	2D robot pose (x, y, theta) in meters and radians. Supports arithmetic operators.
`Parameters`	Runtime parameter bundle populated at node startup from ROS 2 parameter declarations.

Key Constant Groups

Group	Examples
ROS Topics	`RGB_IMAGE_TOPIC`, `DEPTH_IMAGE_TOPIC`, `WHEEL_ODOMETRY_TOPIC`
TF Frame IDs	`KINECT_FRAME_ID`, `BASE_LINK_FRAME_ID`, `VISUAL_ODOM_FRAME_ID`
Camera Model	`F`, `CU`, `CV`, `MAX_AZIMUTH`, `MAX_ALTITUDE`, `ROT_BK`
Depth Filter	`MIN_DEPTH`, `MAX_DEPTH`
RANSAC	`RANSAC_EVALUATION_TOLERANCE`, `RANSAC_ITERATION`, `RANSAC_MIN_INLIER_RATIO`
Particle Filter	`N_ROBOT_SAMPLES`, `NOISE_INCREMENT_RANSAC_FAILURE`
Landmark Management	`MIN_LANDMARK_TRUST`, `INCREASE_TRUST_VALUE`, `DECREASE_TRUST_FACTOR`
Noise / EKF	`SIGMA_X_ODOM_WHEEL_Q`, `ERROR_MIN_DEPTH`, `ERROR_QUADRATIC_DEPTH`

Helper Functions

- `normalize_angle(angle)` – Maps any angle in radians into `[-pi, pi]` using `(angle + pi) % (2*pi) - pi`.

---

landmark.py

Defines the `Landmark` class representing a single tracked 3D map feature. Each landmark owns an `ExtendedKalmanFilterLandmark` instance that refines its odom-frame position over time. Also contains the standalone `kabsch()` function used for 2D rigid-body alignment in RANSAC.

Landmark

On construction, pixel coordinates (u, v) and depth z are back-projected into 3D Kinect-frame coordinates via the pinhole camera model:

```
x = z * (u - CU) / F
y = z * (v - CV) / F
```

The odom-frame position is set separately via `set_odom_coordinates()` after the coordinate frame transform is applied.

Method	Description
`is_visible(pos_camera_odom, theta_robot)`	Checks azimuth, altitude, and depth constraints against the camera FOV.
`landmark_kalman_iteration(pos, theta, pixel_coor)`	Delegates one EKF predict+update cycle to the embedded `ExtendedKalmanFilterLandmark`.
`calculate_likelihood(z_pos_odom, theta)`	Computes the likelihood of a measurement given the current landmark state and covariance. Used for particle weight calculation.

Method	Description
increase_trust() / decrease_trust()	Trust management: additive increment on observation, multiplicative decay when not seen.
get_odom_coordinates() / set_odom_↩ coordinates(c)	Accessors for the odom-frame position.
get_descriptor() / get_kp()	Accessors for the ORB descriptor and keypoint.

kabsch(P_i, Q_i)
Computes the optimal 2D rotation R and translation t that aligns point set Q onto P in a least-squares sense. Mean-centers both sets; theta is derived analytically via atan2. Returns (R, t, theta) or (None, None, None) for degenerate inputs.

ekf_landmark.py

Implements the Extended Kalman Filter for tracking an individual landmark position in the odom frame. The state vector is a 3D Coordinate (x, y, z).

Measurement model

The expected measurement transforms the landmark from odom into the robot's local frame using the current heading rotation:

```
h(x) = R(theta).T * (x_landmark - x_robot)
```

Noise model

The measurement covariance R is depth-dependent, propagated from pixel-space noise to baselink:

```
R_baselink = ROT_BK * J_kinect * R_pixel * J_kinect.T * ROT_BK.T
```

where the depth error grows quadratically: s_z = ERROR_MIN_DEPTH + ERROR_QUADRATIC_DEPTH * (z - z_min)^2.

Method	Description
landmark_kalman_iteration(pos, theta, pixel_coor)	Full predict + update cycle. Returns updated (x, P).
prediction(x)	Applies identity state transition; propagates covariance with process noise Q.
update(x_tt1, P_tt1, pos_robot, theta, kp)	Computes Kalman gain, innovation, and Joseph-form covariance update.
computeKalmanGain(P_tt1)	Returns K = P * H.T * (H * P * H.T + R)^-1.

ekf_robot.py

Implements the Extended Kalman Filter for the robot pose (x, y, theta) using wheel odometry as the prediction input and RANSAC visual odometry as the measurement.
The state transition is additive: x_{t|t-1} = x_{t-1} + delta_wheel_odom. The measurement noise R scales with the RANSAC evaluation tolerance plus any accumulated additional noise from repeated RANSAC failures.

Method	Description
prediction(delta_wheel_odom)	Propagates state and covariance with wheel odometry delta.
update(z, inlier_ratio)	Fuses RANSAC pose measurement using Joseph-form covariance update.
add_noise_to_R()	Increments additional measurement noise after a RANSAC failure, capped at MAX_ADDITIONAL_NOISE_RANSAC_FAILURE.

robot.py

Represents a single particle in the Rao-Blackwellized particle filter. Each particle maintains its own Visual↩ OdomMap, ExtendedKalmanFilterRobot, and accumulated pose. All particles receive the same RANSAC result computed centrally by VisualOdom and apply it independently.

robot_iteration(...)
The per-frame update sequence for one particle:

1. On first iteration: seed the landmark map until INITIAL_LANDMARK_COUNT is reached.
2. Apply the RANSAC delta pose to the particle position with optional Gaussian noise.
3. Query get_visible_landmarks() for landmarks in the current camera FOV.
4. Match visible landmark descriptors against current frame descriptors via BFMatcher.
5. Run landmark_kalman_iteration() on all matched landmarks.
6. Compute log_weight via calculate_log_weight().
7. Run cleanup_old_landmarks(): increase trust for matched landmarks, decay others, remove landmarks below MIN_LANDMARK_TRUST.
8. Add unmatched keypoints as new landmarks if match count is below MATCHES_FOR_NEW_LANDMARKS.

predict_with_wheel_odom(state_wheel_odom)
Computes delta from the last stored wheel odometry state and feeds it to ExtendedKalmanFilterRobot.↩ prediction().
ransac_failed()
Falls back to wheel odometry delta for position update and increments noise in the robot EKF via add_noise_↩ to_R().

visual_odom_map.py

Implements VisualOdomMap, a list subclass holding all Landmark objects for one particle. Provides spatial queries, batch EKF updates, log-weight calculation, and PointCloud2 publishing.

Method	Description
add_landmark(landmark)	Appends a single landmark.
add_landmarks_from_kps(P_init, kps, des, frame_rgb, kp_depth, theta, pos_baselink)	Creates landmarks from ORB keypoints, computes odom-frame coordinates, and add the keypoints as landmarks to the map.
get_visible_landmarks(camera_pos, theta)	Returns a new VisualOdomMap containing only landmarks passing the is_visible() FOV check.
get_descriptors()	Returns all descriptors stacked as a NumPy array for batch BFMatcher input.
landmark_kalman_iteration(pos, theta, indices, kp_pos)	Runs one EKF iteration on each indexed landmark with its matched pixel coordinate.
calculate_log_weight(pos, theta, indices, kp_pos)	Sums log-likelihoods from Landmark.↩ calculate_likelihood() for all matched landmarks.
cleanup_old_landmarks(visible, landmark_index)	Updates trust values and removes landmarks below threshold.
publish_pointcloud_map(publisher, time)	Publishes landmark odom positions and RGB colors as sensor_msgs/PointCloud2.

tf_methods.py

Stateless coordinate frame transformation utilities. All transforms are computed analytically using the calibrated rotation matrix ROT_BK and the fixed camera offset CAMERA_POS_IN_BASELINK. No tf2_ros buffer lookups are performed.

Function	From / To	Notes
`kinect_depth_to_↩ odom(kinect_point, theta, pos_baselink)`	kinect to odom	Applies ROT_BK then heading rotation and translation.
`kinect_depth_to_↩ baselink(kinect_point)`	kinect to base_link	Applies ROT_BK plus camera offset.
`odom_to_baselink(odom↩ _point, theta, pos_↩ baselink)`	odom to base_link	Inverse rotation by heading, subtract robot position.
`pixel_to_kinect(kp)`	pixel + depth to kinect 3D	Back-projects via pinhole model; output in meters.
`pixel_to_kinect_(u, v, z)`	pixel + depth to kinect 3D	Same as above, returns NDArray instead of Coordinate.

visual_odom_node.py

Main ROS 2 node (`VisualOdom`). Orchestrates the particle filter, runs RANSAC between consecutive frames, and broadcasts the pose of the best particle as TF.

Initialization

Parameters are loaded from the ROS 2 parameter server, backed by `param.yaml`. A `cv2.ORB` detector and `cv2.BFMatcher` (NORM_HAMMING, crossCheck) are created. N particles (`VisualRobotSample`) are instantiated on the first valid RGB frame.

Callbacks

Callback	Description
`listener_rgb_callback(msg)`	Main processing callback. Runs ORB detection, depth filtering, RANSAC, particle iteration, best-particle selection, and TF broadcast.
`listener_depth_callback(msg)`	Converts depth image to NumPy array and records timestamp for sync checks.
`listener_wheel_odom_callback(msg)`	Extracts pose from wheel odometry message and calls `predict_with_wheel_odom()` on all particles.

Key methods

Method	Description
`ransac(matches, valid_kp, valid_↩ kp_depth, valid_kp_last, valid_kp_↩ depth_last)`	Frame-to-frame RANSAC: builds P (last frame 3D) and Q (current frame 3D) in base_link, iterates Kabsch hypotheses, refines on inliers. Returns `(translation, delta_theta, draw_↩ keypoints, inlier_count)`.
`img_to_kp_des_filtered(msg)`	Detects ORB keypoints, filters by depth and pixel-neighborhood exclusion, computes descriptors.
`iteration(...)`	Runs `robot_iteration()` on all particles, normalizes log-weights, selects best particle.
`calculate_tf(pos, theta, timestamp, ...)`	Builds `TransformStamped` from current position and heading quaternion.

visual_odom_launch.py

ROS 2 launch file. Reads `param.yaml` for the rosbag path and starts three processes simultaneously:

Process	Description
`visual_odom_node`	Main odometry node with `param.yaml` parameters

Process	Description
`ros2 bag play`	Rosbag playback with `--clock` and topic filter for RGB, depth, and `/tf_static`
`rviz2`	Visualization

1.2.2.2 Configuration (param.yaml)

All parameters are declared under the `visual_odom/ros__parameters` namespace.
| Parameter | Default | Description | |----------—|--------—|------------—| | `ransac.evaluation_tolerance` | Inlier distance threshold | | `ransac.iterations` | Maximum RANSAC loop count | | `ransac.sample_size` | Minimum point pairs per hypothesis | | `ransac.min_inlier_ratio` | Minimum inlier fraction to accept RANSAC result | | `ransac.min_matches_for_ransac` | Minimum matches required to start RANSAC | | `ransac.max_rotation_angle_deg` | Maximum plausible rotation per frame in degrees | | `rgb_depth↩_sync_tolerance_sec` | Max allowed RGB/depth timestamp delta | | `pixel_tolerance` | Pixel-space exclusion radius for keypoint deduplication | | `matches_for_new_landmarks` | Match count threshold below which new landmarks are added | | `min_landmark_trust` | Trust value below which a landmark is pruned | | `max_depth` | Maximum usable sensor depth | | `n_robot_samples` | Number of particles in the particle filter | | `rosbag_path` | Absolute path to the `.mcap` rosbag file |

1.2.3 3. Build and Run

1.2.3.1 Build

```
cd ~/ros2_ws
colcon build --packages-select visual_odom
source install/setup.bash
```

1.2.3.2 Launch (recommended)

```
ros2 launch visual_odom visual_odom_launch.py
```

Starts the node, rosbag playback, and RViz2 simultaneously.

- The rosbag path is read automatically from `param.yaml`.
- bagfiles are saved in SLAM/bagfiles
- The rviz2 config in /rviz will automatically be loaded

1.2.3.3 Node only (without launch file)

```
ros2 run visual_odom visual_odom_node --ros-args --params-file path/to/param.yaml
```

1.2.4 4. Documentation

The package uses Doxygen with Python docstring support.

1.2.4.1 Generate HTML docs

```
doxygen Doxyfile
xdg-open docs/html/index.html
```

1.2.4.2 Generate PDF from LaTeX output

```
cd docs/latex
make
xdg-open refman.pdf
```

Chapter 2

Namespace Index

2.1 Package List

Here are the packages with brief descriptions (if available):

metadata . . . 17
param . . . 20
setup . . . 22
visual_odom . . . 23
visual_odom.config
Central configuration file for the runtime parameters of the visual_odom_node . . . 23
visual_odom.constants
Global configuration, camera model, ROS topic definitions and helper datatypes used by the visual odometry system . . . 23
visual_odom.csv_creator_node
ROS 2 node that logs odometry and IMU measurements to a unified CSV file . . . 32
visual_odom.ekf_landmark
Extended Kalman Filter implementation for tracking individual landmark positions . . . 33
visual_odom.ekf_robot
Extended Kalman Filter for estimating robot pose using wheel odometry and RANSAC visual updates . . . 33
visual_odom.landmark
Defines the Landmark data class and the Kabsch optimal 2D rigid alignment algorithm . . . 33
visual_odom.odom_buffer . . . 34
visual_odom.robot . . . 34
visual_odom.tf_methods
Stateless coordinate frame transformation utilities . . . 34
visual_odom.visual_odom_map
Implements a persistent spatial landmark collection container tracking mapped environment features . . . 37
visual_odom.visual_odom_node
Main ROS 2 execution node managing the particle filter localization and visual odometry loop . . . 37
visual_odom_launch . . . 38

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

visual_odom.constants.Coordinate 39
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark 41
visual_odom.ekf_robot.ExtendedKalmanFilterRobot 47
visual_odom.landmark.Landmark 50
    visual_odom.visual_odom_map.VisualOdomMap 75
list
    visual_odom.visual_odom_map.VisualOdomMap 75
visual_odom.odom_buffer.odom_message 56
visual_odom.odom_buffer.OdomBuffer 57
visual_odom.constants.Parameters 62
visual_odom.constants.PixelCoordinate 63
visual_odom.constants.State 64
visual_odom.robot.VisualRobotSample 82
Node
    visual_odom.csv_creator_node.OdomImuCsvWriter 58
    visual_odom.visual_odom_node.VisualOdom 66

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

visual_odom.constants.Coordinate
: 3D spatial coordinate container . . . 39

visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark
: Extended Kalman Filter for tracking an individual 3D landmark in the global odom frame . . . 41

visual_odom.ekf_robot.ExtendedKalmanFilterRobot
: Extended Kalman Filter tracking the 2D planar robot base pose (x, y, theta) . . . 47

visual_odom.landmark.Landmark
: Landmark class for storing visual features and performing spatial estimation . . . 50

visual_odom.odom_buffer.odom_message . . . 56

visual_odom.odom_buffer.OdomBuffer . . . 57

visual_odom.csv_creator_node.OdomImuCsvWriter
: ROS 2 Node responsible for subscribing to localization data topics and logging state measurements into a unified CSV file for analysis . . . 58

visual_odom.constants.Parameters
: Runtime configuration and threshold parameter package for the SLAM node . . . 62

visual_odom.constants.PixelCoordinate
: Container for a 2D image pixel and its depth value . . . 63

visual_odom.constants.State
: 2D robot pose representation . . . 64

visual_odom.visual_odom_node.VisualOdom
: ROS 2 Visual Odometry node managing features extraction, RANSAC estimation, and particle weight selection . . . 66

visual_odom.visual_odom_map.VisualOdomMap
: Holds and manages persistent Landmark records mapped inside the particle trajectories . . . 75

visual_odom.robot.VisualRobotSample . . . 82

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

bagfiles/20260428_Bag_Without_TF/metadata.yaml . . . 91
bagfiles/20260519_TransXY_bag/metadata.yaml . . . 92
bagfiles/20260616_Around_The_Desks_bag/metadata.yaml . . . 92
ros2_ws/src/visual_odom/setup.py . . . 93
ros2_ws/src/visual_odom/config/param.yaml . . . 92
ros2_ws/src/visual_odom/launch/visual_odom_launch.py . . . 93
ros2_ws/src/visual_odom/visual_odom/__init__.py . . . 93
ros2_ws/src/visual_odom/visual_odom/constants.py . . . 93
ros2_ws/src/visual_odom/visual_odom/csv_creator_node.py . . . 96
ros2_ws/src/visual_odom/visual_odom/ekf_landmark.py . . . 97
ros2_ws/src/visual_odom/visual_odom/ekf_robot.py . . . 97
ros2_ws/src/visual_odom/visual_odom/landmark.py . . . 97
ros2_ws/src/visual_odom/visual_odom/odom_buffer.py . . . 97
ros2_ws/src/visual_odom/visual_odom/robot.py . . . 98
ros2_ws/src/visual_odom/visual_odom/tf_methods.py . . . 98
ros2_ws/src/visual_odom/visual_odom/visual_odom_map.py . . . 98
ros2_ws/src/visual_odom/visual_odom/visual_odom_node.py . . . 99

Chapter 6

Namespace Documentation

6.1 metadata Namespace Reference

Variables

- rosbag2_bagfile_information :
- int version : 9
- mcap storage_identifier
- duration :
- int nanoseconds : 86772902393
- starting_time :
- int nanoseconds_since_epoch : 1777312764407083438
- int message_count : 16425
- topics_with_message_count :
- name : /serf01/nav_rgbd_1/depth/image_raw
- sensor_msgs type /msg/Image
- cdr serialization_format
- offered_qos_profiles :
- int depth : 0
- reliable reliability
- volatile durability
- deadline :
- int sec : 9223372036
- int nsec : 854775807
- lifespan :
- automatic liveliness
- liveliness_lease_duration :
- false avoid_ros_namespace_conventions
- RIHS01_d31d41a9a4c4bc8eae9be757b0beed306564f7526c88ea6a4588fb9582527d47 type_description_hash
- str compression_format : ""
- str compression_mode : ""
- relative_file_paths :
- files :
- custom_data : ~
- jazzy ros_distro

6.1.1 Variable Documentation

6.1.1.1 avoid_ros_namespace_conventions

```
false metadata.avoid_ros_namespace_conventions
```

6.1.1.2 compression_format

```
str metadata.compression_format :  ""
```

6.1.1.3 compression_mode

```
str metadata.compression_mode :  ""
```

6.1.1.4 custom_data

```
metadata.custom_data :  ~
```

6.1.1.5 deadline

```
metadata.deadline :
```

6.1.1.6 depth

```
int metadata.depth :  0
```

6.1.1.7 durability

```
volatile metadata.durability
```

6.1.1.8 duration

```
metadata.duration :
```

6.1.1.9 files

```
metadata.files :
```

6.1.1.10 lifespan

```
metadata.lifespan :
```

6.1.1.11 liveliness

```
automatic metadata.liveliness
```

6.1.1.12 liveliness_lease_duration

```
metadata.liveliness_lease_duration :
```

6.1.1.13 message_count

```
int metadata.message_count :  16425
```

6.1.1.14 name

```
int metadata.name :  /serf01/nav_rgbd_1/depth/image_raw
```

6.1.1.15 nanoseconds

```
int metadata.nanoseconds :  86772902393
```

6.1.1.16 nanoseconds_since_epoch

```
int metadata.nanoseconds_since_epoch :  1777312764407083438
```

6.1.1.17 nsec

```
int metadata.nsec :  854775807
```

6.1.1.18 offered_qos_profiles

```
metadata.offered_qos_profiles :
```

6.1.1.19 relative_file_paths

```
metadata.relative_file_paths :
```

6.1.1.20 reliability

```
reliable metadata.reliability
```

6.1.1.21 ros_distro

```
jazzy metadata.ros_distro
```

6.1.1.22 rosbag2_bagfile_information

```
metadata.rosbag2_bagfile_information :
```

6.1.1.23 sec

```
int metadata.sec :  9223372036
```

6.1.1.24 serialization_format

```
cdr metadata.serialization_format
```

6.1.1.25 starting_time

```
metadata.starting_time :
```

6.1.1.26 storage_identifier

```
mcap metadata.storage_identifier
```

6.1.1.27 topics_with_message_count

```
metadata.topics_with_message_count :
```

6.1.1.28 type

```
rtabmap_msgs metadata.type /msg/Image
```

6.1.1.29 type_description_hash

```
RIHS01_968d5fe5899dd06767063ffe39db48a9661c87d756abaa8448dd6be859905d43 metadata.type_description↩
_hash
```

6.1.1.30 version

```
int metadata.version :  9
```

6.2 param Namespace Reference

Variables

- visual_odom_node :
- ros__parameters :
- int max_depth : 7000

  Maximum reliable sensor depth of the Kinect camera in millimeters (mm).
- float rgb_depth_sync_tolerance_sec : 0.1

  Allowed temporal deviation when synchronizing RGB and depth images (in seconds).
- int pixel_tolerance : 4

  Pixel tolerance range in image space for spatial deduplication of neighboring keypoints.
- int matches_for_new_landmarks : 500

  Match threshold below which new landmarks are added to the map.
- float min_landmark_trust : 15.0

  Minimum trust value a landmark must maintain before being removed from the map.
- int n_robot_samples : 40

  Number of active robot particle samples within the Rao-Blackwellized particle filter.
- ransac :
- float evaluation_tolerance : 0.05

  Allowed Euclidean distance deviation in meters (m) for classifying a point as an inlier.
- int iterations : 30

  Maximum number of iterations for hypothesis evaluation in the RANSAC algorithm.
- int sample_size : 3

  Minimum sample size (point pairs) per iteration for computing a transformation hypothesis.
- int min_matches_for_ransac : 10

  Minimum number of feature matches required before starting RANSAC optimization.
- float max_rotation_angle_deg : 15.0

  Maximum physically plausible robot rotation per frame in degrees (deg).
- float min_inlier_ratio : 0.2

  Minimum required ratio of inliers to total matches for accepting a RANSAC result.
- str rosbag_path : "bagfiles/20260616_Around_The_Desks_bag/rosbag2_2026_06_16-13_12_13_0.mcap"

  Absolute path to the MCAP rosbag file to be recorded or played back.

6.2.1 Variable Documentation

6.2.1.1 evaluation_tolerance

```
float param.evaluation_tolerance :  0.05
```

Allowed Euclidean distance deviation in meters (m) for classifying a point as an inlier.

6.2.1.2 iterations

```
int param.iterations :  30
```

Maximum number of iterations for hypothesis evaluation in the RANSAC algorithm.

6.2.1.3 matches_for_new_landmarks

```
int param.matches_for_new_landmarks :  500
```

Match threshold below which new landmarks are added to the map.

6.2.1.4 max_depth

```
int param.max_depth :  7000
```

Maximum reliable sensor depth of the Kinect camera in millimeters (mm).

6.2.1.5 max_rotation_angle_deg

```
float param.max_rotation_angle_deg :  15.0
```
Maximum physically plausible robot rotation per frame in degrees (deg).

6.2.1.6 min_inlier_ratio

```
float param.min_inlier_ratio :  0.2
```
Minimum required ratio of inliers to total matches for accepting a RANSAC result.

6.2.1.7 min_landmark_trust

```
float param.min_landmark_trust :  15.0
```
Minimum trust value a landmark must maintain before being removed from the map.

6.2.1.8 min_matches_for_ransac

```
int param.min_matches_for_ransac :  10
```
Minimum number of feature matches required before starting RANSAC optimization.

6.2.1.9 n_robot_samples

```
int param.n_robot_samples :  40
```
Number of active robot particle samples within the Rao-Blackwellized particle filter.

6.2.1.10 pixel_tolerance

```
int param.pixel_tolerance :  4
```
Pixel tolerance range in image space for spatial deduplication of neighboring keypoints.

6.2.1.11 ransac

```
param.ransac :
```

6.2.1.12 rgb_depth_sync_tolerance_sec

```
float param.rgb_depth_sync_tolerance_sec :  0.1
```
Allowed temporal deviation when synchronizing RGB and depth images (in seconds).

6.2.1.13 ros__parameters

```
param.ros__parameters :
```

6.2.1.14 rosbag_path

```
str param.rosbag_path :  "bagfiles/20260616_Around_The_Desks_bag/rosbag2_2026_06_16-13_12_13↩
_0.mcap"
```
Absolute path to the MCAP rosbag file to be recorded or played back.
The path for the "Around The Desks" scenario is currently active.

6.2.1.15 sample_size

```
int param.sample_size :  3
```
Minimum sample size (point pairs) per iteration for computing a transformation hypothesis.

6.2.1.16 visual_odom_node

```
param.visual_odom_node :
```

6.3 setup Namespace Reference

Variables

- str package_name = 'visual_odom'
- name
- version
- packages
- data_files
- install_requires
- zip_safe
- maintainer
- maintainer_email
- description
- license
- extras_require
- entry_points

6.3.1 Variable Documentation

6.3.1.1 data_files

```
setup.data_files
```

6.3.1.2 description

```
setup.description
```

6.3.1.3 entry_points

```
setup.entry_points
```

6.3.1.4 extras_require

```
setup.extras_require
```

6.3.1.5 install_requires

```
setup.install_requires
```

6.3.1.6 license

```
setup.license
```

6.3.1.7 maintainer

```
setup.maintainer
```

6.3.1.8 maintainer_email

```
setup.maintainer_email
```

6.3.1.9 name

```
setup.name
```

6.3.1.10 package_name

```
str setup.package_name = 'visual_odom'
```

6.3.1.11 packages

```
setup.packages
```

6.3.1.12 version

```
setup.version
```

6.3.1.13 zip_safe

```
setup.zip_safe
```

6.4 visual_odom Namespace Reference

Namespaces

- namespace config

  Central configuration file for the runtime parameters of the visual_odom_node.
- namespace constants

  Global configuration, camera model, ROS topic definitions and helper datatypes used by the visual odometry system.
- namespace csv_creator_node

  ROS 2 node that logs odometry and IMU measurements to a unified CSV file.
- namespace ekf_landmark

  Extended Kalman Filter implementation for tracking individual landmark positions.
- namespace ekf_robot

  Extended Kalman Filter for estimating robot pose using wheel odometry and RANSAC visual updates.
- namespace landmark

  Defines the Landmark data class and the Kabsch optimal 2D rigid alignment algorithm.
- namespace odom_buffer
- namespace robot
- namespace tf_methods

  Stateless coordinate frame transformation utilities.
- namespace visual_odom_map

  Implements a persistent spatial landmark collection container tracking mapped environment features.
- namespace visual_odom_node

  Main ROS 2 execution node managing the particle filter localization and visual odometry loop.

6.5 visual_odom.config Namespace Reference

Central configuration file for the runtime parameters of the visual_odom_node.

6.5.1 Detailed Description

Central configuration file for the runtime parameters of the visual_odom_node.
This YAML file defines all thresholds for depth filtering, RANSAC optimization, particle count of the localization filter, and the path to the rosbag that should be played back.

6.6 visual_odom.constants Namespace Reference

Global configuration, camera model, ROS topic definitions and helper datatypes used by the visual odometry system.

Classes

- class Coordinate

  3D spatial coordinate container.
- class Parameters

  Runtime configuration and threshold parameter package for the SLAM node.
- class PixelCoordinate

  Container for a 2D image pixel and its depth value.
- class State

  2D robot pose representation.

Functions

- float normalize_angle (float angle)

  Normalize an angle to the range [-pi, pi].

Variables

- str VISUAL_ODOM_MSG_TOPIC = "/serf01/odometry/project_slam"

  Primary ROS 2 publisher output topic for navigation odometry.
- str RGB_IMAGE_TOPIC = "/serf01/nav_rgbd_1/rgb/image_raw"

  ROS 2 topic for raw RGB image input.
- str DEPTH_IMAGE_TOPIC = "/serf01/nav_rgbd_1/depth/image_raw"

  ROS 2 topic for raw depth image input.
- str WHEEL_ODOMETRY_TOPIC = "/serf01/odometry/wheel"

  ROS 2 topic for wheel odometry input.
- str FILTERED_ODOMETRY_TOPIC = "/serf01/odometry/filtered"

  ROS 2 topic for filtered odometry.
- str IMU_ODOMETRY_TOPIC = "/serf01/odometry/imu"

  ROS 2 topic for IMU-based odometry input.
- str TF_STATIC_TOPIC = "/tf_static"

  ROS 2 topic for static TF transforms.
- str VISION_CONE_TOPIC = "/serf01/camera/vision_cone"

  ROS 2 topic for publishing or visualizing the camera vision cone.
- str KP_IMAGE_TOPIC = "/serf01/camera/keypoint_image"

  ROS 2 topic for publishing images with visualized keypoints.
- str VISUAL_ODOM_PATH_TOPIC = "/serf01/visual_odom_path"

  ROS 2 topic for publishing paths of the particles.
- str POINTCLOUD_FRAME_TOPIC = "/serf01/pointcloud/points_3d"

  Frame/topic identifier for the structural environment point cloud.
- str ACCUMULATED_POINTCLOUD_FRAME_TOPIC = "/serf01/pointcloud/accumulated_points_3d"

  Frame/topic identifier for the accumulated environment point cloud.
- str KEYPOINT_POINTCLOUD_FRAME_TOPIC = "/serf01/pointcloud/keypoint_3d"

  Frame/topic identifier for tracked ORB descriptor 3D keypoints.
- str KINECT_FRAME_ID = "kinect_depth"

  TF frame identifier for the optical center of the RGB-D camera.
- str BASE_LINK_FRAME_ID = "base_link"

  TF frame identifier for the physical center of the robot platform.
- str VISUAL_ODOM_FRAME_ID = "odom"

  TF frame identifier for the fixed global odometry origin.
- float F = 526.61

  Horizontal and vertical focal length parameter of the Kinect optical model.

- float CU = 318.525

  Horizontal pixel coordinate of the camera principal point.
- float CV = 241.181

  Vertical pixel coordinate of the camera principal point.
- int MAX_U = 640

  Horizontal image plane resolution in pixels.
- int MAX_V = 480

  Vertical image plane resolution in pixels.
- MAX_AZIMUTH = abs(atan2(CU, F))

  Derived horizontal angle limit matching the sensor field of view.
- MAX_ALTITUDE = abs(atan2(CV, F))

  Derived vertical angle limit matching the sensor field of view.
- CAMERA_POS_IN_BASELINK = Coordinate(0.1, 0.0, 0.75)

  Fixed physical camera offset relative to base_link.
- int CAMERA_ANGLE_VER_RAD = 43 ∗ pi / 180

  Vertical field of view angle of the RGB-D camera in radians.
- int CAMERA_ANGLE_HOR_RAD = 57 ∗ pi / 180

  Horizontal field of view angle of the RGB-D camera in radians.
- int RESOLUTION_CONE_H = 40

  Horizontal discretization resolution of the camera vision cone.
- int RESOLUTION_CONE_V = 30

  Vertical discretization resolution of the camera vision cone.
- ROT_BK

  Rotation matrix from Kinect optical frame to ROS base_link frame.
- int MIN_DEPTH = 400

  Minimum reliable depth value in millimeters.
- int MAX_DEPTH = 7000

  Maximum reliable depth value in millimeters.
- float RANSAC_EVALUATION_TOLERANCE = 0.045

  Inlier distance threshold for evaluating RANSAC hypotheses in meters.
- int RANSAC_ITERATION = 100

  Maximum number of RANSAC iterations.
- int RANSAC_SAMPLE_SIZE = 3

  Number of matched point pairs used for one RANSAC hypothesis.
- float RANSAC_MIN_INLIER_RATIO = 0.3

  Minimum inlier ratio required to accept a RANSAC solution.
- int MIN_MATCHES_FOR_RANSAC = 15

  Minimum number of feature matches required to start RANSAC.
- float MAX_ROTATION_ANGLE_DEG = 15.0

  Maximum plausible robot rotation per frame step in degrees.
- float RGB_DEPTH_SYNC_TOLERANCE_SEC = 0.05

  Maximum allowed temporal difference between RGB and depth frames in seconds.
- int PIXEL_TOLERANCE = 8

  Pixel-space search radius used for visual match validation.
- int N_ROBOT_SAMPLES = 1000

  Default number of particles/samples used for localization.
- float NOISE_INCREMENT_RANSAC_FAILURE = 0.002

  Noise increment added after a failed RANSAC estimation.
- float MAX_ADDITIONAL_NOISE_RANSAC_FAILURE = 0.22

  Maximum allowed additional noise caused by repeated RANSAC failures.
- int MATCHES_FOR_NEW_LANDMARKS = 50

Minimum tracking count required to stabilize a new landmark.

- int INITIAL_LANDMARK_COUNT = 260

  Initial landmark count threshold used for map bootstrapping.
- float MIN_LANDMARK_TRUST = 20.0

  Minimum trust value before a landmark is considered unreliable.
- float INCREASE_TRUST_VALUE = 10.0

  Trust value added to a landmark after successful observation.
- float DECREASE_TRUST_FACTOR = 0.95

  Multiplicative trust decay factor for unobserved landmarks.
- float SIGMA_X_ODOM_WHEEL_Q = 0.05

  Wheel odometry process noise standard deviation in x-direction.
- float SIGMA_Y_ODOM_WHEEL_Q = 0.05

  Wheel odometry process noise standard deviation in y-direction.
- float SIGMA_THETA_ODOM_WHEEL_Q = 0.03

  Wheel odometry process noise standard deviation for heading angle.
- float SIGMA_PIXEL = 0.8 / 3

  Pixel projection noise coefficient for image-plane coordinates.
- float ERROR_MIN_DEPTH = 0.001477

  Base depth measurement noise offset at closest sensor range in meters.
- float ERROR_QUADRATIC_DEPTH = 0.002294

  Quadratic depth error factor modeling RGB-D precision degradation.
- int MIN_DET_VALUE = 1e-12

  Small epsilon cutoff used to avoid determinant singularities.
- float GAUSS_NOISE_X_SIGMA = 0.006

  Gaussian noise x-coordinate in meters.
- float GAUSS_NOISE_Y_SIGMA = 0.006

  Gaussian noise y-coordinate in meters.
- float GAUSS_NOISE_THETA_SIGMA = 0.003

  Gaussian noise heading angle in radians.
- float NOISE_TRANSLATION_THRESHOLD = 0.10

  Threshold for accumulated translation noise before adding random perturbation to the robot state in meters.
- float NOISE_ROTATION_THRESHOLD = 0.06

  Threshold for accumulated rotation noise before adding random perturbation to the robot state in radians.
- float RESAMPLE_THETA_TOLERANCE = 4.0

  Tolerance for resampling based on heading angle difference in degrees.
- float RESAMPLE_TIME_TOLERANCE = 40.0

  Time tolerance for resampling based on elapsed time since the first frame in seconds.
- float RESAMPLE_POS_TOLERANCE = 0.35

  Position tolerance for resampling based on Euclidean distance in meters.
- Parameters parameters = None

  Global runtime parameter instance populated at node startup.
- float PERCENTAGE_OF_PLAY_SPEED = 0.1

6.6.1 Detailed Description

Global configuration, camera model, ROS topic definitions and helper datatypes used by the visual odometry system.
This module centralizes all runtime constants, camera calibration parameters, ROS topic names, TF frame identifiers and helper dataclasses required by the visual odometry pipeline.

6.6.2 Function Documentation

6.6.2.1 normalize_angle()

```
float visual_odom.constants.normalize_angle (
            float angle )
```

Normalize an angle to the range [-pi, pi].
Normalizes a given angle into the standard bounding range of [-pi, pi].

Parameters

angle	Input angle in radians.

Returns

Normalized angle in radians.

6.6.3 Variable Documentation

6.6.3.1 ACCUMULATED_POINTCLOUD_FRAME_TOPIC

```
str visual_odom.constants.ACCUMULATED_POINTCLOUD_FRAME_TOPIC = "/serf01/pointcloud/accumulated↩
_points_3d"
```

Frame/topic identifier for the accumulated environment point cloud.

6.6.3.2 BASE_LINK_FRAME_ID

```
str visual_odom.constants.BASE_LINK_FRAME_ID = "base_link"
```

TF frame identifier for the physical center of the robot platform.

6.6.3.3 CAMERA_ANGLE_HOR_RAD

```
int visual_odom.constants.CAMERA_ANGLE_HOR_RAD = 57 * pi / 180
```

Horizontal field of view angle of the RGB-D camera in radians.

6.6.3.4 CAMERA_ANGLE_VER_RAD

```
int visual_odom.constants.CAMERA_ANGLE_VER_RAD = 43 * pi / 180
```

Vertical field of view angle of the RGB-D camera in radians.

6.6.3.5 CAMERA_POS_IN_BASELINK

```
visual_odom.constants.CAMERA_POS_IN_BASELINK = Coordinate(0.1, 0.0, 0.75)
```

Fixed physical camera offset relative to base_link.

6.6.3.6 CU

```
float visual_odom.constants.CU = 318.525
```

Horizontal pixel coordinate of the camera principal point.

6.6.3.7 CV

```
float visual_odom.constants.CV = 241.181
```

Vertical pixel coordinate of the camera principal point.

6.6.3.8 DECREASE_TRUST_FACTOR

```
float visual_odom.constants.DECREASE_TRUST_FACTOR = 0.95
```

Multiplicative trust decay factor for unobserved landmarks.

6.6.3.9 DEPTH_IMAGE_TOPIC

```
str visual_odom.constants.DEPTH_IMAGE_TOPIC = "/serf01/nav_rgbd_1/depth/image_raw"
```
ROS 2 topic for raw depth image input.

6.6.3.10 ERROR_MIN_DEPTH

```
float visual_odom.constants.ERROR_MIN_DEPTH = 0.001477
```
Base depth measurement noise offset at closest sensor range in meters.

6.6.3.11 ERROR_QUADRATIC_DEPTH

```
float visual_odom.constants.ERROR_QUADRATIC_DEPTH = 0.002294
```
Quadratic depth error factor modeling RGB-D precision degradation.

6.6.3.12 F

```
float visual_odom.constants.F = 526.61
```
Horizontal and vertical focal length parameter of the Kinect optical model.

6.6.3.13 FILTERED_ODOMETRY_TOPIC

```
str visual_odom.constants.FILTERED_ODOMETRY_TOPIC = "/serf01/odometry/filtered"
```
ROS 2 topic for filtered odometry.

6.6.3.14 GAUSS_NOISE_THETA_SIGMA

```
float visual_odom.constants.GAUSS_NOISE_THETA_SIGMA = 0.003
```
Gaussian noise heading angle in radians.

6.6.3.15 GAUSS_NOISE_X_SIGMA

```
float visual_odom.constants.GAUSS_NOISE_X_SIGMA = 0.006
```
Gaussian noise x-coordinate in meters.

6.6.3.16 GAUSS_NOISE_Y_SIGMA

```
float visual_odom.constants.GAUSS_NOISE_Y_SIGMA = 0.006
```
Gaussian noise y-coordinate in meters.

6.6.3.17 IMU_ODOMETRY_TOPIC

```
str visual_odom.constants.IMU_ODOMETRY_TOPIC = "/serf01/odometry/imu"
```
ROS 2 topic for IMU-based odometry input.

6.6.3.18 INCREASE_TRUST_VALUE

```
float visual_odom.constants.INCREASE_TRUST_VALUE = 10.0
```
Trust value added to a landmark after successful observation.

6.6.3.19 INITIAL_LANDMARK_COUNT

```
int visual_odom.constants.INITIAL_LANDMARK_COUNT = 260
```
Initial landmark count threshold used for map bootstrapping.

6.6.3.20 KEYPOINT_POINTCLOUD_FRAME_TOPIC

```
str visual_odom.constants.KEYPOINT_POINTCLOUD_FRAME_TOPIC = "/serf01/pointcloud/keypoint_3d"
```
Frame/topic identifier for tracked ORB descriptor 3D keypoints.

6.6.3.21 KINECT_FRAME_ID

```
str visual_odom.constants.KINECT_FRAME_ID = "kinect_depth"
```
TF frame identifier for the optical center of the RGB-D camera.

6.6.3.22 KP_IMAGE_TOPIC

```
str visual_odom.constants.KP_IMAGE_TOPIC = "/serf01/camera/keypoint_image"
```
ROS 2 topic for publishing images with visualized keypoints.

6.6.3.23 MATCHES_FOR_NEW_LANDMARKS

```
int visual_odom.constants.MATCHES_FOR_NEW_LANDMARKS = 50
```
Minimum tracking count required to stabilize a new landmark.

6.6.3.24 MAX_ADDITIONAL_NOISE_RANSAC_FAILURE

```
float visual_odom.constants.MAX_ADDITIONAL_NOISE_RANSAC_FAILURE = 0.22
```
Maximum allowed additional noise caused by repeated RANSAC failures.

6.6.3.25 MAX_ALTITUDE

```
visual_odom.constants.MAX_ALTITUDE = abs(atan2(CV, F))
```
Derived vertical angle limit matching the sensor field of view.

6.6.3.26 MAX_AZIMUTH

```
visual_odom.constants.MAX_AZIMUTH = abs(atan2(CU, F))
```
Derived horizontal angle limit matching the sensor field of view.

6.6.3.27 MAX_DEPTH

```
int visual_odom.constants.MAX_DEPTH = 7000
```
Maximum reliable depth value in millimeters.

6.6.3.28 MAX_ROTATION_ANGLE_DEG

```
float visual_odom.constants.MAX_ROTATION_ANGLE_DEG = 15.0
```
Maximum plausible robot rotation per frame step in degrees.

6.6.3.29 MAX_U

```
int visual_odom.constants.MAX_U = 640
```
Horizontal image plane resolution in pixels.

6.6.3.30 MAX_V

```
int visual_odom.constants.MAX_V = 480
```
Vertical image plane resolution in pixels.

6.6.3.31 MIN_DEPTH

```
int visual_odom.constants.MIN_DEPTH = 400
```
Minimum reliable depth value in millimeters.

6.6.3.32 MIN_DET_VALUE

```
int visual_odom.constants.MIN_DET_VALUE = 1e-12
```
Small epsilon cutoff used to avoid determinant singularities.

6.6.3.33 MIN_LANDMARK_TRUST

```
float visual_odom.constants.MIN_LANDMARK_TRUST = 20.0
```
Minimum trust value before a landmark is considered unreliable.

6.6.3.34 MIN_MATCHES_FOR_RANSAC

```
int visual_odom.constants.MIN_MATCHES_FOR_RANSAC = 15
```
Minimum number of feature matches required to start RANSAC.

6.6.3.35 N_ROBOT_SAMPLES

```
int visual_odom.constants.N_ROBOT_SAMPLES = 1000
```
Default number of particles/samples used for localization.

6.6.3.36 NOISE_INCREMENT_RANSAC_FAILURE

```
float visual_odom.constants.NOISE_INCREMENT_RANSAC_FAILURE = 0.002
```
Noise increment added after a failed RANSAC estimation.

6.6.3.37 NOISE_ROTATION_THRESHOLD

```
float visual_odom.constants.NOISE_ROTATION_THRESHOLD = 0.06
```
Threshold for accumulated rotation noise before adding random perturbation to the robot state in radians.

6.6.3.38 NOISE_TRANSLATION_THRESHOLD

```
float visual_odom.constants.NOISE_TRANSLATION_THRESHOLD = 0.10
```
Threshold for accumulated translation noise before adding random perturbation to the robot state in meters.

6.6.3.39 parameters

```
Parameters visual_odom.constants.parameters = None
```
Global runtime parameter instance populated at node startup.

6.6.3.40 PERCENTAGE_OF_PLAY_SPEED

```
float visual_odom.constants.PERCENTAGE_OF_PLAY_SPEED = 0.1
```

6.6.3.41 PIXEL_TOLERANCE

```
int visual_odom.constants.PIXEL_TOLERANCE = 8
```
Pixel-space search radius used for visual match validation.

6.6.3.42 POINTCLOUD_FRAME_TOPIC

```
str visual_odom.constants.POINTCLOUD_FRAME_TOPIC = "/serf01/pointcloud/points_3d"
```
Frame/topic identifier for the structural environment point cloud.

6.6.3.43 RANSAC_EVALUATION_TOLERANCE

```
float visual_odom.constants.RANSAC_EVALUATION_TOLERANCE = 0.045
```
Inlier distance threshold for evaluating RANSAC hypotheses in meters.

6.6.3.44 RANSAC_ITERATION

```
int visual_odom.constants.RANSAC_ITERATION = 100
```
Maximum number of RANSAC iterations.

6.6.3.45 RANSAC_MIN_INLIER_RATIO

```
float visual_odom.constants.RANSAC_MIN_INLIER_RATIO = 0.3
```

Minimum inlier ratio required to accept a RANSAC solution.

6.6.3.46 RANSAC_SAMPLE_SIZE

```
int visual_odom.constants.RANSAC_SAMPLE_SIZE = 3
```

Number of matched point pairs used for one RANSAC hypothesis.

6.6.3.47 RESAMPLE_POS_TOLERANCE

```
float visual_odom.constants.RESAMPLE_POS_TOLERANCE = 0.35
```

Position tolerance for resampling based on Euclidean distance in meters.

6.6.3.48 RESAMPLE_THETA_TOLERANCE

```
float visual_odom.constants.RESAMPLE_THETA_TOLERANCE = 4.0
```

Tolerance for resampling based on heading angle difference in degrees.

6.6.3.49 RESAMPLE_TIME_TOLERANCE

```
float visual_odom.constants.RESAMPLE_TIME_TOLERANCE = 40.0
```

Time tolerance for resampling based on elapsed time since the first frame in seconds.

6.6.3.50 RESOLUTION_CONE_H

```
int visual_odom.constants.RESOLUTION_CONE_H = 40
```

Horizontal discretization resolution of the camera vision cone.

6.6.3.51 RESOLUTION_CONE_V

```
int visual_odom.constants.RESOLUTION_CONE_V = 30
```

Vertical discretization resolution of the camera vision cone.

6.6.3.52 RGB_DEPTH_SYNC_TOLERANCE_SEC

```
float visual_odom.constants.RGB_DEPTH_SYNC_TOLERANCE_SEC = 0.05
```

Maximum allowed temporal difference between RGB and depth frames in seconds.

6.6.3.53 RGB_IMAGE_TOPIC

```
str visual_odom.constants.RGB_IMAGE_TOPIC = "/serf01/nav_rgbd_1/rgb/image_raw"
```

ROS 2 topic for raw RGB image input.

6.6.3.54 ROT_BK

```
visual_odom.constants.ROT_BK
```

Initial value:

```
00001 =  np.array([[0.0, 0.0, 1.0],
00002                   [-1.0, 0.0, 0.0],
00003                   [0.0, -1.0, 0.0]], dtype=np.float64)
```

Rotation matrix from Kinect optical frame to ROS base_link frame.
Maps Kinect optical frame conventions to the standard ROS base_link convention. The optical Z-axis is mapped to the base_link X-axis, and the optical X-axis is mapped to the negative base_link Y-axis.

6.6.3.55 SIGMA_PIXEL

```
float visual_odom.constants.SIGMA_PIXEL = 0.8 / 3
```

Pixel projection noise coefficient for image-plane coordinates.

6.6.3.56 SIGMA_THETA_ODOM_WHEEL_Q

```
float visual_odom.constants.SIGMA_THETA_ODOM_WHEEL_Q = 0.03
```

Wheel odometry process noise standard deviation for heading angle.

6.6.3.57 SIGMA_X_ODOM_WHEEL_Q

```
float visual_odom.constants.SIGMA_X_ODOM_WHEEL_Q = 0.05
```

Wheel odometry process noise standard deviation in x-direction.

6.6.3.58 SIGMA_Y_ODOM_WHEEL_Q

```
float visual_odom.constants.SIGMA_Y_ODOM_WHEEL_Q = 0.05
```

Wheel odometry process noise standard deviation in y-direction.

6.6.3.59 TF_STATIC_TOPIC

```
str visual_odom.constants.TF_STATIC_TOPIC = "/tf_static"
```

ROS 2 topic for static TF transforms.

6.6.3.60 VISION_CONE_TOPIC

```
str visual_odom.constants.VISION_CONE_TOPIC = "/serf01/camera/vision_cone"
```

ROS 2 topic for publishing or visualizing the camera vision cone.

6.6.3.61 VISUAL_ODOM_FRAME_ID

```
str visual_odom.constants.VISUAL_ODOM_FRAME_ID = "odom"
```

TF frame identifier for the fixed global odometry origin.

6.6.3.62 VISUAL_ODOM_MSG_TOPIC

```
str visual_odom.constants.VISUAL_ODOM_MSG_TOPIC = "/serf01/odometry/project_slam"
```

Primary ROS 2 publisher output topic for navigation odometry.

6.6.3.63 VISUAL_ODOM_PATH_TOPIC

```
str visual_odom.constants.VISUAL_ODOM_PATH_TOPIC = "/serf01/visual_odom_path"
```

ROS 2 topic for publishing paths of the particles.

6.6.3.64 WHEEL_ODOMETRY_TOPIC

```
str visual_odom.constants.WHEEL_ODOMETRY_TOPIC = "/serf01/odometry/wheel"
```

ROS 2 topic for wheel odometry input.

6.7 visual_odom.csv_creator_node Namespace Reference

ROS 2 node that logs odometry and IMU measurements to a unified CSV file.

Classes

- class OdomImuCsvWriter

  ROS 2 Node responsible for subscribing to localization data topics and logging state measurements into a unified CSV file for analysis.

Functions

- main ()

  Starts the infrastructure runtime framework layer, instantiates OdomImuCsvWriter execution loops, and ensures safe process cleanups.

6.7.1 Detailed Description

ROS 2 node that logs odometry and IMU measurements to a unified CSV file.

6.7.2 Function Documentation

6.7.2.1 main()

`visual_odom.csv_creator_node.main ( )`
Starts the infrastructure runtime framework layer, instantiates OdomImuCsvWriter execution loops, and ensures safe process cleanups.

6.8 visual_odom.ekf_landmark Namespace Reference

Extended Kalman Filter implementation for tracking individual landmark positions.

Classes

- class ExtendedKalmanFilterLandmark

  Extended Kalman Filter for tracking an individual 3D landmark in the global odom frame.

6.8.1 Detailed Description

Extended Kalman Filter implementation for tracking individual landmark positions.

6.9 visual_odom.ekf_robot Namespace Reference

Extended Kalman Filter for estimating robot pose using wheel odometry and RANSAC visual updates.

Classes

- class ExtendedKalmanFilterRobot

  Extended Kalman Filter tracking the 2D planar robot base pose (x, y, theta).

6.9.1 Detailed Description

Extended Kalman Filter for estimating robot pose using wheel odometry and RANSAC visual updates.

6.10 visual_odom.landmark Namespace Reference

Defines the Landmark data class and the Kabsch optimal 2D rigid alignment algorithm.

Classes

- class Landmark

  Landmark class for storing visual features and performing spatial estimation.

Functions

- Tuple[np.ndarray, np.ndarray, float] kabsch (np.ndarray P_i, np.ndarray Q_i, float max_rotation_angle_↩ deg=15.0)

  Computes the optimal 2D rotation and translation to align point set Q onto set P.

6.10.1 Detailed Description

Defines the Landmark data class and the Kabsch optimal 2D rigid alignment algorithm.

6.10.2 Function Documentation

6.10.2.1 kabsch()

```
Tuple[np.ndarray, np.ndarray, float] visual_odom.landmark.kabsch (
            np.ndarray P_i,
            np.ndarray Q_i,
            float  max_rotation_angle_deg = 15.0 )
```

Computes the optimal 2D rotation and translation to align point set Q onto set P.
Implements a 2D closed-form Kabsch algorithm to find least-squares alignment parameters.

Parameters

P_i	2D point coordinate array in the reference coordinate frame.
Q_i	2D point coordinate array in the target coordinate frame.
max_rotation_angle_deg	Plausibility constraint capping estimated rotation.

Returns

A tuple containing:

- R: Optimal 2x2 rotation matrix, or None if degenerate.
- t: Optimal 2D translation offset vector, or None if degenerate.
- theta: Rotational heading orientation change in radians, or None if degenerate.

6.11 visual_odom.odom_buffer Namespace Reference

Classes

- class odom_message
- class OdomBuffer

6.12 visual_odom.robot Namespace Reference

Classes

- class VisualRobotSample

6.13 visual_odom.tf_methods Namespace Reference

Stateless coordinate frame transformation utilities.

Functions

- Coordinate kinect_depth_to_odom (Coordinate kinect_point, float theta, Coordinate pos_baselink)

  Transform a point from the Kinect frame into the global odom frame using the LATEST available robot state.
- Coordinate kinect_depth_to_baselink (Coordinate kinect_point)

  Transform a point from the Kinect optical frame into the robot base_link frame using static calibrations.
- Coordinate odom_to_baselink (Coordinate odom_point, float theta, Coordinate pos_baselink)

  Transform a point from the global odom frame back into the local base_link frame.
- Coordinate pixel_to_kinect (PixelCoordinate kp)

  Calculate 3D coordinates from pixel indices and its registered depth values.
- NDArray pixel_to_kinect_ (int u, int v, float z)

  Calculate 3D coordinates from raw pixel indices and depth values directly to a NumPy representation.
- TransformStamped calculate_tf (Coordinate Position, float theta, Time timestamp, str parent_frame_id, str child_frame_id)

  Formats a ROS 2 coordinate transform stamped message.

6.13.1 Detailed Description

Stateless coordinate frame transformation utilities.

6.13.2 Function Documentation

6.13.2.1 calculate_tf()

```
TransformStamped visual_odom.tf_methods.calculate_tf (
            Coordinate Position,
            float theta,
            Time timestamp,
            str parent_frame_id,
            str child_frame_id )
```

Formats a ROS 2 coordinate transform stamped message.

Parameters

Position	Position translation Coordinate offset.
theta	2D planar rotation heading yaw in radians.
timestamp	Synchronized timeline timestamp.
parent_frame_id	Frame ID of the static coordinate origin ("odom").
child_frame_id	Link ID of the robot footprint frame base ("base_link").

Returns

Formatted transform frame ready for tf_broadcaster.

6.13.2.2 kinect_depth_to_baselink()

```
Coordinate visual_odom.tf_methods.kinect_depth_to_baselink (
            Coordinate kinect_point )
```

Transform a point from the Kinect optical frame into the robot base_link frame using static calibrations.
Applies the static calibration matrix and the mounting offset translations relative to the base center.

Parameters

kinect_point	Coordinate object in the kinect_depth optical frame.

Returns

Coordinate object transformed into the local robot base_link frame.

6.13.2.3 kinect_depth_to_odom()

```
Coordinate visual_odom.tf_methods.kinect_depth_to_odom (
            Coordinate kinect_point,
            float theta,
            Coordinate pos_baselink )
```

Transform a point from the Kinect frame into the global odom frame using the LATEST available robot state.
Ignores historical timestamps and uses the most recent transformation matrices to avoid extrapolation errors.

Parameters

kinect_point	Coordinate object in the kinect_depth frame.
theta	Current yaw heading orientation angle of the robot base in radians.
pos_baselink	Global tracking position coordinate of the robot base link center.

Returns

Coordinate object mapped completely into the global odom map frame.

6.13.2.4 odom_to_baselink()

```
Coordinate visual_odom.tf_methods.odom_to_baselink (
            Coordinate odom_point,
            float theta,
            Coordinate pos_baselink )
```

Transform a point from the global odom frame back into the local base_link frame.
Applies the inverse heading rotation and structural translations relative to the current robot base pose.

Parameters

odom_point	Coordinate object in the global odom map frame.
theta	Current heading orientation angle of the robot base in radians.
pos_baselink	Global tracking position coordinate of the robot base link center.

Returns

Coordinate object referenced in the local robot base_link frame.

6.13.2.5 pixel_to_kinect()

```
Coordinate visual_odom.tf_methods.pixel_to_kinect (
            PixelCoordinate kp )
```

Calculate 3D coordinates from pixel indices and its registered depth values.
Applies the pinhole model and scales the output values from millimeters to meters.

Parameters

kp	PixelCoordinate object containing pixel indices (u, v) and depth z (in millimeters).

Returns

Coordinate object in the Kinect optical frame (meters).

6.13.2.6 pixel_to_kinect_()

```
NDArray visual_odom.tf_methods.pixel_to_kinect_ (
            int u,
            int v,
            float z )
```

Calculate 3D coordinates from raw pixel indices and depth values directly to a NumPy representation.
Applies the pinhole projection equations and returns a standardized numpy vector array scaled in meters.

Parameters

u	Horizontal pixel column index in the image plane.
v	Vertical pixel row index in the image plane.
z	Raw depth value from the sensor (in millimeters).

Returns

3D NumPy array vector [x, y, z] in the Kinect frame (meters).

6.14 visual_odom.visual_odom_map Namespace Reference

Implements a persistent spatial landmark collection container tracking mapped environment features.

Classes

- class VisualOdomMap

  Holds and manages persistent Landmark records mapped inside the particle trajectories.

6.14.1 Detailed Description

Implements a persistent spatial landmark collection container tracking mapped environment features.

6.15 visual_odom.visual_odom_node Namespace Reference

Main ROS 2 execution node managing the particle filter localization and visual odometry loop.

Classes

- class VisualOdom

  ROS 2 Visual Odometry node managing features extraction, RANSAC estimation, and particle weight selection.

Functions

- create_particle_path_markers (self, List[VisualRobotSample] particles, int best_particle_idx)

  Generates visualization markers for particle trajectories.
- main ()

  Core node runtime initialization routine.

6.15.1 Detailed Description

Main ROS 2 execution node managing the particle filter localization and visual odometry loop.

6.15.2 Function Documentation

6.15.2.1 create_particle_path_markers()

```
visual_odom.visual_odom_node.create_particle_path_markers (
              self,
             List[VisualRobotSample] particles,
             int best_particle_idx )
```

Generates visualization markers for particle trajectories.

Parameters

particles	Active particle set.
best_particle_idx	Best particle index.

Returns

MarkerArray for RViz visualization.

6.15.2.2 main()

```
visual_odom.visual_odom_node.main ( )
```

Core node runtime initialization routine.

6.16 visual_odom_launch Namespace Reference

Functions

- generate_launch_description ()

6.16.1 Function Documentation

6.16.1.1 generate_launch_description()

```
visual_odom_launch.generate_launch_description ( )
```

Chapter 7

Class Documentation

7.1 visual_odom.constants.Coordinate Class Reference

3D spatial coordinate container.

Public Member Functions

- "Coordinate" __add__ (self, "Coordinate" other)

  Add two Coordinate objects component-wise.
- "Coordinate" __sub__ (self, "Coordinate" other)

  Subtract two Coordinate objects component-wise.
- "Coordinate" __mul__ (self, factor)

  Multiply a Coordinate object by a scalar value.
- "Coordinate" __truediv__ (self, float factor)

  Divide a Coordinate object by a scalar value.

Static Public Attributes

- float x

  X-coordinate component in meters.
- float y

  Y-coordinate component in meters.
- float z

  Z-coordinate component in meters.

7.1.1 Detailed Description

3D spatial coordinate container.
Typically used for positions in the `base_link`, `kinect_depth`, or `odom` coordinate frames.

7.1.2 Member Function Documentation

7.1.2.1 __add__()

```
"Coordinate" visual_odom.constants.Coordinate.__add__ (
              self,
             "Coordinate" other )
```

Add two Coordinate objects component-wise.

Parameters

other	Coordinate object to add.

Returns

New Coordinate containing the component-wise sum.

7.1.2.2 __mul__()

```
"Coordinate" visual_odom.constants.Coordinate.__mul__ (
             self,
             factor )
```

Multiply a Coordinate object by a scalar value.

Parameters

factor	Scalar multiplication factor.

Returns

New Coordinate with scaled components.

7.1.2.3 __sub__()

```
"Coordinate" visual_odom.constants.Coordinate.__sub__ (
             self,
            "Coordinate" other )
```

Subtract two Coordinate objects component-wise.

Parameters

other	Coordinate object to subtract.

Returns

New Coordinate containing the component-wise difference.

7.1.2.4 __truediv__()

```
"Coordinate" visual_odom.constants.Coordinate.__truediv__ (
             self,
            float factor )
```

Divide a Coordinate object by a scalar value.

Parameters

factor	Scalar divisor.

Returns

New Coordinate with divided components.

7.1.3 Member Data Documentation

7.1.3.1 x

```
float visual_odom.constants.Coordinate.x  [static]
```

X-coordinate component in meters.

7.1.3.2 y

`float visual_odom.constants.Coordinate.y [static]`
Y-coordinate component in meters.

7.1.3.3 z

`float visual_odom.constants.Coordinate.z [static]`
Z-coordinate component in meters.
The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/constants.py

7.2 visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark Class Reference

Extended Kalman Filter for tracking an individual 3D landmark in the global odom frame.

Public Member Functions

- None __init__ (self, Coordinate x, NDArray P, PixelCoordinate pix_coor, NDArray R=None)

  Initializes the landmark EKF state and covariance matrices.
- Tuple[Coordinate, NDArray] landmark_kalman_iteration (self, Coordinate pos_baselink, float theta, PixelCoordinate pixel_coor)

  Performs a full prediction and update cycle for the landmark position.
- Coordinate state_func (self, Coordinate x)

  State transition model representing stationary landmarks.
- NDArray calculate_jacobian_F (self)

  Calculates the state transition Jacobian matrix.
- NDArray calculate_Q_matrix (self)

  Calculates the stationary process noise covariance matrix.
- NDArray meas_func (self, Coordinate x_tt1, Coordinate pos_robot, float theta_robot)

  Measurement projection model mapping odom-frame coordinates into base_link space.
- NDArray calculate_jacobian_H (self, float theta_robot)

  Calculates the measurement model Jacobian matrix.
- NDArray sigma_R_approximation (self, PixelCoordinate kp)

  Approximates the raw pixel-space and depth-space measurement variances.
- NDArray calculate_R_sigma_matrix (self, PixelCoordinate kp)

  Calculates the measurement noise covariance matrix R transformed into the base_link frame.
- Tuple[Coordinate, NDArray] prediction (self, Coordinate x)

  Runs the EKF prediction step.
- NDArray predictMeasurement (self, Coordinate x_tt1, Coordinate pos_robot, float theta_robot)

  Projects the predicted state into the expected measurement vector space.
- NDArray computeKalmanGain (self, NDArray P_tt1)

  Computes the optimal Kalman gain matrix.
- Tuple[Coordinate, NDArray] update (self, Coordinate x_tt1, NDArray P_tt1, Coordinate pos_robot, float theta_robot, PixelCoordinate kp)

  Performs the measurement update step of the EKF using Joseph-form covariance updates.
- None set_jacobian_F (self)

  Updates the internal state transition Jacobian.
- None set_jacobian_H (self, float theta_robot)

  Updates the internal measurement model Jacobian.
- None set_R (self, PixelCoordinate kp)

Calculates and stores the measurement noise covariance matrix.

- None set_Q (self)

  Stores the process noise covariance matrix.

- NDArray get_R (self)

  Gets the active measurement noise covariance matrix in the base_link frame.

Public Attributes

- x
- P
- R
- Q
- JF
- JH

7.2.1 Detailed Description

Extended Kalman Filter for tracking an individual 3D landmark in the global odom frame.

7.2.2 Constructor & Destructor Documentation

7.2.2.1 __init__()

```
None visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.__init__ (
              self,
             Coordinate x,
             NDArray P,
             PixelCoordinate pix_coor,
             NDArray  R = None )
```

Initializes the landmark EKF state and covariance matrices.

Parameters

x	Initial 3D coordinate state estimate of the landmark.
P	Initial estimation error covariance matrix.
pix_coor	Initial pixel measurement coordinate used to initialize noise.
R	Optional pre-calculated measurement noise covariance matrix.

7.2.3 Member Function Documentation

7.2.3.1 calculate_jacobian_F()

```
NDArray visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.calculate_jacobian_F (
              self )
```

Calculates the state transition Jacobian matrix.

Returns

A 3x3 identity Jacobian matrix since the landmarks are static.

7.2.3.2 calculate_jacobian_H()

```
NDArray visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.calculate_jacobian_H (
              self,
             float theta_robot )
```

Calculates the measurement model Jacobian matrix.

Parameters

theta_robot	Current robot yaw heading orientation in radians.

Returns

3x3 projection Jacobian matrix representing rotation into local base_link.

7.2.3.3 calculate_Q_matrix()

```
NDArray visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.calculate_Q_matrix (
              self )
```

Calculates the stationary process noise covariance matrix.

Returns

A 3x3 diagonal process noise covariance matrix.

7.2.3.4 calculate_R_sigma_matrix()

```
NDArray visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.calculate_R_sigma_matrix (
              self,
             PixelCoordinate kp )
```

Calculates the measurement noise covariance matrix R transformed into the base_link frame.
Applies the pinhole projection Jacobian followed by the camera-to-base_link rotation matrix.

Parameters

kp	The observed PixelCoordinate containing current feature matching data.

Returns

3x3 measurement covariance matrix in the local robot base_link frame.

7.2.3.5 computeKalmanGain()

```
NDArray visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.computeKalmanGain (
              self,
             NDArray P_tt1 )
```

Computes the optimal Kalman gain matrix.

Parameters

P_tt1	Predicted state covariance matrix.

Returns

Calculated 3x3 Kalman gain matrix.

7.2.3.6 get_R()

```
NDArray visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.get_R (
              self )
```

Gets the active measurement noise covariance matrix in the base_link frame.

Returns

3x3 covariance matrix.

7.2.3.7 landmark_kalman_iteration()

```
Tuple[Coordinate, NDArray] visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.landmark_↩
kalman_iteration (
               self,
              Coordinate pos_baselink,
              float theta,
              PixelCoordinate pixel_coor )
```

Performs a full prediction and update cycle for the landmark position.

Parameters

pos_baselink	The current position of the robot's base_link in the odom frame.
theta	The current yaw angle heading of the robot.
pixel_coor	The newly observed pixel coordinate of this landmark.

Returns

A tuple containing the updated Coordinate state and the updated covariance matrix.

7.2.3.8 meas_func()

```
NDArray visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.meas_func (
               self,
              Coordinate x_tt1,
              Coordinate pos_robot,
              float theta_robot )
```

Measurement projection model mapping odom-frame coordinates into base_link space.

Parameters

x_tt1	Predicted landmark coordinate in the global odom frame.
pos_robot	Current robot position coordinates in the global odom frame.
theta_robot	Current robot yaw heading orientation in radians.

Returns

Projected 3D coordinate vector in the robot's local base_link frame.

7.2.3.9 prediction()

```
Tuple[Coordinate, NDArray] visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.prediction (
               self,
              Coordinate x )
```

Runs the EKF prediction step.
Propagates state and covariance forwards in time with process noise.

Parameters

x	Current coordinate state estimate.

Returns

A tuple of (predicted state, predicted covariance).

7.2.3.10 predictMeasurement()

```
NDArray visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.predictMeasurement (
              self,
             Coordinate x_tt1,
             Coordinate pos_robot,
             float theta_robot )
```

Projects the predicted state into the expected measurement vector space.

Parameters

x_tt1	Predicted landmark coordinate in the global odom frame.
pos_robot	Current robot base link position coordinates.
theta_robot	Current robot yaw heading orientation.

Returns

Projected local base_link coordinate array.

7.2.3.11 set_jacobian_F()

```
None visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.set_jacobian_F (
              self )
```

Updates the internal state transition Jacobian.

7.2.3.12 set_jacobian_H()

```
None visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.set_jacobian_H (
              self,
             float theta_robot )
```

Updates the internal measurement model Jacobian.

Parameters

theta_robot	Current robot yaw heading orientation in radians.

7.2.3.13 set_Q()

```
None visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.set_Q (
              self )
```

Stores the process noise covariance matrix.

7.2.3.14 set_R()

```
None visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.set_R (
              self,
             PixelCoordinate kp )
```

Calculates and stores the measurement noise covariance matrix.

Parameters

kp	Observed pixel coordinate from the raw sensor stream.

7.2.3.15 sigma_R_approximation()

```
NDArray visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.sigma_R_approximation (
             self,
            PixelCoordinate kp )
```

Approximates the raw pixel-space and depth-space measurement variances.

Parameters

kp	The observed PixelCoordinate containing pixel indices and raw depth.

Returns

3x3 diagonal measurement covariance matrix in pixel/depth space.

7.2.3.16 state_func()

```
Coordinate visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.state_func (
             self,
            Coordinate x )
```

State transition model representing stationary landmarks.

Parameters

x	The current state estimate of the landmark.

Returns

The predicted landmark state (identity transformation).

7.2.3.17 update()

```
Tuple[Coordinate, NDArray] visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.update (
             self,
            Coordinate x_tt1,
            NDArray P_tt1,
            Coordinate pos_robot,
            float theta_robot,
            PixelCoordinate kp )
```

Performs the measurement update step of the EKF using Joseph-form covariance updates.

Parameters

x_tt1	Predicted landmark coordinate in the global odom frame.
P_tt1	Predicted state covariance matrix.
pos_robot	Current global robot position coordinates.
theta_robot	Current robot yaw heading orientation in radians.
kp	PixelCoordinate corresponding to the observed landmark feature.

Returns

A tuple of (updated state, updated covariance).

7.2.4 Member Data Documentation

7.2.4.1 JF

`visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.JF`

7.2.4.2 JH

`visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.JH`

7.2.4.3 P

`visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.P`

7.2.4.4 Q

`visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.Q`

7.2.4.5 R

`visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.R`

7.2.4.6 x

`visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark.x`

The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/ekf_landmark.py

7.3 visual_odom.ekf_robot.ExtendedKalmanFilterRobot Class Reference

Extended Kalman Filter tracking the 2D planar robot base pose (x, y, theta).

Public Member Functions

- __init__ (self, State x, NDArray P, NDArray Q=None)
  Initializes the robot EKF tracker.
- None prediction (self, State delta_wheel_odom)
  Predicts the robot pose forward based on relative wheel odometry increments.
- NDArray computeKalmanGain (self, NDArray P_tt1)
  Computes the optimal Kalman gain matrix.
- None update (self, State z)
  Fuses a RANSAC visual odometry pose calculation into the tracking state.
- None set_R (self)
  Configures the visual odometry measurement noise covariance matrix R.
- None set_Q (self)
  Configures the process noise covariance matrix Q based on configured standard deviations.
- State get_state (self)
  Gets the current robot base tracking pose.
- NDArray get_covariance_p (self)
  Gets the current pose estimation error covariance matrix.
- None add_noise_to_R (self)
  Increments the tracking update error offset following a RANSAC visual execution failure.
- None set_state (self, State x)
  Directly sets the robot pose state.

Public Attributes

- x
- P
- Q
- F
- H
- additional_noise
- R

7.3.1 Detailed Description

Extended Kalman Filter tracking the 2D planar robot base pose (x, y, theta).

7.3.2 Constructor & Destructor Documentation

7.3.2.1 __init__()

```
visual_odom.ekf_robot.ExtendedKalmanFilterRobot.__init__ (
              self,
             State x,
             NDArray P,
             NDArray  Q = None )
```

Initializes the robot EKF tracker.

Parameters

x	Initial 2D state estimate of the robot platform.
P	Initial estimation error covariance matrix.
Q	Optional pre-calculated process noise covariance matrix.

7.3.3 Member Function Documentation

7.3.3.1 add_noise_to_R()

```
None visual_odom.ekf_robot.ExtendedKalmanFilterRobot.add_noise_to_R (
              self )
```

Increments the tracking update error offset following a RANSAC visual execution failure.
Ensures that repeated visual tracking errors gracefully inflate uncertainty parameters.

7.3.3.2 computeKalmanGain()

```
NDArray visual_odom.ekf_robot.ExtendedKalmanFilterRobot.computeKalmanGain (
              self,
             NDArray P_tt1 )
```

Computes the optimal Kalman gain matrix.

Parameters

P_tt1	Predicted robot state error covariance matrix.

Returns

Calculated 3x3 Kalman gain matrix.

7.3.3.3 get_covariance_p()

```
NDArray visual_odom.ekf_robot.ExtendedKalmanFilterRobot.get_covariance_p (
            self )
```

Gets the current pose estimation error covariance matrix.

Returns

3x3 error covariance matrix.

7.3.3.4 get_state()

```
State visual_odom.ekf_robot.ExtendedKalmanFilterRobot.get_state (
            self )
```

Gets the current robot base tracking pose.

Returns

Estimated State object containing positions and heading.

7.3.3.5 prediction()

```
None visual_odom.ekf_robot.ExtendedKalmanFilterRobot.prediction (
            self,
           State delta_wheel_odom )
```

Predicts the robot pose forward based on relative wheel odometry increments.

Parameters

delta_wheel_odom	Relative displacement step calculated from wheel encoders.

7.3.3.6 set_Q()

```
None visual_odom.ekf_robot.ExtendedKalmanFilterRobot.set_Q (
            self )
```

Configures the process noise covariance matrix Q based on configured standard deviations.

7.3.3.7 set_R()

```
None visual_odom.ekf_robot.ExtendedKalmanFilterRobot.set_R (
            self )
```

Configures the visual odometry measurement noise covariance matrix R.
The measurement noise scales dynamically based on recent RANSAC failures.

Parameters

inlier_ratio	Ratio of inliers used during standard visual processing loops.

7.3.3.8 set_state()

```
None visual_odom.ekf_robot.ExtendedKalmanFilterRobot.set_state (
            self,
           State x )
```

Directly sets the robot pose state.

Parameters

x	New robot pose state to set.

7.3.3.9 update()

```
None visual_odom.ekf_robot.ExtendedKalmanFilterRobot.update (
              self,
             State z )
```

Fuses a RANSAC visual odometry pose calculation into the tracking state.

Parameters

z	Evaluated pose estimate from the RANSAC visual alignment pipeline.
inlier_ratio	Percentage of robust inliers tracked during visual alignment.

7.3.4 Member Data Documentation

7.3.4.1 additional_noise

```
visual_odom.ekf_robot.ExtendedKalmanFilterRobot.additional_noise
```

7.3.4.2 F

```
visual_odom.ekf_robot.ExtendedKalmanFilterRobot.F
```

7.3.4.3 H

```
visual_odom.ekf_robot.ExtendedKalmanFilterRobot.H
```

7.3.4.4 P

```
visual_odom.ekf_robot.ExtendedKalmanFilterRobot.P
```

7.3.4.5 Q

```
visual_odom.ekf_robot.ExtendedKalmanFilterRobot.Q
```

7.3.4.6 R

```
visual_odom.ekf_robot.ExtendedKalmanFilterRobot.R
```

7.3.4.7 x

```
visual_odom.ekf_robot.ExtendedKalmanFilterRobot.x
```

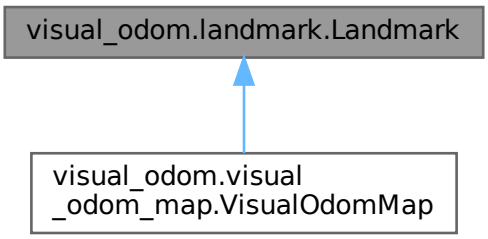
The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/ekf_robot.py

7.4 visual_odom.landmark.Landmark Class Reference

Landmark class for storing visual features and performing spatial estimation.

Inheritance diagram for visual_odom.landmark.Landmark:



Public Member Functions

- None __init__ (self, NDArray P_init, PixelCoordinate pixel_coor, cv2.KeyPoint kp, np.ndarray des, float trust=30.0, int color=0, Coordinate odom_coordinates=Coordinate(0.0, 0.0, 0.0))

  Initializes a Landmark tracking instance.
- np.ndarray get_descriptor (self)

  Gets the binary feature descriptor array.
- float get_depth (self)

  Gets the raw depth tracking value.
- float get_u (self)

  Gets the horizontal pixel coordinate.
- float get_v (self)

  Gets the vertical pixel coordinate.
- float get_trust (self)

  Gets the current visual trust tracking parameter.
- int get_color (self)

  Gets the color property used for visual representation.
- Coordinate get_odom_coordinates (self)

  Gets the estimated global odom-frame coordinates.
- None set_odom_coordinates (self, Coordinate odom_coordinates)

  Sets the global odom-frame coordinates and updates the internal EKF.
- Coordinate get_kinect_coordinates (self)

  Gets the local Kinect frame coordinates.
- NDArray get_P (self)

  Gets the estimated covariance matrix.
- bool is_visible (self, Coordinate pos_camera_odom, float theta_robot)

  Checks if the landmark falls within the horizontal and vertical field of view of the sensor.
- None reset_trust (self)

  Resets the visual tracking trust property to its initial baseline value of 30.0.
- None increase_trust (self)

  Increments the trust value after the landmark is successfully re-observed.
- None decrease_trust (self)

  Applies an exponential decay to the trust value when the landmark is missed.
- None landmark_kalman_iteration (self, Coordinate pos_baselink, float theta, PixelCoordinate pixel_coor)

  Runs a prediction-update EKF cycle to refine estimated global odom coordinates.
- float calculate_likelihood (self, Coordinate z_pos_odom, float theta)

  Calculates the measurement likelihood of the landmark given its current state.

Public Attributes

- pixel_coor
- des
- trust
- color
- kinect_coordinates
- odom_coordinates
- P
- landmark_ekf

7.4.1 Detailed Description

Landmark class for storing visual features and performing spatial estimation.

7.4.2 Constructor & Destructor Documentation

7.4.2.1 __init__()

```
None visual_odom.landmark.Landmark.__init__ (
              self,
             NDArray P_init,
             PixelCoordinate pixel_coor,
             cv2.KeyPoint kp,
             np.ndarray des,
             float  trust = 30.0,
             int  color = 0,
             Coordinate  odom_coordinates = Coordinate(0.0, 0.0, 0.0) )
```

Initializes a Landmark tracking instance.

Parameters

P_init	Initial EKF error covariance matrix.
pixel_coor	2D pixel coordinate containing columns, rows, and raw depth values.
kp	OpenCV KeyPoint object associated with this visual feature.
des	Binary descriptor array of the ORB keypoint.
trust	Initial confidence tracking metric assigned to this feature.
color	RGB packing integer value used to render point cloud structures.
odom_coordinates	Initial transformed Coordinate frame estimation.

Reimplemented in visual_odom.visual_odom_map.VisualOdomMap.

7.4.3 Member Function Documentation

7.4.3.1 calculate_likelihood()

```
float visual_odom.landmark.Landmark.calculate_likelihood (
              self,
             Coordinate z_pos_odom,
             float theta )
```

Calculates the measurement likelihood of the landmark given its current state.
Must be called after running the EKF tracking iteration.

Parameters

z_pos_odom	Measured coordinate in the odom frame.
theta	Current robot heading yaw orientation.

Returns

Gaussian probability density likelihood value.

7.4.3.2 decrease_trust()

```
None visual_odom.landmark.Landmark.decrease_trust (
            self )
```

Applies an exponential decay to the trust value when the landmark is missed.

7.4.3.3 get_color()

```
int visual_odom.landmark.Landmark.get_color (
            self )
```

Gets the color property used for visual representation.

Returns

Packed color integer.

7.4.3.4 get_depth()

```
float visual_odom.landmark.Landmark.get_depth (
            self )
```

Gets the raw depth tracking value.

Returns

Sensor depth value in millimeters.

Reimplemented in visual_odom.visual_odom_map.VisualOdomMap.

7.4.3.5 get_descriptor()

```
np.ndarray visual_odom.landmark.Landmark.get_descriptor (
            self )
```

Gets the binary feature descriptor array.

Returns

1D feature descriptor.

7.4.3.6 get_kinect_coordinates()

```
Coordinate visual_odom.landmark.Landmark.get_kinect_coordinates (
            self )
```

Gets the local Kinect frame coordinates.

Returns

Coordinate object in meters.

Reimplemented in visual_odom.visual_odom_map.VisualOdomMap.

7.4.3.7 get_odom_coordinates()

```
Coordinate visual_odom.landmark.Landmark.get_odom_coordinates (
            self )
```

Gets the estimated global odom-frame coordinates.

Returns

Coordinate object in meters.

Reimplemented in visual_odom.visual_odom_map.VisualOdomMap.

7.4.3.8 get_P()

```
NDArray visual_odom.landmark.Landmark.get_P (
            self )
```

Gets the estimated covariance matrix.

Returns

3x3 error covariance matrix.

7.4.3.9 get_trust()

```
float visual_odom.landmark.Landmark.get_trust (
            self )
```

Gets the current visual trust tracking parameter.

Returns

Trust metric.

7.4.3.10 get_u()

```
float visual_odom.landmark.Landmark.get_u (
            self )
```

Gets the horizontal pixel coordinate.

Returns

Column pixel index.

7.4.3.11 get_v()

```
float visual_odom.landmark.Landmark.get_v (
            self )
```

Gets the vertical pixel coordinate.

Returns

Row pixel index.

7.4.3.12 increase_trust()

```
None visual_odom.landmark.Landmark.increase_trust (
            self )
```

Increments the trust value after the landmark is successfully re-observed.

7.4.3.13 is_visible()

```
bool visual_odom.landmark.Landmark.is_visible (
            self,
           Coordinate pos_camera_odom,
           float theta_robot )
```

Checks if the landmark falls within the horizontal and vertical field of view of the sensor.

Parameters

pos_camera_odom	Position of the camera in the global odom frame.
theta_robot	Robot heading yaw orientation in radians.

Returns

True if the landmark is within depth, azimuth, and altitude boundaries, otherwise False.

7.4.3.14 landmark_kalman_iteration()

```
None visual_odom.landmark.Landmark.landmark_kalman_iteration (
              self,
             Coordinate pos_baselink,
             float theta,
             PixelCoordinate pixel_coor )
```

Runs a prediction-update EKF cycle to refine estimated global odom coordinates.

Parameters

pos_baselink	Current position of the robot's base_link in the odom frame.
theta	Current yaw angle heading of the robot.
pixel_coor	Updated raw sensor pixel observation.

Reimplemented in visual_odom.visual_odom_map.VisualOdomMap.

7.4.3.15 reset_trust()

```
None visual_odom.landmark.Landmark.reset_trust (
              self )
```

Resets the visual tracking trust property to its initial baseline value of 30.0.

7.4.3.16 set_odom_coordinates()

```
None visual_odom.landmark.Landmark.set_odom_coordinates (
              self,
             Coordinate odom_coordinates )
```

Sets the global odom-frame coordinates and updates the internal EKF.

Parameters

odom_coordinates	New Coordinate location.

7.4.4 Member Data Documentation

7.4.4.1 color

```
visual_odom.landmark.Landmark.color
```

7.4.4.2 des

```
visual_odom.landmark.Landmark.des
```

7.4.4.3 kinect_coordinates

```
visual_odom.landmark.Landmark.kinect_coordinates
```

7.4.4.4 landmark_ekf

```
visual_odom.landmark.Landmark.landmark_ekf
```

7.4.4.5 odom_coordinates

```
visual_odom.landmark.Landmark.odom_coordinates
```

7.4.4.6 P

```
visual_odom.landmark.Landmark.P
```

7.4.4.7 pixel_coor

```
visual_odom.landmark.Landmark.pixel_coor
```

7.4.4.8 trust

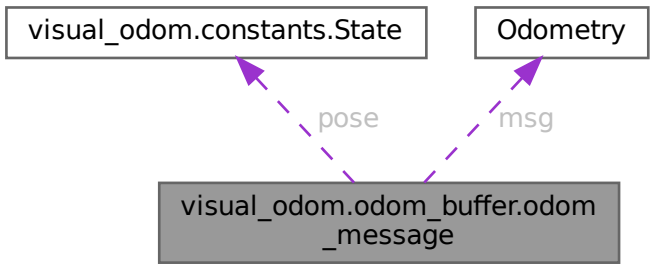
```
visual_odom.landmark.Landmark.trust
```

The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/landmark.py

7.5 visual_odom.odom_buffer.odom_message Class Reference

Collaboration diagram for visual_odom.odom_buffer.odom_message:



Static Public Attributes

- float stamp
- State pose
- Odometry msg

7.5.1 Member Data Documentation

7.5.1.1 msg

```
Odometry visual_odom.odom_buffer.odom_message.msg  [static]
```

7.5.1.2 pose

```
State visual_odom.odom_buffer.odom_message.pose  [static]
```

7.5.1.3 stamp

```
float visual_odom.odom_buffer.odom_message.stamp  [static]
```

The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/odom_buffer.py

7.6 visual_odom.odom_buffer.OdomBuffer Class Reference

Public Member Functions

- __init__ (self, maxlen=1000)
- add_odom_message (self, Odometry msg)
- get_latest (self)
- get_interpolated (self, float stamp)

Public Attributes

- Odom_Buffer

7.6.1 Constructor & Destructor Documentation

7.6.1.1 __init__()

```
visual_odom.odom_buffer.OdomBuffer.__init__ (
              self,
              maxlen = 1000 )
```

7.6.2 Member Function Documentation

7.6.2.1 add_odom_message()

```
visual_odom.odom_buffer.OdomBuffer.add_odom_message (
              self,
             Odometry msg )
```

7.6.2.2 get_interpolated()

```
visual_odom.odom_buffer.OdomBuffer.get_interpolated (
              self,
             float stamp )
```

7.6.2.3 get_latest()

```
visual_odom.odom_buffer.OdomBuffer.get_latest (
              self )
```

7.6.3 Member Data Documentation

7.6.3.1 Odom_Buffer

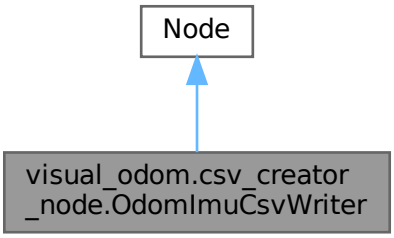
```
visual_odom.odom_buffer.OdomBuffer.Odom_Buffer
```

The documentation for this class was generated from the following file:

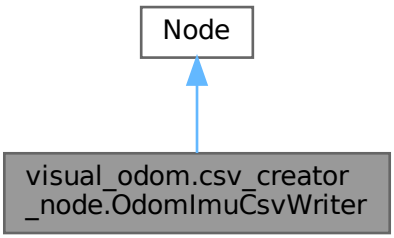
- ros2_ws/src/visual_odom/visual_odom/odom_buffer.py

7.7 visual_odom.csv_creator_node.OdomImuCsvWriter Class Reference

ROS 2 Node responsible for subscribing to localization data topics and logging state measurements into a unified CSV file for analysis.
Inheritance diagram for visual_odom.csv_creator_node.OdomImuCsvWriter:



Collaboration diagram for visual_odom.csv_creator_node.OdomImuCsvWriter:



Public Member Functions

- __init__ (self)

  Initializes the data logging node, configures ROS parameters, files, and subscriptions.
- odom_callback (self, Odometry msg)

  Processes standard wheel odometry data updates, calculates float timestamps, and flushes synchronized state snapshots down into the CSV table log.
- imu_callback (self, Imu msg)

  Receives structural IMU orientation frames, extracts yaw orientation data, and applies a rigid frame offset transformation to align it with reference norms.
- visual_odom_callback (self, Odometry msg)

  Processes updates published by the visual odometry algorithm pipeline, unpacking 2D translation states and rotation yaw angles.
- float quaternion_to_yaw (self, q)

  Calculates planar yaw angle heading out of 3D spatial rotation quaternion parameters.
- destroy_node (self)

  Safely terminates subscription ties and guarantees that data buffers flush completely before closing the file streams.

- float normalize_angle (self, float angle)

  Wraps an input angle parameter into standard bounded circle space [-pi, pi].

Public Attributes

- csv_path
- wheel_x

  < *Absolute file path where the generated CSV data log will be stored.*
- csv_file

  < *Latest recorded global X position from wheel odometry in meters.*
- odom_callback

  < *Internal file descriptor handling active stream writes to disk.*
- imu_callback

  < *Internal file descriptor handling active stream writes to disk.*
- visual_odom_callback

  < *Internal file descriptor handling active stream writes to disk.*
- wheel_y
- imu_theta
- odom_visual_x
- odom_visual_y
- odom_visual_theta

7.7.1 Detailed Description

ROS 2 Node responsible for subscribing to localization data topics and logging state measurements into a unified CSV file for analysis.

7.7.2 Constructor & Destructor Documentation

7.7.2.1 __init__()

```
visual_odom.csv_creator_node.OdomImuCsvWriter.__init__ (
              self )
```

Initializes the data logging node, configures ROS parameters, files, and subscriptions.

7.7.3 Member Function Documentation

7.7.3.1 destroy_node()

```
visual_odom.csv_creator_node.OdomImuCsvWriter.destroy_node (
              self )
```

Safely terminates subscription ties and guarantees that data buffers flush completely before closing the file streams.

7.7.3.2 imu_callback()

```
visual_odom.csv_creator_node.OdomImuCsvWriter.imu_callback (
              self,
            Imu msg )
```

Receives structural IMU orientation frames, extracts yaw orientation data, and applies a rigid frame offset transformation to align it with reference norms.

Parameters

msg	Incoming raw IMU reading container (sensor_msgs.msg.Imu).

7.7.3.3 normalize_angle()

```
float visual_odom.csv_creator_node.OdomImuCsvWriter.normalize_angle (
             self,
            float angle )
```

Wraps an input angle parameter into standard bounded circle space [-pi, pi].

Parameters

angle	Arbitrary orientation scale input in radians.

Returns

Wrapped equivalent within circular system boundaries (float).

7.7.3.4 odom_callback()

```
visual_odom.csv_creator_node.OdomImuCsvWriter.odom_callback (
             self,
            Odometry msg )
```

Processes standard wheel odometry data updates, calculates float timestamps, and flushes synchronized state snapshots down into the CSV table log.

Parameters

msg	The incoming wheel odometry message package (nav_msgs.msg.Odometry).

7.7.3.5 quaternion_to_yaw()

```
float visual_odom.csv_creator_node.OdomImuCsvWriter.quaternion_to_yaw (
             self,
             q )
```

Calculates planar yaw angle heading out of 3D spatial rotation quaternion parameters.

Parameters

q	Input object storing spatial orientation data (geometry_msgs.msg.Quaternion).

Returns

Planar heading representation mapped in radians (float).

7.7.3.6 visual_odom_callback()

```
visual_odom.csv_creator_node.OdomImuCsvWriter.visual_odom_callback (
             self,
            Odometry msg )
```

Processes updates published by the visual odometry algorithm pipeline, unpacking 2D translation states and rotation yaw angles.

Parameters

msg	The incoming camera odometry message packet (nav_msgs.msg.Odometry).

7.7.4 Member Data Documentation

7.7.4.1 csv_file

visual_odom.csv_creator_node.OdomImuCsvWriter.csv_file
< Latest recorded global X position from wheel odometry in meters.
< Latest recorded global Y position from wheel odometry in meters.
< Latest recorded planar yaw heading orientation from the IMU sensor in radians.
< Latest recorded global X position calculated by visual odometry in meters.
< Latest recorded global Y position calculated by visual odometry in meters.
< Latest recorded planar yaw heading calculated by visual odometry in radians.

7.7.4.2 csv_path

visual_odom.csv_creator_node.OdomImuCsvWriter.csv_path

7.7.4.3 imu_callback

visual_odom.csv_creator_node.OdomImuCsvWriter.imu_callback
< Internal file descriptor handling active stream writes to disk.
< CSV writer interface mapping programmatic arrays to formatted textual table records.
< File structure line defining data labels inside column headers.

7.7.4.4 imu_theta

visual_odom.csv_creator_node.OdomImuCsvWriter.imu_theta

7.7.4.5 odom_callback

visual_odom.csv_creator_node.OdomImuCsvWriter.odom_callback
< Internal file descriptor handling active stream writes to disk.
< CSV writer interface mapping programmatic arrays to formatted textual table records.
< File structure line defining data labels inside column headers.

7.7.4.6 odom_visual_theta

visual_odom.csv_creator_node.OdomImuCsvWriter.odom_visual_theta

7.7.4.7 odom_visual_x

visual_odom.csv_creator_node.OdomImuCsvWriter.odom_visual_x

7.7.4.8 odom_visual_y

visual_odom.csv_creator_node.OdomImuCsvWriter.odom_visual_y

7.7.4.9 visual_odom_callback

visual_odom.csv_creator_node.OdomImuCsvWriter.visual_odom_callback
< Internal file descriptor handling active stream writes to disk.
< CSV writer interface mapping programmatic arrays to formatted textual table records.
< File structure line defining data labels inside column headers.

7.7.4.10 wheel_x

visual_odom.csv_creator_node.OdomImuCsvWriter.wheel_x
< Absolute file path where the generated CSV data log will be stored.

7.7.4.11 wheel_y

`visual_odom.csv_creator_node.OdomImuCsvWriter.wheel_y`

The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/csv_creator_node.py

7.8 visual_odom.constants.Parameters Class Reference

Runtime configuration and threshold parameter package for the SLAM node.

Static Public Attributes

- int max_depth

  Maximum reliable sensor range threshold.
- float ransac_evaluation_tolerance

  Inlier distance threshold for RANSAC outlier rejection in meters.
- int ransac_iteration

  Maximum number of optimization loops for the RANSAC routine.
- int ransac_sample_size

  Minimum data subset size used to compute a transform hypothesis.
- float rgb_depth_sync_tolerance_sec

  Maximum allowed time delta between synchronized RGB and depth frames.
- int pixel_tolerance

  Pixel window distance threshold for visual matching checks.
- int matches_for_new_landmarks

  Required match counter threshold before declaring a stable new map landmark.
- int min_matches_for_ransac

  Minimum required feature matches to attempt a RANSAC visual odometry calculation.
- float max_rotation_angle_deg

  Safety constraint capping the maximum plausible robot rotation per frame step.
- float min_landmark_trust

  Minimum trust value before an unobserved landmark is pruned from map memory.
- float ransac_min_inlier_ratio

  Minimum acceptable percentage of inliers needed to trust a RANSAC solution.
- int n_robot_samples

  Number of particles/samples generated within the localization filter.
- str topic_visual_odometry_msg

  ROS 2 topic name where the calculated odometry path is published.

7.8.1 Detailed Description

Runtime configuration and threshold parameter package for the SLAM node.
Stores all relevant runtime parameters loaded from ROS 2 parameter declarations.

7.8.2 Member Data Documentation

7.8.2.1 matches_for_new_landmarks

`int visual_odom.constants.Parameters.matches_for_new_landmarks [static]`

Required match counter threshold before declaring a stable new map landmark.

7.8.2.2 max_depth

`int visual_odom.constants.Parameters.max_depth [static]`

Maximum reliable sensor range threshold.

7.8.2.3 max_rotation_angle_deg

`float visual_odom.constants.Parameters.max_rotation_angle_deg [static]`
Safety constraint capping the maximum plausible robot rotation per frame step.

7.8.2.4 min_landmark_trust

`float visual_odom.constants.Parameters.min_landmark_trust [static]`
Minimum trust value before an unobserved landmark is pruned from map memory.

7.8.2.5 min_matches_for_ransac

`int visual_odom.constants.Parameters.min_matches_for_ransac [static]`
Minimum required feature matches to attempt a RANSAC visual odometry calculation.

7.8.2.6 n_robot_samples

`int visual_odom.constants.Parameters.n_robot_samples [static]`
Number of particles/samples generated within the localization filter.

7.8.2.7 pixel_tolerance

`int visual_odom.constants.Parameters.pixel_tolerance [static]`
Pixel window distance threshold for visual matching checks.

7.8.2.8 ransac_evaluation_tolerance

`float visual_odom.constants.Parameters.ransac_evaluation_tolerance [static]`
Inlier distance threshold for RANSAC outlier rejection in meters.

7.8.2.9 ransac_iteration

`int visual_odom.constants.Parameters.ransac_iteration [static]`
Maximum number of optimization loops for the RANSAC routine.

7.8.2.10 ransac_min_inlier_ratio

`float visual_odom.constants.Parameters.ransac_min_inlier_ratio [static]`
Minimum acceptable percentage of inliers needed to trust a RANSAC solution.

7.8.2.11 ransac_sample_size

`int visual_odom.constants.Parameters.ransac_sample_size [static]`
Minimum data subset size used to compute a transform hypothesis.

7.8.2.12 rgb_depth_sync_tolerance_sec

`float visual_odom.constants.Parameters.rgb_depth_sync_tolerance_sec [static]`
Maximum allowed time delta between synchronized RGB and depth frames.

7.8.2.13 topic_visual_odometry_msg

`str visual_odom.constants.Parameters.topic_visual_odometry_msg [static]`
ROS 2 topic name where the calculated odometry path is published.
The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/constants.py

7.9 visual_odom.constants.PixelCoordinate Class Reference

Container for a 2D image pixel and its depth value.

Static Public Attributes

- float u

  Horizontal pixel column index in the image plane.
- float v

  Vertical pixel row index in the image plane.
- float z

  Raw depth value from the sensor corresponding to this pixel.

7.9.1 Detailed Description

Container for a 2D image pixel and its depth value.
Stores a pixel coordinate in the image plane together with the corresponding registered raw depth value.

7.9.2 Member Data Documentation

7.9.2.1 u

`float visual_odom.constants.PixelCoordinate.u [static]`
Horizontal pixel column index in the image plane.

7.9.2.2 v

`float visual_odom.constants.PixelCoordinate.v [static]`
Vertical pixel row index in the image plane.

7.9.2.3 z

`float visual_odom.constants.PixelCoordinate.z [static]`
Raw depth value from the sensor corresponding to this pixel.
The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/constants.py

7.10 visual_odom.constants.State Class Reference

2D robot pose representation.

Public Member Functions

- "State" __add__ (self, "State" other)

  Add two State objects component-wise.
- "State" __sub__ (self, "State" other)

  Subtract two State objects component-wise.
- "State" __mul__ (self, factor)

  Multiply a State object by a scalar value.
- "State" __truediv__ (self, float factor)

  Divide a State object by a scalar value.

Static Public Attributes

- float x

  Global X-position of the robot base in meters.
- float y

  Global Y-position of the robot base in meters.
- float theta

  Heading/yaw orientation of the robot in radians.

7.10.1 Detailed Description

2D robot pose representation.
Represents planar robot translation and heading.

7.10.2 Member Function Documentation

7.10.2.1 __add__()

```
"State" visual_odom.constants.State.__add__ (
            self,
           "State" other )
```

Add two State objects component-wise.

Parameters

other	State object to add.

Returns

New State containing the component-wise sum.

7.10.2.2 __mul__()

```
"State" visual_odom.constants.State.__mul__ (
            self,
            factor )
```

Multiply a State object by a scalar value.

Parameters

factor	Scalar multiplication factor.

Returns

New State with scaled values.

7.10.2.3 __sub__()

```
"State" visual_odom.constants.State.__sub__ (
            self,
           "State" other )
```

Subtract two State objects component-wise.

Parameters

other	State object to subtract.

Returns

New State containing the component-wise difference.

7.10.2.4 __truediv__()

```
"State" visual_odom.constants.State.__truediv__ (
            self,
           float factor )
```

Divide a State object by a scalar value.

Parameters

factor	Scalar divisor.

Returns

New State with divided values.

7.10.3 Member Data Documentation

7.10.3.1 theta

`float visual_odom.constants.State.theta [static]`
Heading/yaw orientation of the robot in radians.

7.10.3.2 x

`float visual_odom.constants.State.x [static]`
Global X-position of the robot base in meters.

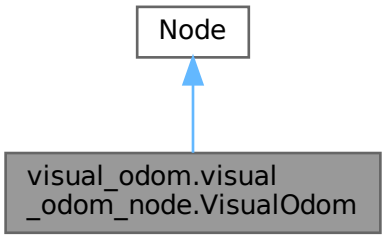
7.10.3.3 y

`float visual_odom.constants.State.y [static]`
Global Y-position of the robot base in meters.
The documentation for this class was generated from the following file:

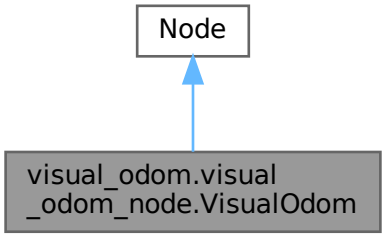
- ros2_ws/src/visual_odom/visual_odom/constants.py

7.11 visual_odom.visual_odom_node.VisualOdom Class Reference

ROS 2 Visual Odometry node managing features extraction, RANSAC estimation, and particle weight selection.
Inheritance diagram for visual_odom.visual_odom_node.VisualOdom:



Collaboration diagram for visual_odom.visual_odom_node.VisualOdom:



Public Member Functions

- __init__ (self)

  Initializes the visual odometry node, subscribers, publishers, and ORB keypoint detectors.
- None listener_wheel_odom_callback (self, Odometry msg)

  Receives wheel odometry pose measurements and initializes baseline tracking.
- None listener_rgb_callback (self, Image msg)

  Handles primary RGB camera frames.
- None possible_resample (self, List[VisualRobotSample] robots, int best_robot_idx, float first_frame_stamp)

  Performs resampling by matching visible landmarks to the first frame.
- Tuple[Coordinate, float, List[int], int] ransac_frame_to_map (self, List[cv2.DMatch] matches, List[Coordinate] landmarks_odom_pos, List[cv2.KeyPoint] valid_kp, np.ndarray valid_kp_depth)

  Estimates the transformation between map landmarks and the current frame using RANSAC.
- Tuple[Coordinate, float, List[cv2.KeyPoint], int] ransac_frame_to_frame (self, List[cv2.DMatch] matches, List[cv2.KeyPoint] valid_kp, np.ndarray valid_kp_depth, List[cv2.KeyPoint] valid_kp_last, np.ndarray valid_kp_depth_last)

  Estimates translation and rotation between consecutive frames using RANSAC and 2D Kabsch point set alignment.
- ransac_calculation (self, List P, List Q, List[cv2.DMatch] matches, List[cv2.KeyPoint] valid_kp)

  Executes the core RANSAC loop and computes the best transformation using Kabsch.
- None listener_depth_callback (self, Image msg)

  Subscribes to raw depth frames and registers active depth timestamps.
- Tuple[List[cv2.KeyPoint], np.ndarray, np.ndarray, Time] img_to_kp_des_filtered (self, Image msg)

  Detects ORB features and filters out close feature duplicates.
- None iteration (self, List[cv2.KeyPoint] valid_kp, np.ndarray valid_des, np.ndarray valid_kp_depth, NDArray frame_rgb, NDArray frame_depth, Time rgb_stamp, Tuple[Coordinate, float, List[cv2.KeyPoint], int] ransac↩ _result)

  Runs a single localization iteration over all particles.
- Parameters parameter_initialization (self)

  Declares and loads configuration parameters from the ROS 2 parameters server.
- SetParametersResult parameter_callback (self, List[rclpy.parameter.Parameter] params)

  Dynamic parameter callback handling updates during node executions.

Public Attributes

- parameter_callback
- bridge
- orb

- bf
- subscription_rgb
- subscription_depth
- subscription_wheel_odom
- publisher_keypoints_3d
- total_publisher
- publisher_visual_odometry_msg
- publisher_cone
- publisher_image
- publisher_accumulated_pixels
- visual_odom_path
- tf_broadcaster
- tf_buffer
- tf_listener
- frame_depth
- frame_depth_stamp
- frame_rgb
- theta
- valid_kp_last
- valid_des_last
- valid_kp_depth_last
- valid_kp
- valid_des
- first_kp
- first_des
- pos_baselink
- best_robot_idx
- best_weight
- first_iteration
- position_initialized
- covariance_P
- frame_stamp
- valid_kp_depth
- first_kp_depth
- first_frame_stamp
- first_theta
- first_pos
- robots
- visible_landmarks
- best_robot

Protected Member Functions

- float _stamp_to_sec (self, builtin_interfaces.msg.Time stamp)

  Converts standard ROS timeline stamps to floating point seconds.

7.11.1 Detailed Description

ROS 2 Visual Odometry node managing features extraction, RANSAC estimation, and particle weight selection.

7.11.2 Constructor & Destructor Documentation

7.11.2.1 __init__()

```
visual_odom.visual_odom_node.VisualOdom.__init__ (
            self )
```

Initializes the visual odometry node, subscribers, publishers, and ORB keypoint detectors.

7.11.3 Member Function Documentation

7.11.3.1 _stamp_to_sec()

```
float visual_odom.visual_odom_node.VisualOdom._stamp_to_sec (
              self,
             builtin_interfaces.msg.Time stamp )  [protected]
```

Converts standard ROS timeline stamps to floating point seconds.

Parameters

stamp	ROS timeline stamp.

Returns

floating point seconds.

7.11.3.2 img_to_kp_des_filtered()

```
Tuple[List[cv2.KeyPoint], np.ndarray, np.ndarray, Time] visual_odom.visual_odom_node.Visual↩
Odom.img_to_kp_des_filtered (
              self,
             Image msg )
```

Detects ORB features and filters out close feature duplicates.
Ensure that features have valid depth observations within range thresholds.

Parameters

msg	Incoming color image message.

Returns

A tuple of (valid keypoints list, descriptors, depth values, timestamp).

7.11.3.3 iteration()

```
None visual_odom.visual_odom_node.VisualOdom.iteration (
              self,
             List[cv2.KeyPoint] valid_kp,
             np.ndarray valid_des,
             np.ndarray valid_kp_depth,
             NDArray frame_rgb,
             NDArray frame_depth,
             Time rgb_stamp,
             Tuple[Coordinate, float, List[cv2.KeyPoint], int] ransac_result )
```

Runs a single localization iteration over all particles.
Calculates normalized weights and selects the best candidate particle estimate.

Parameters

valid_kp	Set of visual features.
valid_des	Binary visual descriptors.
valid_kp_depth	Feature depth values.
frame_rgb	Color reference frame image.
frame_depth	Depth reference frame image.
rgb_stamp	Timeline synchronization timestamp.
ransac_result	Estimated relative rigid transformation data.

7.11.3.4 listener_depth_callback()

```
None visual_odom.visual_odom_node.VisualOdom.listener_depth_callback (
              self,
             Image msg )
```

Subscribes to raw depth frames and registers active depth timestamps.

Parameters

msg	Incoming raw depth image.

7.11.3.5 listener_rgb_callback()

```
None visual_odom.visual_odom_node.VisualOdom.listener_rgb_callback (
              self,
             Image msg )
```

Handles primary RGB camera frames.
Triggers visual processing loops and publishes transforms.

Parameters

msg	Incoming RGB reference image.

7.11.3.6 listener_wheel_odom_callback()

```
None visual_odom.visual_odom_node.VisualOdom.listener_wheel_odom_callback (
              self,
             Odometry msg )
```

Receives wheel odometry pose measurements and initializes baseline tracking.

Parameters

msg	Incoming wheel odometry message.

7.11.3.7 parameter_callback()

```
SetParametersResult visual_odom.visual_odom_node.VisualOdom.parameter_callback (
              self,
             List[rclpy.parameter.Parameter] params )
```

Dynamic parameter callback handling updates during node executions.

Parameters

params	Updated parameters list.

Returns

ROS 2 SetParametersResult status indicator.

7.11.3.8 parameter_initialization()

```
Parameters visual_odom.visual_odom_node.VisualOdom.parameter_initialization (
              self )
```

Declares and loads configuration parameters from the ROS 2 parameters server.

Returns

Initialized Parameters data package.

7.11.3.9 possible_resample()

```
None visual_odom.visual_odom_node.VisualOdom.possible_resample (
              self,
             List[VisualRobotSample] robots,
             int best_robot_idx,
             float first_frame_stamp )
```

Performs resampling by matching visible landmarks to the first frame.

Parameters

robots	List of active robot particles.
best_robot_idx	Index of the best particle hypothesis.
first_frame_stamp	Timestamp of the first recorded frame.

Returns

None

7.11.3.10 ransac_calculation()

```
visual_odom.visual_odom_node.VisualOdom.ransac_calculation (
              self,
             List P,
             List Q,
             List[cv2.DMatch] matches,
             List[cv2.KeyPoint] valid_kp )
```

Executes the core RANSAC loop and computes the best transformation using Kabsch.

Parameters

P	Reference point set.
Q	Observed point set.
matches	Descriptor matches corresponding to P/Q pairs.
valid_kp	Valid keypoints used for visualization of inliers.

Returns

Best translation, rotation, inlier keypoints, and number of inliers.

7.11.3.11 ransac_frame_to_frame()

```
Tuple[Coordinate, float, List[cv2.KeyPoint], int] visual_odom.visual_odom_node.VisualOdom.↩
ransac_frame_to_frame (
              self,
             List[cv2.DMatch] matches,
             List[cv2.KeyPoint] valid_kp,
             np.ndarray valid_kp_depth,
             List[cv2.KeyPoint] valid_kp_last,
             np.ndarray valid_kp_depth_last )
```

Estimates translation and rotation between consecutive frames using RANSAC and 2D Kabsch point set alignment.

Parameters

matches	Vector list of descriptor matches between consecutive visual frames.
valid_kp	Feature coordinates extracted from the current frame.
valid_kp_depth	Pixel depth values matching current features.
valid_kp_last	Feature coordinates extracted from the last frame.
valid_kp_depth_last	Pixel depth values matching last frame features.

Returns

A tuple of (estimated Coordinate translation, rotation yaw, visual inlier keypoints, total inliers).

7.11.3.12 ransac_frame_to_map()

```
Tuple[Coordinate, float, List[int], int] visual_odom.visual_odom_node.VisualOdom.ransac_↩
frame_to_map (
              self,
             List[cv2.DMatch] matches,
             List[Coordinate] landmarks_odom_pos,
             List[cv2.KeyPoint] valid_kp,
             np.ndarray valid_kp_depth )
```

Estimates the transformation between map landmarks and the current frame using RANSAC.

Parameters

matches	Descriptor matches between map landmarks and current frame features.
landmarks_odom_pos	Landmark positions in odom frame.
valid_kp	Valid keypoints from current frame.
valid_kp_depth	Depth values of current frame keypoints.

Returns

Estimated translation, rotation, inlier keypoints, and number of inliers.

7.11.4 Member Data Documentation

7.11.4.1 best_robot

```
visual_odom.visual_odom_node.VisualOdom.best_robot
```

7.11.4.2 best_robot_idx

```
visual_odom.visual_odom_node.VisualOdom.best_robot_idx
```

7.11.4.3 best_weight

```
visual_odom.visual_odom_node.VisualOdom.best_weight
```

7.11.4.4 bf

```
visual_odom.visual_odom_node.VisualOdom.bf
```

7.11.4.5 bridge

```
visual_odom.visual_odom_node.VisualOdom.bridge
```

7.11.4.6 covariance_P

visual_odom.visual_odom_node.VisualOdom.covariance_P

7.11.4.7 first_des

visual_odom.visual_odom_node.VisualOdom.first_des

7.11.4.8 first_frame_stamp

visual_odom.visual_odom_node.VisualOdom.first_frame_stamp

7.11.4.9 first_iteration

visual_odom.visual_odom_node.VisualOdom.first_iteration

7.11.4.10 first_kp

visual_odom.visual_odom_node.VisualOdom.first_kp

7.11.4.11 first_kp_depth

visual_odom.visual_odom_node.VisualOdom.first_kp_depth

7.11.4.12 first_pos

visual_odom.visual_odom_node.VisualOdom.first_pos

7.11.4.13 first_theta

visual_odom.visual_odom_node.VisualOdom.first_theta

7.11.4.14 frame_depth

visual_odom.visual_odom_node.VisualOdom.frame_depth

7.11.4.15 frame_depth_stamp

visual_odom.visual_odom_node.VisualOdom.frame_depth_stamp

7.11.4.16 frame_rgb

visual_odom.visual_odom_node.VisualOdom.frame_rgb

7.11.4.17 frame_stamp

visual_odom.visual_odom_node.VisualOdom.frame_stamp

7.11.4.18 orb

visual_odom.visual_odom_node.VisualOdom.orb

7.11.4.19 parameter_callback

visual_odom.visual_odom_node.VisualOdom.parameter_callback

7.11.4.20 pos_baselink

visual_odom.visual_odom_node.VisualOdom.pos_baselink

7.11.4.21 position_initialized

visual_odom.visual_odom_node.VisualOdom.position_initialized

7.11.4.22 publisher_accumulated_pixels

visual_odom.visual_odom_node.VisualOdom.publisher_accumulated_pixels

7.11.4.23 publisher_cone

visual_odom.visual_odom_node.VisualOdom.publisher_cone

7.11.4.24 publisher_image

visual_odom.visual_odom_node.VisualOdom.publisher_image

7.11.4.25 publisher_keypoints_3d

visual_odom.visual_odom_node.VisualOdom.publisher_keypoints_3d

7.11.4.26 publisher_visual_odometry_msg

visual_odom.visual_odom_node.VisualOdom.publisher_visual_odometry_msg

7.11.4.27 robots

visual_odom.visual_odom_node.VisualOdom.robots

7.11.4.28 subscription_depth

visual_odom.visual_odom_node.VisualOdom.subscription_depth

7.11.4.29 subscription_rgb

visual_odom.visual_odom_node.VisualOdom.subscription_rgb

7.11.4.30 subscription_wheel_odom

visual_odom.visual_odom_node.VisualOdom.subscription_wheel_odom

7.11.4.31 tf_broadcaster

visual_odom.visual_odom_node.VisualOdom.tf_broadcaster

7.11.4.32 tf_buffer

visual_odom.visual_odom_node.VisualOdom.tf_buffer

7.11.4.33 tf_listener

visual_odom.visual_odom_node.VisualOdom.tf_listener

7.11.4.34 theta

visual_odom.visual_odom_node.VisualOdom.theta

7.11.4.35 total_publisher

visual_odom.visual_odom_node.VisualOdom.total_publisher

7.11.4.36 valid_des

visual_odom.visual_odom_node.VisualOdom.valid_des

7.11.4.37 valid_des_last

visual_odom.visual_odom_node.VisualOdom.valid_des_last

7.11.4.38 valid_kp

visual_odom.visual_odom_node.VisualOdom.valid_kp

7.11.4.39 valid_kp_depth

visual_odom.visual_odom_node.VisualOdom.valid_kp_depth

7.11.4.40 valid_kp_depth_last

visual_odom.visual_odom_node.VisualOdom.valid_kp_depth_last

7.11.4.41 valid_kp_last

visual_odom.visual_odom_node.VisualOdom.valid_kp_last

7.11.4.42 visible_landmarks

visual_odom.visual_odom_node.VisualOdom.visible_landmarks

7.11.4.43 visual_odom_path

visual_odom.visual_odom_node.VisualOdom.visual_odom_path

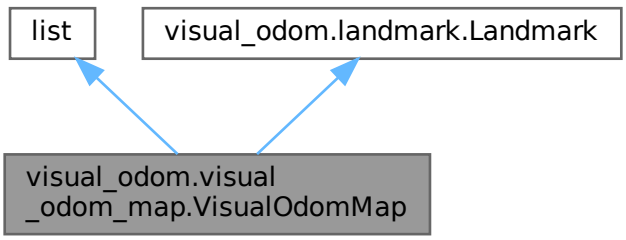
The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/visual_odom_node.py

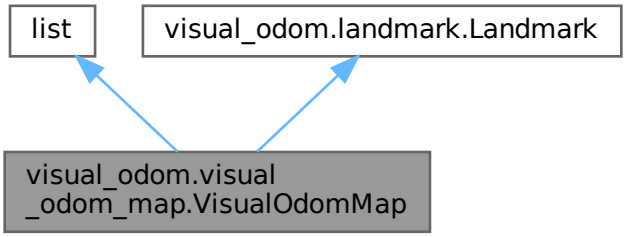
7.12 visual_odom.visual_odom_map.VisualOdomMap Class Reference

Holds and manages persistent Landmark records mapped inside the particle trajectories.

Inheritance diagram for visual_odom.visual_odom_map.VisualOdomMap:



Collaboration diagram for visual_odom.visual_odom_map.VisualOdomMap:



Public Member Functions

- None __init__ (self, Optional[Iterable[Landmark]] landmarks=None)

  Initializes a VisualOdomMap collection instance.
- Landmark get_landmark (self, int index)

  Retrieves a tracked landmark record by its indexing identifier.
- None add_landmark (self, Landmark landmark)

  Appends a newly created landmark record to the collection.
- list[Landmark] get_all_landmarks (self)

  Gets the raw list structure containing all active landmark instances.
- int get_size (self)

  Gets the total landmark population size currently inside the map.
- None add_landmarks_from_kps (self, NDArray P_init, Iterable[cv2.KeyPoint] kps, np.ndarray des, np.ndarray frame_rgb, np.ndarray kp_depth, float theta, Coordinate pos_baselink)

  Instantiates new landmarks from visual feature parameters and adds them to the map.
- VisualOdomMap get_visible_landmarks (self, Coordinate camera_pos, float theta)

  Queries the map for all landmarks falling inside the camera's visual field-of-view frustum.
- np.ndarray get_descriptors (self)

  Stack all internal visual binary descriptors as a single NumPy array for batch processing.
- list[int] get_depth (self)

  Gets raw depth values from all active landmarks in the map.
- list[int] get_u_coordinates (self)

  Gets horizontal pixel indices from all active landmarks in the map.
- list[int] get_v_coordinates (self)

  Gets vertical pixel indices from all active landmarks in the map.
- list[int] get_colors (self)

  Gets packed RGB rendering properties from all active landmarks in the map.
- list[Coordinate] get_odom_coordinates (self)

  Gets estimated odom-frame coordinates from all active landmarks in the map.
- list[Coordinate] get_kinect_coordinates (self)

  Gets local camera-frame coordinates from all active landmarks in the map.
- None publish_pointcloud_map (self, rclpy.publisher.Publisher publisher, Time time)

  Formats and publishes active landmarks as a standard PointCloud2 sensor message for RViz.
- None cleanup_old_landmarks (self, VisualOdomMap visible_landmarks, List[int] landmark_index)

  Manages landmark trust properties and purges unobserved/unstable features from memory.

- None landmark_kalman_iteration (self, Coordinate pos_baselink, float theta, List[int] landmark_indices, List[PixelCoordinate] kp_pos)

  Runs a single EKF iteration step across all actively matched landmarks.
- float calculate_log_weight (self, Coordinate pos_baselink, float theta, List[int] landmark_indices, List[PixelCoordinate] kp_pos)

  Sums the log-likelihood of all matched landmarks to update particle log weights.

Public Member Functions inherited from visual_odom.landmark.Landmark

- np.ndarray get_descriptor (self)

  Gets the binary feature descriptor array.
- float get_u (self)

  Gets the horizontal pixel coordinate.
- float get_v (self)

  Gets the vertical pixel coordinate.
- float get_trust (self)

  Gets the current visual trust tracking parameter.
- int get_color (self)

  Gets the color property used for visual representation.
- None set_odom_coordinates (self, Coordinate odom_coordinates)

  Sets the global odom-frame coordinates and updates the internal EKF.
- NDArray get_P (self)

  Gets the estimated covariance matrix.
- bool is_visible (self, Coordinate pos_camera_odom, float theta_robot)

  Checks if the landmark falls within the horizontal and vertical field of view of the sensor.
- None reset_trust (self)

  Resets the visual tracking trust property to its initial baseline value of 30.0.
- None increase_trust (self)

  Increments the trust value after the landmark is successfully re-observed.
- None decrease_trust (self)

  Applies an exponential decay to the trust value when the landmark is missed.
- float calculate_likelihood (self, Coordinate z_pos_odom, float theta)

  Calculates the measurement likelihood of the landmark given its current state.

Public Attributes

- descriptors

Public Attributes inherited from visual_odom.landmark.Landmark

- pixel_coor
- des
- trust
- color
- kinect_coordinates
- odom_coordinates
- P
- landmark_ekf

7.12.1 Detailed Description

Holds and manages persistent Landmark records mapped inside the particle trajectories.

7.12.2 Constructor & Destructor Documentation

7.12.2.1 __init__()

```
None visual_odom.visual_odom_map.VisualOdomMap.__init__ (
              self,
             Optional[Iterable[Landmark]]  landmarks = None )
```

Initializes a VisualOdomMap collection instance.

Parameters

landmarks	Optional collection of pre-existing landmark records.

Reimplemented from visual_odom.landmark.Landmark.

7.12.3 Member Function Documentation

7.12.3.1 add_landmark()

```
None visual_odom.visual_odom_map.VisualOdomMap.add_landmark (
              self,
             Landmark landmark )
```

Appends a newly created landmark record to the collection.

Parameters

landmark	Landmark target instance.

7.12.3.2 add_landmarks_from_kps()

```
None visual_odom.visual_odom_map.VisualOdomMap.add_landmarks_from_kps (
              self,
             NDArray P_init,
             Iterable[cv2.KeyPoint] kps,
             np.ndarray des,
             np.ndarray frame_rgb,
             np.ndarray kp_depth,
             float theta,
             Coordinate pos_baselink )
```

Instantiates new landmarks from visual feature parameters and adds them to the map.
Filters out spatial duplicates using binary ORB descriptor hash structures.

Parameters

P_init	Initial covariance matrix assigned to newly registered features.
kps	Detected OpenCV KeyPoint structures.
des	Binary descriptors corresponding to kps.
frame_rgb	Color reference frame image.
kp_depth	Raw sensor depth value matches.
theta	Current robot heading yaw orientation.
pos_baselink	Current robot translation position.

7.12.3.3 calculate_log_weight()

```
float visual_odom.visual_odom_map.VisualOdomMap.calculate_log_weight (
              self,
```

```
            Coordinate pos_baselink,
            float theta,
            List[int] landmark_indices,
            List[PixelCoordinate] kp_pos )
```

Sums the log-likelihood of all matched landmarks to update particle log weights.

Parameters

pos_baselink	Current position of the robot's base_link.
theta	Current robot heading yaw orientation.
landmark_indices	Map indices corresponding to matched landmarks.
kp_pos	Observed pixel coordinate measurements.

Returns

The accumulated particle log-weight float value.

7.12.3.4 cleanup_old_landmarks()

```
None visual_odom.visual_odom_map.VisualOdomMap.cleanup_old_landmarks (
             self,
            VisualOdomMap visible_landmarks,
            List[int] landmark_index )
```

Manages landmark trust properties and purges unobserved/unstable features from memory.
Increments trust metrics for actively observed features and applies decay factors to missed ones.

Parameters

visible_landmarks	Subset collection containing currently visible landmarks.
landmark_index	Vector of matched inlier indices.

7.12.3.5 get_all_landmarks()

```
list[Landmark] visual_odom.visual_odom_map.VisualOdomMap.get_all_landmarks (
             self )
```

Gets the raw list structure containing all active landmark instances.

Returns

Collection list.

7.12.3.6 get_colors()

```
list[int] visual_odom.visual_odom_map.VisualOdomMap.get_colors (
             self )
```

Gets packed RGB rendering properties from all active landmarks in the map.

Returns

Packed color integers list.

7.12.3.7 get_depth()

```
list[int] visual_odom.visual_odom_map.VisualOdomMap.get_depth (
             self )
```

Gets raw depth values from all active landmarks in the map.

Returns

Depth values list.

Reimplemented from visual_odom.landmark.Landmark.

7.12.3.8 get_descriptors()

```
np.ndarray visual_odom.visual_odom_map.VisualOdomMap.get_descriptors (
            self )
```

Stack all internal visual binary descriptors as a single NumPy array for batch processing.

Returns

2D descriptor matrix matching active landmarks.

7.12.3.9 get_kinect_coordinates()

```
list[Coordinate] visual_odom.visual_odom_map.VisualOdomMap.get_kinect_coordinates (
            self )
```

Gets local camera-frame coordinates from all active landmarks in the map.

Returns

Coordinates list.

Reimplemented from visual_odom.landmark.Landmark.

7.12.3.10 get_landmark()

```
Landmark visual_odom.visual_odom_map.VisualOdomMap.get_landmark (
            self,
           int index )
```

Retrieves a tracked landmark record by its indexing identifier.

Parameters

index	Target list position.

Returns

Tracked Landmark instance.

7.12.3.11 get_odom_coordinates()

```
list[Coordinate] visual_odom.visual_odom_map.VisualOdomMap.get_odom_coordinates (
            self )
```

Gets estimated odom-frame coordinates from all active landmarks in the map.

Returns

Coordinates list.

Reimplemented from visual_odom.landmark.Landmark.

7.12.3.12 get_size()

```
int visual_odom.visual_odom_map.VisualOdomMap.get_size (
            self )
```

Gets the total landmark population size currently inside the map.

Returns

Landmark population size.

7.12.3.13 get_u_coordinates()

```
list[int] visual_odom.visual_odom_map.VisualOdomMap.get_u_coordinates (
              self )
```

Gets horizontal pixel indices from all active landmarks in the map.

Returns

Horizontal columns list.

7.12.3.14 get_v_coordinates()

```
list[int] visual_odom.visual_odom_map.VisualOdomMap.get_v_coordinates (
              self )
```

Gets vertical pixel indices from all active landmarks in the map.

Returns

Vertical rows list.

7.12.3.15 get_visible_landmarks()

```
VisualOdomMap visual_odom.visual_odom_map.VisualOdomMap.get_visible_landmarks (
              self,
             Coordinate camera_pos,
             float theta )
```

Queries the map for all landmarks falling inside the camera's visual field-of-view frustum.

Parameters

camera_pos	Position of the camera in the global odom frame.
theta	Current robot heading yaw orientation.

Returns

New map collection containing only visible landmark candidates.

7.12.3.16 landmark_kalman_iteration()

```
None visual_odom.visual_odom_map.VisualOdomMap.landmark_kalman_iteration (
              self,
             Coordinate pos_baselink,
             float theta,
             List[int] landmark_indices,
             List[PixelCoordinate] kp_pos )
```

Runs a single EKF iteration step across all actively matched landmarks.

Parameters

pos_baselink	Current position of the robot's base_link.
theta	Current robot heading yaw orientation.
landmark_indices	Map indices corresponding to matched landmarks.
kp_pos	Observed pixel coordinate measurements.

Reimplemented from visual_odom.landmark.Landmark.

7.12.3.17 publish_pointcloud_map()

```
None visual_odom.visual_odom_map.VisualOdomMap.publish_pointcloud_map (
              self,
             rclpy.publisher.Publisher publisher,
             Time time )
```

Formats and publishes active landmarks as a standard PointCloud2 sensor message for RViz.

Parameters

publisher	Target PointCloud2 publisher channel.
time	Target timeline timestamp.

7.12.4 Member Data Documentation

7.12.4.1 descriptors

```
visual_odom.visual_odom_map.VisualOdomMap.descriptors
```

The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/visual_odom_map.py

7.13 visual_odom.robot.VisualRobotSample Class Reference

Public Member Functions

- __init__ (self, Coordinate pos, float theta, NDArray covariance_P, cv2.BFMatcher matcher, Publisher odom↩ _publisher, Publisher map_publisher, Publisher cone_publisher, Publisher total_publisher, List[cv2.KeyPoint] valid_kp, np.ndarray valid_des, np.ndarray valid_kp_depth, NDArray frame_rgb, int index, map=None)
- publish_yourself (self, rgb_stamp)

  Publishes all visualization and odometry messages for the particle.
- predict_with_wheel_odom (self, State state_wheel_odom)

  Updates the EKF prediction step using wheel odometry increments.
- robot_iteration (self, List[cv2.KeyPoint] valid_kp, np.ndarray valid_des, np.ndarray valid_kp_depth, frame_rgb, depth_frame, rgb_stamp, ransac_result)
- ransac_failed (self)
- publish_odometry_msg (self, publisher, Coordinate pos, float theta, NDArray covariance, timestamp, str parent_frame_id, str child_frame_id)
- publish_vision_cone (self, Publisher cone_publisher, Time rgb_stamp)
- init_camera_cone (self, rgb_stamp)
- publish_pointcloud_map (self, Publisher publisher, Time time)
- None publish_pixels (self, NDArray frame_depth, NDArray frame_rgb, rclpy.publisher.Publisher publisher, Time time)

  Downsamples and formats dense pixel coordinates into PointCloud2 structures.
- publish_accumulated_pixels (self, Publisher publisher, Time time)
- None accumulate_pixels (self, NDArray frame_depth, NDArray frame_rgb)

  Accumulate pixels for every robot to publish depending on best robot.
- get_drawn_keypoints (self)
- Coordinate get_position (self)
- float get_theta (self)
- NDArray get_covariance_P (self)
- float get_log_weight (self)
- None set_log_weight (self, float log_weight)

- float get_weight (self)
- None set_weight (self, float weight)
- None set_state (self, Coordinate pos, float theta)

  Sets the current robot base tracking pose.

Public Attributes

- theta
- pos_baselink
- index
- visual_odom_map
- bf
- odometry_msg_publisher
- map_publisher
- cone_publisher
- covariance_P
- total_publisher
- first_iteration
- ransac_delta_p
- ransac_inlier_ratio
- last_wheel_odom
- log_weight
- noise_Q
- ransac_draw_keypoints
- path
- accumulated_pixels
- delta_wheel_odom_ransac
- acc_ransac_noise_delta
- pos_visual_odom
- ekf
- path_color
- frame_depth
- frame_rgb
- visible_landmarks
- ransac_delta_theta
- delta_ransac_state

7.13.1 Constructor & Destructor Documentation

7.13.1.1 __init__()

```
visual_odom.robot.VisualRobotSample.__init__ (
              self,
             Coordinate pos,
             float theta,
             NDArray covariance_P,
             cv2.BFMatcher matcher,
             Publisher odom_publisher,
             Publisher map_publisher,
             Publisher cone_publisher,
             Publisher total_publisher,
             List[cv2.KeyPoint] valid_kp,
             np.ndarray valid_des,
             np.ndarray valid_kp_depth,
             NDArray frame_rgb,
             int index,
              map = None )
```

7.13.2 Member Function Documentation

7.13.2.1 accumulate_pixels()

```
None visual_odom.robot.VisualRobotSample.accumulate_pixels (
              self,
             NDArray frame_depth,
             NDArray frame_rgb )
```

Accumulate pixels for every robot to publish depending on best robot.

Parameters

frame_depth	Input depth reference image.
frame_rgb	Color reference frame image.
publisher	Target ROS 2 PointCloud2 publisher channel.
time	Active system timestamp.

7.13.2.2 get_covariance_P()

```
NDArray visual_odom.robot.VisualRobotSample.get_covariance_P (
              self )
```

```
@brief Returns the current pose covariance matrix.

@return 3x3 covariance matrix.
```

7.13.2.3 get_drawn_keypoints()

```
visual_odom.robot.VisualRobotSample.get_drawn_keypoints (
              self )
```

```
@brief Returns the keypoints used as RANSAC inliers for visualization.

@return List of matched keypoints.
```

7.13.2.4 get_log_weight()

```
float visual_odom.robot.VisualRobotSample.get_log_weight (
              self )
```

```
@brief Returns the particle log weight.

@return Current log weight value.
```

7.13.2.5 get_position()

```
Coordinate visual_odom.robot.VisualRobotSample.get_position (
              self )
```

```
@brief Returns the current robot position.

@return Current base_link position.
```

7.13.2.6 get_theta()

```
float visual_odom.robot.VisualRobotSample.get_theta (
              self )
```

```
@brief Returns the current robot heading.

@return Current yaw angle in radians.
```

7.13.2.7 get_weight()

```
float visual_odom.robot.VisualRobotSample.get_weight (
              self )

@brief Returns the particle weight in linear scale.

@return Particle weight or None if overflow occurs.
```

7.13.2.8 init_camera_cone()

```
visual_odom.robot.VisualRobotSample.init_camera_cone (
              self,
              rgb_stamp )

@brief Creates a triangular marker representing the camera vision cone.

@param rgb_stamp Timestamp used for marker synchronization.
@return ROS marker imitating the camera FOV.
```

7.13.2.9 predict_with_wheel_odom()

```
visual_odom.robot.VisualRobotSample.predict_with_wheel_odom (
              self,
             State state_wheel_odom )
```

Updates the EKF prediction step using wheel odometry increments.

Parameters

state_wheel_odom	Current wheel odometry state.

Returns

None

7.13.2.10 publish_accumulated_pixels()

```
visual_odom.robot.VisualRobotSample.publish_accumulated_pixels (
              self,
             Publisher publisher,
             Time time )
```

7.13.2.11 publish_odometry_msg()

```
visual_odom.robot.VisualRobotSample.publish_odometry_msg (
              self,
              publisher,
             Coordinate pos,
             float theta,
             NDArray covariance,
              timestamp,
             str parent_frame_id,
             str child_frame_id )

@brief Publishes the robot pose and covariance as a ROS odometry message.

@param publisher ROS publisher used to publish the odometry message.
@param pos Current robot position in odometry coordinates.
@param theta Current robot yaw orientation.
@param covariance 3x3 robot pose covariance matrix.
```

```
@param timestamp ROS timestamp for the odometry message.
@param parent_frame_id Parent coordinate frame ID.
@param child_frame_id Child coordinate frame ID.
@return None.
```

7.13.2.12 publish_pixels()

```
None visual_odom.robot.VisualRobotSample.publish_pixels (
              self,
             NDArray frame_depth,
             NDArray frame_rgb,
             rclpy.publisher.Publisher publisher,
             Time time )
```

Downsamples and formats dense pixel coordinates into PointCloud2 structures.

Parameters

frame_depth	Input depth reference image.
frame_rgb	Color reference frame image.
publisher	Target ROS 2 PointCloud2 publisher channel.
time	Active system timestamp.
frame_id	Relative spatial coordinate frame identifier.

7.13.2.13 publish_pointcloud_map()

```
visual_odom.robot.VisualRobotSample.publish_pointcloud_map (
              self,
             Publisher publisher,
             Time time )

@brief Calls function to publish active landmarks as a standard PointCloud2 sensor message for RViz.

@param publisher ROS publisher for the point cloud map.
@param time Timestamp used for point cloud synchronization.
```

7.13.2.14 publish_vision_cone()

```
visual_odom.robot.VisualRobotSample.publish_vision_cone (
              self,
             Publisher cone_publisher,
             Time rgb_stamp )

@brief Publishes the camera field-of-view cone as a visualization marker.

@param cone_publisher ROS publisher for the vision cone marker.
@param rgb_stamp Timestamp of the current RGB frame.
```

7.13.2.15 publish_yourself()

```
visual_odom.robot.VisualRobotSample.publish_yourself (
              self,
              rgb_stamp )
```

Publishes all visualization and odometry messages for the particle.

Parameters

rgb_stamp	Timestamp of the current frame.

Returns

None

7.13.2.16 ransac_failed()

```
visual_odom.robot.VisualRobotSample.ransac_failed (
              self )

@brief Fallback EKF update when RANSAC validation fails.

@return None. Updates pose, covariance, and internal state in-place.
```

7.13.2.17 robot_iteration()

```
visual_odom.robot.VisualRobotSample.robot_iteration (
              self,
             List[cv2.KeyPoint] valid_kp,
             np.ndarray valid_des,
             np.ndarray valid_kp_depth,
              frame_rgb,
              depth_frame,
              rgb_stamp,
              ransac_result )

@brief Executes one visual odometry iteration including feature matching, RANSAC validation, EKF fusion, and l

@param valid_kp List of valid ORB keypoints extracted from the current RGB frame.
@param valid_des Descriptor matrix corresponding to the detected keypoints.
@param valid_kp_depth Depth values for each detected keypoint.
@param frame_rgb Current RGB image frame.
@param depth_frame Current aligned depth image.
@param rgb_stamp ROS timestamp of the current RGB frame.
@param ransac_result Precomputed RANSAC motion estimation and inlier information.
@return None. Updates robot pose, covariance, landmark map, and trajectory in-place.
```

7.13.2.18 set_log_weight()

```
None visual_odom.robot.VisualRobotSample.set_log_weight (
              self,
             float log_weight )

@brief Sets the particle log weight.

@param log_weight New log weight value.
@return None.
```

7.13.2.19 set_state()

```
None visual_odom.robot.VisualRobotSample.set_state (
              self,
             Coordinate pos,
             float theta )
```

Sets the current robot base tracking pose.

Parameters

pos	New estimated position of the robot base.
theta	New estimated heading of the robot base.

7.13.2.20 set_weight()

```
None visual_odom.robot.VisualRobotSample.set_weight (
              self,
             float weight )
```

@brief Sets the particle weight using linear scale.

@param weight New particle weight.
@return None.

7.13.3 Member Data Documentation

7.13.3.1 acc_ransac_noise_delta

visual_odom.robot.VisualRobotSample.acc_ransac_noise_delta

7.13.3.2 accumulated_pixels

visual_odom.robot.VisualRobotSample.accumulated_pixels

7.13.3.3 bf

visual_odom.robot.VisualRobotSample.bf

7.13.3.4 cone_publisher

visual_odom.robot.VisualRobotSample.cone_publisher

7.13.3.5 covariance_P

visual_odom.robot.VisualRobotSample.covariance_P

7.13.3.6 delta_ransac_state

visual_odom.robot.VisualRobotSample.delta_ransac_state

7.13.3.7 delta_wheel_odom_ransac

visual_odom.robot.VisualRobotSample.delta_wheel_odom_ransac

7.13.3.8 ekf

visual_odom.robot.VisualRobotSample.ekf

7.13.3.9 first_iteration

visual_odom.robot.VisualRobotSample.first_iteration

7.13.3.10 frame_depth

visual_odom.robot.VisualRobotSample.frame_depth

7.13.3.11 frame_rgb

visual_odom.robot.VisualRobotSample.frame_rgb

7.13.3.12 index

visual_odom.robot.VisualRobotSample.index

7.13.3.13 last_wheel_odom

visual_odom.robot.VisualRobotSample.last_wheel_odom

7.13.3.14 log_weight

visual_odom.robot.VisualRobotSample.log_weight

7.13.3.15 map_publisher

visual_odom.robot.VisualRobotSample.map_publisher

7.13.3.16 noise_Q

visual_odom.robot.VisualRobotSample.noise_Q

7.13.3.17 odometry_msg_publisher

visual_odom.robot.VisualRobotSample.odometry_msg_publisher

7.13.3.18 path

visual_odom.robot.VisualRobotSample.path

7.13.3.19 path_color

visual_odom.robot.VisualRobotSample.path_color

7.13.3.20 pos_baselink

visual_odom.robot.VisualRobotSample.pos_baselink

7.13.3.21 pos_visual_odom

visual_odom.robot.VisualRobotSample.pos_visual_odom

7.13.3.22 ransac_delta_p

visual_odom.robot.VisualRobotSample.ransac_delta_p

7.13.3.23 ransac_delta_theta

visual_odom.robot.VisualRobotSample.ransac_delta_theta

7.13.3.24 ransac_draw_keypoints

visual_odom.robot.VisualRobotSample.ransac_draw_keypoints

7.13.3.25 ransac_inlier_ratio

visual_odom.robot.VisualRobotSample.ransac_inlier_ratio

7.13.3.26 theta

visual_odom.robot.VisualRobotSample.theta

7.13.3.27 total_publisher

visual_odom.robot.VisualRobotSample.total_publisher

7.13.3.28 visible_landmarks

`visual_odom.robot.VisualRobotSample.visible_landmarks`

7.13.3.29 visual_odom_map

`visual_odom.robot.VisualRobotSample.visual_odom_map`

The documentation for this class was generated from the following file:

- ros2_ws/src/visual_odom/visual_odom/robot.py

Chapter 8

File Documentation

8.1 bagfiles/20260428_Bag_Without_TF/metadata.yaml File Reference

Namespaces

- namespace metadata

Variables

- metadata.rosbag2_bagfile_information :
- int metadata.version : 9
- mcap metadata.storage_identifier
- metadata.duration :
- int metadata.nanoseconds : 86772902393
- metadata.starting_time :
- int metadata.nanoseconds_since_epoch : 1777312764407083438
- int metadata.message_count : 16425
- metadata.topics_with_message_count :
- metadata.name : /serf01/nav_rgbd_1/depth/image_raw
- sensor_msgs metadata.type /msg/Image
- cdr metadata.serialization_format
- metadata.offered_qos_profiles :
- int metadata.depth : 0
- reliable metadata.reliability
- volatile metadata.durability
- metadata.deadline :
- int metadata.sec : 9223372036
- int metadata.nsec : 854775807
- metadata.lifespan :
- automatic metadata.liveliness
- metadata.liveliness_lease_duration :
- false metadata.avoid_ros_namespace_conventions
- RIHS01_d31d41a9a4c4bc8eae9be757b0beed306564f7526c88ea6a4588fb9582527d47 metadata.type_description_hash
- str metadata.compression_format : ""
- str metadata.compression_mode : ""
- metadata.relative_file_paths :
- metadata.files :
- metadata.custom_data : ~
- jazzy metadata.ros_distro

8.2 bagfiles/20260519_TransXY_bag/metadata.yaml File Reference

Namespaces

- namespace metadata

8.3 bagfiles/20260616_Around_The_Desks_bag/metadata.yaml File Reference

Namespaces

- namespace metadata

8.4 README.md File Reference

8.5 ros2_ws/src/visual_odom/config/param.yaml File Reference

Namespaces

- namespace param
- namespace visual_odom.config

  Central configuration file for the runtime parameters of the visual_odom_node.

Variables

- param.visual_odom_node :
- param.ros__parameters :
- int param.max_depth : 7000

  Maximum reliable sensor depth of the Kinect camera in millimeters (mm).
- float param.rgb_depth_sync_tolerance_sec : 0.1

  Allowed temporal deviation when synchronizing RGB and depth images (in seconds).
- int param.pixel_tolerance : 4

  Pixel tolerance range in image space for spatial deduplication of neighboring keypoints.
- int param.matches_for_new_landmarks : 500

  Match threshold below which new landmarks are added to the map.
- float param.min_landmark_trust : 15.0

  Minimum trust value a landmark must maintain before being removed from the map.
- int param.n_robot_samples : 40

  Number of active robot particle samples within the Rao-Blackwellized particle filter.
- param.ransac :
- float param.evaluation_tolerance : 0.05

  Allowed Euclidean distance deviation in meters (m) for classifying a point as an inlier.
- int param.iterations : 30

  Maximum number of iterations for hypothesis evaluation in the RANSAC algorithm.
- int param.sample_size : 3

  Minimum sample size (point pairs) per iteration for computing a transformation hypothesis.
- int param.min_matches_for_ransac : 10

  Minimum number of feature matches required before starting RANSAC optimization.
- float param.max_rotation_angle_deg : 15.0

  Maximum physically plausible robot rotation per frame in degrees (deg).
- float param.min_inlier_ratio : 0.2

  Minimum required ratio of inliers to total matches for accepting a RANSAC result.
- str param.rosbag_path : "bagfiles/20260616_Around_The_Desks_bag/rosbag2_2026_06_16-13_12_13_↩0.mcap"

  Absolute path to the MCAP rosbag file to be recorded or played back.

8.6 ros2_ws/src/visual_odom/launch/visual_odom_launch.py File Reference

Namespaces

- namespace visual_odom_launch

Functions

- visual_odom_launch.generate_launch_description ()

8.7 ros2_ws/src/visual_odom/setup.py File Reference

Namespaces

- namespace setup

Variables

- str setup.package_name = 'visual_odom'
- setup.name
- setup.version
- setup.packages
- setup.data_files
- setup.install_requires
- setup.zip_safe
- setup.maintainer
- setup.maintainer_email
- setup.description
- setup.license
- setup.extras_require
- setup.entry_points

8.8 ros2_ws/src/visual_odom/visual_odom/__init__.py File Reference

8.9 ros2_ws/src/visual_odom/visual_odom/constants.py File Reference

Classes

- class visual_odom.constants.Coordinate

  3D spatial coordinate container.
- class visual_odom.constants.PixelCoordinate

  Container for a 2D image pixel and its depth value.
- class visual_odom.constants.State

  2D robot pose representation.
- class visual_odom.constants.Parameters

  Runtime configuration and threshold parameter package for the SLAM node.

Namespaces

- namespace visual_odom
- namespace visual_odom.constants

  Global configuration, camera model, ROS topic definitions and helper datatypes used by the visual odometry system.

Functions

- float visual_odom.constants.normalize_angle (float angle)

  Normalize an angle to the range [-pi, pi].

Variables

- str visual_odom.constants.VISUAL_ODOM_MSG_TOPIC = "/serf01/odometry/project_slam"

  Primary ROS 2 publisher output topic for navigation odometry.
- str visual_odom.constants.RGB_IMAGE_TOPIC = "/serf01/nav_rgbd_1/rgb/image_raw"

  ROS 2 topic for raw RGB image input.
- str visual_odom.constants.DEPTH_IMAGE_TOPIC = "/serf01/nav_rgbd_1/depth/image_raw"

  ROS 2 topic for raw depth image input.
- str visual_odom.constants.WHEEL_ODOMETRY_TOPIC = "/serf01/odometry/wheel"

  ROS 2 topic for wheel odometry input.
- str visual_odom.constants.FILTERED_ODOMETRY_TOPIC = "/serf01/odometry/filtered"

  ROS 2 topic for filtered odometry.
- str visual_odom.constants.IMU_ODOMETRY_TOPIC = "/serf01/odometry/imu"

  ROS 2 topic for IMU-based odometry input.
- str visual_odom.constants.TF_STATIC_TOPIC = "/tf_static"

  ROS 2 topic for static TF transforms.
- str visual_odom.constants.VISION_CONE_TOPIC = "/serf01/camera/vision_cone"

  ROS 2 topic for publishing or visualizing the camera vision cone.
- str visual_odom.constants.KP_IMAGE_TOPIC = "/serf01/camera/keypoint_image"

  ROS 2 topic for publishing images with visualized keypoints.
- str visual_odom.constants.VISUAL_ODOM_PATH_TOPIC = "/serf01/visual_odom_path"

  ROS 2 topic for publishing paths of the particles.
- str visual_odom.constants.POINTCLOUD_FRAME_TOPIC = "/serf01/pointcloud/points_3d"

  Frame/topic identifier for the structural environment point cloud.
- str visual_odom.constants.ACCUMULATED_POINTCLOUD_FRAME_TOPIC = "/serf01/pointcloud/accumulated↩_points_3d"

  Frame/topic identifier for the accumulated environment point cloud.
- str visual_odom.constants.KEYPOINT_POINTCLOUD_FRAME_TOPIC = "/serf01/pointcloud/keypoint_3d"

  Frame/topic identifier for tracked ORB descriptor 3D keypoints.
- str visual_odom.constants.KINECT_FRAME_ID = "kinect_depth"

  TF frame identifier for the optical center of the RGB-D camera.
- str visual_odom.constants.BASE_LINK_FRAME_ID = "base_link"

  TF frame identifier for the physical center of the robot platform.
- str visual_odom.constants.VISUAL_ODOM_FRAME_ID = "odom"

  TF frame identifier for the fixed global odometry origin.
- float visual_odom.constants.F = 526.61

  Horizontal and vertical focal length parameter of the Kinect optical model.
- float visual_odom.constants.CU = 318.525

  Horizontal pixel coordinate of the camera principal point.
- float visual_odom.constants.CV = 241.181

  Vertical pixel coordinate of the camera principal point.
- int visual_odom.constants.MAX_U = 640

  Horizontal image plane resolution in pixels.
- int visual_odom.constants.MAX_V = 480

  Vertical image plane resolution in pixels.
- visual_odom.constants.MAX_AZIMUTH = abs(atan2(CU, F))

  Derived horizontal angle limit matching the sensor field of view.

- visual_odom.constants.MAX_ALTITUDE = abs(atan2(CV, F))

  Derived vertical angle limit matching the sensor field of view.
- visual_odom.constants.CAMERA_POS_IN_BASELINK = Coordinate(0.1, 0.0, 0.75)

  Fixed physical camera offset relative to base_link.
- int visual_odom.constants.CAMERA_ANGLE_VER_RAD = 43 ∗ pi / 180

  Vertical field of view angle of the RGB-D camera in radians.
- int visual_odom.constants.CAMERA_ANGLE_HOR_RAD = 57 ∗ pi / 180

  Horizontal field of view angle of the RGB-D camera in radians.
- int visual_odom.constants.RESOLUTION_CONE_H = 40

  Horizontal discretization resolution of the camera vision cone.
- int visual_odom.constants.RESOLUTION_CONE_V = 30

  Vertical discretization resolution of the camera vision cone.
- visual_odom.constants.ROT_BK

  Rotation matrix from Kinect optical frame to ROS base_link frame.
- int visual_odom.constants.MIN_DEPTH = 400

  Minimum reliable depth value in millimeters.
- int visual_odom.constants.MAX_DEPTH = 7000

  Maximum reliable depth value in millimeters.
- float visual_odom.constants.RANSAC_EVALUATION_TOLERANCE = 0.045

  Inlier distance threshold for evaluating RANSAC hypotheses in meters.
- int visual_odom.constants.RANSAC_ITERATION = 100

  Maximum number of RANSAC iterations.
- int visual_odom.constants.RANSAC_SAMPLE_SIZE = 3

  Number of matched point pairs used for one RANSAC hypothesis.
- float visual_odom.constants.RANSAC_MIN_INLIER_RATIO = 0.3

  Minimum inlier ratio required to accept a RANSAC solution.
- int visual_odom.constants.MIN_MATCHES_FOR_RANSAC = 15

  Minimum number of feature matches required to start RANSAC.
- float visual_odom.constants.MAX_ROTATION_ANGLE_DEG = 15.0

  Maximum plausible robot rotation per frame step in degrees.
- float visual_odom.constants.RGB_DEPTH_SYNC_TOLERANCE_SEC = 0.05

  Maximum allowed temporal difference between RGB and depth frames in seconds.
- int visual_odom.constants.PIXEL_TOLERANCE = 8

  Pixel-space search radius used for visual match validation.
- int visual_odom.constants.N_ROBOT_SAMPLES = 1000

  Default number of particles/samples used for localization.
- float visual_odom.constants.NOISE_INCREMENT_RANSAC_FAILURE = 0.002

  Noise increment added after a failed RANSAC estimation.
- float visual_odom.constants.MAX_ADDITIONAL_NOISE_RANSAC_FAILURE = 0.22

  Maximum allowed additional noise caused by repeated RANSAC failures.
- int visual_odom.constants.MATCHES_FOR_NEW_LANDMARKS = 50

  Minimum tracking count required to stabilize a new landmark.
- int visual_odom.constants.INITIAL_LANDMARK_COUNT = 260

  Initial landmark count threshold used for map bootstrapping.
- float visual_odom.constants.MIN_LANDMARK_TRUST = 20.0

  Minimum trust value before a landmark is considered unreliable.
- float visual_odom.constants.INCREASE_TRUST_VALUE = 10.0

  Trust value added to a landmark after successful observation.
- float visual_odom.constants.DECREASE_TRUST_FACTOR = 0.95

  Multiplicative trust decay factor for unobserved landmarks.
- float visual_odom.constants.SIGMA_X_ODOM_WHEEL_Q = 0.05

Wheel odometry process noise standard deviation in x-direction.

- float visual_odom.constants.SIGMA_Y_ODOM_WHEEL_Q = 0.05

  Wheel odometry process noise standard deviation in y-direction.
- float visual_odom.constants.SIGMA_THETA_ODOM_WHEEL_Q = 0.03

  Wheel odometry process noise standard deviation for heading angle.
- float visual_odom.constants.SIGMA_PIXEL = 0.8 / 3

  Pixel projection noise coefficient for image-plane coordinates.
- float visual_odom.constants.ERROR_MIN_DEPTH = 0.001477

  Base depth measurement noise offset at closest sensor range in meters.
- float visual_odom.constants.ERROR_QUADRATIC_DEPTH = 0.002294

  Quadratic depth error factor modeling RGB-D precision degradation.
- int visual_odom.constants.MIN_DET_VALUE = 1e-12

  Small epsilon cutoff used to avoid determinant singularities.
- float visual_odom.constants.GAUSS_NOISE_X_SIGMA = 0.006

  Gaussian noise x-coordinate in meters.
- float visual_odom.constants.GAUSS_NOISE_Y_SIGMA = 0.006

  Gaussian noise y-coordinate in meters.
- float visual_odom.constants.GAUSS_NOISE_THETA_SIGMA = 0.003

  Gaussian noise heading angle in radians.
- float visual_odom.constants.NOISE_TRANSLATION_THRESHOLD = 0.10

  Threshold for accumulated translation noise before adding random perturbation to the robot state in meters.
- float visual_odom.constants.NOISE_ROTATION_THRESHOLD = 0.06

  Threshold for accumulated rotation noise before adding random perturbation to the robot state in radians.
- float visual_odom.constants.RESAMPLE_THETA_TOLERANCE = 4.0

  Tolerance for resampling based on heading angle difference in degrees.
- float visual_odom.constants.RESAMPLE_TIME_TOLERANCE = 40.0

  Time tolerance for resampling based on elapsed time since the first frame in seconds.
- float visual_odom.constants.RESAMPLE_POS_TOLERANCE = 0.35

  Position tolerance for resampling based on Euclidean distance in meters.
- Parameters visual_odom.constants.parameters = None

  Global runtime parameter instance populated at node startup.
- float visual_odom.constants.PERCENTAGE_OF_PLAY_SPEED = 0.1

8.10 ros2_ws/src/visual_odom/visual_odom/csv_creator_node.py File Reference

Classes

- class visual_odom.csv_creator_node.OdomImuCsvWriter

  ROS 2 Node responsible for subscribing to localization data topics and logging state measurements into a unified CSV file for analysis.

Namespaces

- namespace visual_odom
- namespace visual_odom.csv_creator_node

  ROS 2 node that logs odometry and IMU measurements to a unified CSV file.

Functions

- visual_odom.csv_creator_node.main ()

  Starts the infrastructure runtime framework layer, instantiates OdomImuCsvWriter execution loops, and ensures safe process cleanups.

8.11 ros2_ws/src/visual_odom/visual_odom/ekf_landmark.py File Reference

Classes

- class visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark

  Extended Kalman Filter for tracking an individual 3D landmark in the global odom frame.

Namespaces

- namespace visual_odom
- namespace visual_odom.ekf_landmark

  Extended Kalman Filter implementation for tracking individual landmark positions.

8.12 ros2_ws/src/visual_odom/visual_odom/ekf_robot.py File Reference

Classes

- class visual_odom.ekf_robot.ExtendedKalmanFilterRobot

  Extended Kalman Filter tracking the 2D planar robot base pose (x, y, theta).

Namespaces

- namespace visual_odom
- namespace visual_odom.ekf_robot

  Extended Kalman Filter for estimating robot pose using wheel odometry and RANSAC visual updates.

8.13 ros2_ws/src/visual_odom/visual_odom/landmark.py File Reference

Classes

- class visual_odom.landmark.Landmark

  Landmark class for storing visual features and performing spatial estimation.

Namespaces

- namespace visual_odom
- namespace visual_odom.landmark

  Defines the Landmark data class and the Kabsch optimal 2D rigid alignment algorithm.

Functions

- Tuple[np.ndarray, np.ndarray, float] visual_odom.landmark.kabsch (np.ndarray P_i, np.ndarray Q_i, float max_rotation_angle_deg=15.0)

  Computes the optimal 2D rotation and translation to align point set Q onto set P.

8.14 ros2_ws/src/visual_odom/visual_odom/odom_buffer.py File Reference

Classes

- class visual_odom.odom_buffer.odom_message
- class visual_odom.odom_buffer.OdomBuffer

Namespaces

- namespace visual_odom
- namespace visual_odom.odom_buffer

8.15 ros2_ws/src/visual_odom/visual_odom/robot.py File Reference

Classes

- class visual_odom.robot.VisualRobotSample

Namespaces

- namespace visual_odom
- namespace visual_odom.robot

8.16 ros2_ws/src/visual_odom/visual_odom/tf_methods.py File Reference

Namespaces

- namespace visual_odom
- namespace visual_odom.tf_methods

  Stateless coordinate frame transformation utilities.

Functions

- Coordinate visual_odom.tf_methods.kinect_depth_to_odom (Coordinate kinect_point, float theta, Coordinate pos_baselink)

  Transform a point from the Kinect frame into the global odom frame using the LATEST available robot state.
- Coordinate visual_odom.tf_methods.kinect_depth_to_baselink (Coordinate kinect_point)

  Transform a point from the Kinect optical frame into the robot base_link frame using static calibrations.
- Coordinate visual_odom.tf_methods.odom_to_baselink (Coordinate odom_point, float theta, Coordinate pos_baselink)

  Transform a point from the global odom frame back into the local base_link frame.
- Coordinate visual_odom.tf_methods.pixel_to_kinect (PixelCoordinate kp)

  Calculate 3D coordinates from pixel indices and its registered depth values.
- NDArray visual_odom.tf_methods.pixel_to_kinect_ (int u, int v, float z)

  Calculate 3D coordinates from raw pixel indices and depth values directly to a NumPy representation.
- TransformStamped visual_odom.tf_methods.calculate_tf (Coordinate Position, float theta, Time timestamp, str parent_frame_id, str child_frame_id)

  Formats a ROS 2 coordinate transform stamped message.

8.17 ros2_ws/src/visual_odom/visual_odom/visual_odom_map.py File Reference

Classes

- class visual_odom.visual_odom_map.VisualOdomMap

  Holds and manages persistent Landmark records mapped inside the particle trajectories.

Namespaces

- namespace visual_odom
- namespace visual_odom.visual_odom_map

  Implements a persistent spatial landmark collection container tracking mapped environment features.

8.18 ros2_ws/src/visual_odom/visual_odom/visual_odom_node.py File Reference

Classes

- class visual_odom.visual_odom_node.VisualOdom

  ROS 2 Visual Odometry node managing features extraction, RANSAC estimation, and particle weight selection.

Namespaces

- namespace visual_odom
- namespace visual_odom.visual_odom_node

  Main ROS 2 execution node managing the particle filter localization and visual odometry loop.

Functions

- visual_odom.visual_odom_node.create_particle_path_markers (self, List[VisualRobotSample] particles, int best_particle_idx)

  Generates visualization markers for particle trajectories.
- visual_odom.visual_odom_node.main ()

  Core node runtime initialization routine.

Index

__add__
visual_odom.constants.Coordinate, 39
visual_odom.constants.State, 65
__init__
visual_odom.csv_creator_node.OdomImuCsvWriter, 59
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 42
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 48
visual_odom.landmark.Landmark, 52
visual_odom.odom_buffer.OdomBuffer, 57
visual_odom.robot.VisualRobotSample, 83
visual_odom.visual_odom_map.VisualOdomMap, 78
visual_odom.visual_odom_node.VisualOdom, 68
__mul__
visual_odom.constants.Coordinate, 40
visual_odom.constants.State, 65
__sub__
visual_odom.constants.Coordinate, 40
visual_odom.constants.State, 65
__truediv__
visual_odom.constants.Coordinate, 40
visual_odom.constants.State, 65
_stamp_to_sec
visual_odom.visual_odom_node.VisualOdom, 69

acc_ransac_noise_delta
visual_odom.robot.VisualRobotSample, 88
accumulate_pixels
visual_odom.robot.VisualRobotSample, 84
accumulated_pixels
visual_odom.robot.VisualRobotSample, 88
ACCUMULATED_POINTCLOUD_FRAME_TOPIC
visual_odom.constants, 27
add_landmark
visual_odom.visual_odom_map.VisualOdomMap, 78
add_landmarks_from_kps
visual_odom.visual_odom_map.VisualOdomMap, 78
add_noise_to_R
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 48
add_odom_message
visual_odom.odom_buffer.OdomBuffer, 57
additional_noise
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 50
avoid_ros_namespace_conventions
metadata, 17

bagfiles/20260428_Bag_Without_TF/metadata.yaml, 91
bagfiles/20260519_TransXY_bag/metadata.yaml, 92
bagfiles/20260616_Around_The_Desks_bag/metadata.yaml, 92
BASE_LINK_FRAME_ID
visual_odom.constants, 27
best_robot
visual_odom.visual_odom_node.VisualOdom, 72
best_robot_idx
visual_odom.visual_odom_node.VisualOdom, 72
best_weight
visual_odom.visual_odom_node.VisualOdom, 72
bf
visual_odom.robot.VisualRobotSample, 88
visual_odom.visual_odom_node.VisualOdom, 72
bridge
visual_odom.visual_odom_node.VisualOdom, 72

calculate_jacobian_F
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 42
calculate_jacobian_H
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 42
calculate_likelihood
visual_odom.landmark.Landmark, 52
calculate_log_weight
visual_odom.visual_odom_map.VisualOdomMap, 78
calculate_Q_matrix
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 43
calculate_R_sigma_matrix
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 43
calculate_tf
visual_odom.tf_methods, 35
CAMERA_ANGLE_HOR_RAD
visual_odom.constants, 27
CAMERA_ANGLE_VER_RAD
visual_odom.constants, 27
CAMERA_POS_IN_BASELINK
visual_odom.constants, 27
cleanup_old_landmarks
visual_odom.visual_odom_map.VisualOdomMap, 79
color

visual_odom.landmark.Landmark, 55
compression_format
metadata, 17
compression_mode
metadata, 18
computeKalmanGain
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 43
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 48
cone_publisher
visual_odom.robot.VisualRobotSample, 88
covariance_P
visual_odom.robot.VisualRobotSample, 88
visual_odom.visual_odom_node.VisualOdom, 72
create_particle_path_markers
visual_odom.visual_odom_node, 37
csv_file
visual_odom.csv_creator_node.OdomImuCsvWriter, 61
csv_path
visual_odom.csv_creator_node.OdomImuCsvWriter, 61
CU
visual_odom.constants, 27
custom_data
metadata, 18
CV
visual_odom.constants, 27

data_files
setup, 22
deadline
metadata, 18
decrease_trust
visual_odom.landmark.Landmark, 53
DECREASE_TRUST_FACTOR
visual_odom.constants, 27
delta_ransac_state
visual_odom.robot.VisualRobotSample, 88
delta_wheel_odom_ransac
visual_odom.robot.VisualRobotSample, 88
depth
metadata, 18
DEPTH_IMAGE_TOPIC
visual_odom.constants, 27
des
visual_odom.landmark.Landmark, 55
description
setup, 22
descriptors
visual_odom.visual_odom_map.VisualOdomMap, 82
destroy_node
visual_odom.csv_creator_node.OdomImuCsvWriter, 59
durability
metadata, 18
duration
metadata, 18

ekf
visual_odom.robot.VisualRobotSample, 88
entry_points
setup, 22
ERROR_MIN_DEPTH
visual_odom.constants, 28
ERROR_QUADRATIC_DEPTH
visual_odom.constants, 28
evaluation_tolerance
param, 20
extras_require
setup, 22

F
visual_odom.constants, 28
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 50
files
metadata, 18
FILTERED_ODOMETRY_TOPIC
visual_odom.constants, 28
first_des
visual_odom.visual_odom_node.VisualOdom, 73
first_frame_stamp
visual_odom.visual_odom_node.VisualOdom, 73
first_iteration
visual_odom.robot.VisualRobotSample, 88
visual_odom.visual_odom_node.VisualOdom, 73
first_kp
visual_odom.visual_odom_node.VisualOdom, 73
first_kp_depth
visual_odom.visual_odom_node.VisualOdom, 73
first_pos
visual_odom.visual_odom_node.VisualOdom, 73
first_theta
visual_odom.visual_odom_node.VisualOdom, 73
frame_depth
visual_odom.robot.VisualRobotSample, 88
visual_odom.visual_odom_node.VisualOdom, 73
frame_depth_stamp
visual_odom.visual_odom_node.VisualOdom, 73
frame_rgb
visual_odom.robot.VisualRobotSample, 88
visual_odom.visual_odom_node.VisualOdom, 73
frame_stamp
visual_odom.visual_odom_node.VisualOdom, 73

GAUSS_NOISE_THETA_SIGMA
visual_odom.constants, 28
GAUSS_NOISE_X_SIGMA
visual_odom.constants, 28
GAUSS_NOISE_Y_SIGMA
visual_odom.constants, 28
generate_launch_description
visual_odom_launch, 38
get_all_landmarks

visual_odom.visual_odom_map.VisualOdomMap, 79
get_color
visual_odom.landmark.Landmark, 53
get_colors
visual_odom.visual_odom_map.VisualOdomMap, 79
get_covariance_P
visual_odom.robot.VisualRobotSample, 84
get_covariance_p
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 49
get_depth
visual_odom.landmark.Landmark, 53
visual_odom.visual_odom_map.VisualOdomMap, 79
get_descriptor
visual_odom.landmark.Landmark, 53
get_descriptors
visual_odom.visual_odom_map.VisualOdomMap, 80
get_drawn_keypoints
visual_odom.robot.VisualRobotSample, 84
get_interpolated
visual_odom.odom_buffer.OdomBuffer, 57
get_kinect_coordinates
visual_odom.landmark.Landmark, 53
visual_odom.visual_odom_map.VisualOdomMap, 80
get_landmark
visual_odom.visual_odom_map.VisualOdomMap, 80
get_latest
visual_odom.odom_buffer.OdomBuffer, 57
get_log_weight
visual_odom.robot.VisualRobotSample, 84
get_odom_coordinates
visual_odom.landmark.Landmark, 53
visual_odom.visual_odom_map.VisualOdomMap, 80
get_P
visual_odom.landmark.Landmark, 53
get_position
visual_odom.robot.VisualRobotSample, 84
get_R
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 43
get_size
visual_odom.visual_odom_map.VisualOdomMap, 80
get_state
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 49
get_theta
visual_odom.robot.VisualRobotSample, 84
get_trust
visual_odom.landmark.Landmark, 54
get_u
visual_odom.landmark.Landmark, 54
get_u_coordinates
visual_odom.visual_odom_map.VisualOdomMap, 81
get_v
visual_odom.landmark.Landmark, 54
get_v_coordinates
visual_odom.visual_odom_map.VisualOdomMap, 81
get_visible_landmarks
visual_odom.visual_odom_map.VisualOdomMap, 81
get_weight
visual_odom.robot.VisualRobotSample, 84

H
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 50

img_to_kp_des_filtered
visual_odom.visual_odom_node.VisualOdom, 69
imu_callback
visual_odom.csv_creator_node.OdomImuCsvWriter, 59, 61
IMU_ODOMETRY_TOPIC
visual_odom.constants, 28
imu_theta
visual_odom.csv_creator_node.OdomImuCsvWriter, 61
increase_trust
visual_odom.landmark.Landmark, 54
INCREASE_TRUST_VALUE
visual_odom.constants, 28
index
visual_odom.robot.VisualRobotSample, 88
init_camera_cone
visual_odom.robot.VisualRobotSample, 85
INITIAL_LANDMARK_COUNT
visual_odom.constants, 28
install_requires
setup, 22
is_visible
visual_odom.landmark.Landmark, 54
iteration
visual_odom.visual_odom_node.VisualOdom, 69
iterations
param, 20

JF
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 47
JH
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 47

kabsch
visual_odom.landmark, 34
KEYPOINT_POINTCLOUD_FRAME_TOPIC
visual_odom.constants, 28

kinect_coordinates
visual_odom.landmark.Landmark, 55
kinect_depth_to_baselink
visual_odom.tf_methods, 35
kinect_depth_to_odom
visual_odom.tf_methods, 35
KINECT_FRAME_ID
visual_odom.constants, 28
KP_IMAGE_TOPIC
visual_odom.constants, 29

landmark_ekf
visual_odom.landmark.Landmark, 55
landmark_kalman_iteration
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 44
visual_odom.landmark.Landmark, 55
visual_odom.visual_odom_map.VisualOdomMap, 81
last_wheel_odom
visual_odom.robot.VisualRobotSample, 88
license
setup, 22
lifespan
metadata, 18
listener_depth_callback
visual_odom.visual_odom_node.VisualOdom, 70
listener_rgb_callback
visual_odom.visual_odom_node.VisualOdom, 70
listener_wheel_odom_callback
visual_odom.visual_odom_node.VisualOdom, 70
liveliness
metadata, 18
liveliness_lease_duration
metadata, 18
log_weight
visual_odom.robot.VisualRobotSample, 89

main
visual_odom.csv_creator_node, 33
visual_odom.visual_odom_node, 37
maintainer
setup, 22
maintainer_email
setup, 22
map_publisher
visual_odom.robot.VisualRobotSample, 89
MATCHES_FOR_NEW_LANDMARKS
visual_odom.constants, 29
matches_for_new_landmarks
param, 20
visual_odom.constants.Parameters, 62
MAX_ADDITIONAL_NOISE_RANSAC_FAILURE
visual_odom.constants, 29
MAX_ALTITUDE
visual_odom.constants, 29
MAX_AZIMUTH
visual_odom.constants, 29
MAX_DEPTH
visual_odom.constants, 29
max_depth
param, 20
visual_odom.constants.Parameters, 62
MAX_ROTATION_ANGLE_DEG
visual_odom.constants, 29
max_rotation_angle_deg
param, 20
visual_odom.constants.Parameters, 62
MAX_U
visual_odom.constants, 29
MAX_V
visual_odom.constants, 29
meas_func
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 44
message_count
metadata, 18
metadata, 17
avoid_ros_namespace_conventions, 17
compression_format, 17
compression_mode, 18
custom_data, 18
deadline, 18
depth, 18
durability, 18
duration, 18
files, 18
lifespan, 18
liveliness, 18
liveliness_lease_duration, 18
message_count, 18
name, 18
nanoseconds, 18
nanoseconds_since_epoch, 18
nsec, 18
offered_qos_profiles, 19
relative_file_paths, 19
reliability, 19
ros_distro, 19
rosbag2_bagfile_information, 19
sec, 19
serialization_format, 19
starting_time, 19
storage_identifier, 19
topics_with_message_count, 19
type, 19
type_description_hash, 19
version, 19
MIN_DEPTH
visual_odom.constants, 29
MIN_DET_VALUE
visual_odom.constants, 29
min_inlier_ratio
param, 21
MIN_LANDMARK_TRUST
visual_odom.constants, 29
min_landmark_trust

param, 21
visual_odom.constants.Parameters, 63
MIN_MATCHES_FOR_RANSAC
visual_odom.constants, 30
min_matches_for_ransac
param, 21
visual_odom.constants.Parameters, 63
msg
visual_odom.odom_buffer.odom_message, 56

N_ROBOT_SAMPLES
visual_odom.constants, 30
n_robot_samples
param, 21
visual_odom.constants.Parameters, 63
name
metadata, 18
setup, 22
nanoseconds
metadata, 18
nanoseconds_since_epoch
metadata, 18
NOISE_INCREMENT_RANSAC_FAILURE
visual_odom.constants, 30
noise_Q
visual_odom.robot.VisualRobotSample, 89
NOISE_ROTATION_THRESHOLD
visual_odom.constants, 30
NOISE_TRANSLATION_THRESHOLD
visual_odom.constants, 30
normalize_angle
visual_odom.constants, 27
visual_odom.csv_creator_node.OdomImuCsvWriter, 59
nsec
metadata, 18

Odom_Buffer
visual_odom.odom_buffer.OdomBuffer, 57
odom_callback
visual_odom.csv_creator_node.OdomImuCsvWriter, 60, 61
odom_coordinates
visual_odom.landmark.Landmark, 55
odom_to_baselink
visual_odom.tf_methods, 36
odom_visual_theta
visual_odom.csv_creator_node.OdomImuCsvWriter, 61
odom_visual_x
visual_odom.csv_creator_node.OdomImuCsvWriter, 61
odom_visual_y
visual_odom.csv_creator_node.OdomImuCsvWriter, 61
odometry_msg_publisher
visual_odom.robot.VisualRobotSample, 89
offered_qos_profiles
metadata, 19
orb
visual_odom.visual_odom_node.VisualOdom, 73

P
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 47
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 50
visual_odom.landmark.Landmark, 56
package_name
setup, 22
packages
setup, 22
param, 20
evaluation_tolerance, 20
iterations, 20
matches_for_new_landmarks, 20
max_depth, 20
max_rotation_angle_deg, 20
min_inlier_ratio, 21
min_landmark_trust, 21
min_matches_for_ransac, 21
n_robot_samples, 21
pixel_tolerance, 21
ransac, 21
rgb_depth_sync_tolerance_sec, 21
ros__parameters, 21
rosbag_path, 21
sample_size, 21
visual_odom_node, 21
parameter_callback
visual_odom.visual_odom_node.VisualOdom, 70, 73
parameter_initialization
visual_odom.visual_odom_node.VisualOdom, 70
parameters
visual_odom.constants, 30
path
visual_odom.robot.VisualRobotSample, 89
path_color
visual_odom.robot.VisualRobotSample, 89
PERCENTAGE_OF_PLAY_SPEED
visual_odom.constants, 30
pixel_coor
visual_odom.landmark.Landmark, 56
pixel_to_kinect
visual_odom.tf_methods, 36
pixel_to_kinect_
visual_odom.tf_methods, 36
PIXEL_TOLERANCE
visual_odom.constants, 30
pixel_tolerance
param, 21
visual_odom.constants.Parameters, 63
POINTCLOUD_FRAME_TOPIC
visual_odom.constants, 30
pos_baselink
visual_odom.robot.VisualRobotSample, 89
visual_odom.visual_odom_node.VisualOdom, 73

pos_visual_odom
visual_odom.robot.VisualRobotSample, 89
pose
visual_odom.odom_buffer.odom_message, 56
position_initialized
visual_odom.visual_odom_node.VisualOdom, 73
possible_resample
visual_odom.visual_odom_node.VisualOdom, 71
predict_with_wheel_odom
visual_odom.robot.VisualRobotSample, 85
prediction
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 44
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 49
predictMeasurement
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 45
publish_accumulated_pixels
visual_odom.robot.VisualRobotSample, 85
publish_odometry_msg
visual_odom.robot.VisualRobotSample, 85
publish_pixels
visual_odom.robot.VisualRobotSample, 86
publish_pointcloud_map
visual_odom.robot.VisualRobotSample, 86
visual_odom.visual_odom_map.VisualOdomMap, 82
publish_vision_cone
visual_odom.robot.VisualRobotSample, 86
publish_yourself
visual_odom.robot.VisualRobotSample, 86
publisher_accumulated_pixels
visual_odom.visual_odom_node.VisualOdom, 74
publisher_cone
visual_odom.visual_odom_node.VisualOdom, 74
publisher_image
visual_odom.visual_odom_node.VisualOdom, 74
publisher_keypoints_3d
visual_odom.visual_odom_node.VisualOdom, 74
publisher_visual_odometry_msg
visual_odom.visual_odom_node.VisualOdom, 74

Q
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 47
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 50
quaternion_to_yaw
visual_odom.csv_creator_node.OdomImuCsvWriter, 60

R
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 47
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 50
ransac
param, 21
ransac_calculation
visual_odom.visual_odom_node.VisualOdom, 71
ransac_delta_p
visual_odom.robot.VisualRobotSample, 89
ransac_delta_theta
visual_odom.robot.VisualRobotSample, 89
ransac_draw_keypoints
visual_odom.robot.VisualRobotSample, 89
RANSAC_EVALUATION_TOLERANCE
visual_odom.constants, 30
ransac_evaluation_tolerance
visual_odom.constants.Parameters, 63
ransac_failed
visual_odom.robot.VisualRobotSample, 87
ransac_frame_to_frame
visual_odom.visual_odom_node.VisualOdom, 71
ransac_frame_to_map
visual_odom.visual_odom_node.VisualOdom, 72
ransac_inlier_ratio
visual_odom.robot.VisualRobotSample, 89
RANSAC_ITERATION
visual_odom.constants, 30
ransac_iteration
visual_odom.constants.Parameters, 63
RANSAC_MIN_INLIER_RATIO
visual_odom.constants, 30
ransac_min_inlier_ratio
visual_odom.constants.Parameters, 63
RANSAC_SAMPLE_SIZE
visual_odom.constants, 31
ransac_sample_size
visual_odom.constants.Parameters, 63
README.md, 92
relative_file_paths
metadata, 19
reliability
metadata, 19
RESAMPLE_POS_TOLERANCE
visual_odom.constants, 31
RESAMPLE_THETA_TOLERANCE
visual_odom.constants, 31
RESAMPLE_TIME_TOLERANCE
visual_odom.constants, 31
reset_trust
visual_odom.landmark.Landmark, 55
RESOLUTION_CONE_H
visual_odom.constants, 31
RESOLUTION_CONE_V
visual_odom.constants, 31
RGB_DEPTH_SYNC_TOLERANCE_SEC
visual_odom.constants, 31
rgb_depth_sync_tolerance_sec
param, 21
visual_odom.constants.Parameters, 63
RGB_IMAGE_TOPIC
visual_odom.constants, 31
robot_iteration
visual_odom.robot.VisualRobotSample, 87

robots
visual_odom.visual_odom_node.VisualOdom, 74
ros2_ws/src/visual_odom/config/param.yaml, 92
ros2_ws/src/visual_odom/launch/visual_odom_launch.py, 93
ros2_ws/src/visual_odom/setup.py, 93
ros2_ws/src/visual_odom/visual_odom/__init__.py, 93
ros2_ws/src/visual_odom/visual_odom/constants.py, 93
ros2_ws/src/visual_odom/visual_odom/csv_creator_node.py, 96
ros2_ws/src/visual_odom/visual_odom/ekf_landmark.py, 97
ros2_ws/src/visual_odom/visual_odom/ekf_robot.py, 97
ros2_ws/src/visual_odom/visual_odom/landmark.py, 97
ros2_ws/src/visual_odom/visual_odom/odom_buffer.py, 97
ros2_ws/src/visual_odom/visual_odom/robot.py, 98
ros2_ws/src/visual_odom/visual_odom/tf_methods.py, 98
ros2_ws/src/visual_odom/visual_odom/visual_odom_map.py, 98
ros2_ws/src/visual_odom/visual_odom/visual_odom_node.py, 99
ros__parameters
param, 21
ros_distro
metadata, 19
rosbag2_bagfile_information
metadata, 19
rosbag_path
param, 21
ROT_BK
visual_odom.constants, 31

sample_size
param, 21
sec
metadata, 19
serialization_format
metadata, 19
set_jacobian_F
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 45
set_jacobian_H
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 45
set_log_weight
visual_odom.robot.VisualRobotSample, 87
set_odom_coordinates
visual_odom.landmark.Landmark, 55
set_Q
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 45
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 49
set_R
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 45
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 49
set_state
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 49
visual_odom.robot.VisualRobotSample, 87
set_weight
visual_odom.robot.VisualRobotSample, 87
setup, 22
data_files, 22
description, 22
entry_points, 22
extras_require, 22
install_requires, 22
license, 22
maintainer, 22
maintainer_email, 22
name, 22
package_name, 22
packages, 22
version, 23
zip_safe, 23
SIGMA_PIXEL
visual_odom.constants, 31
sigma_R_approximation
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 46
SIGMA_THETA_ODOM_WHEEL_Q
visual_odom.constants, 31
SIGMA_X_ODOM_WHEEL_Q
visual_odom.constants, 32
SIGMA_Y_ODOM_WHEEL_Q
visual_odom.constants, 32
stamp
visual_odom.odom_buffer.odom_message, 56
starting_time
metadata, 19
state_func
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 46
storage_identifier
metadata, 19
subscription_depth
visual_odom.visual_odom_node.VisualOdom, 74
subscription_rgb
visual_odom.visual_odom_node.VisualOdom, 74
subscription_wheel_odom
visual_odom.visual_odom_node.VisualOdom, 74

tf_broadcaster
visual_odom.visual_odom_node.VisualOdom, 74
tf_buffer
visual_odom.visual_odom_node.VisualOdom, 74
tf_listener
visual_odom.visual_odom_node.VisualOdom, 74
TF_STATIC_TOPIC
visual_odom.constants, 32
theta
visual_odom.constants.State, 66

visual_odom.robot.VisualRobotSample, 89
visual_odom.visual_odom_node.VisualOdom, 74
topic_visual_odometry_msg
visual_odom.constants.Parameters, 63
topics_with_message_count
metadata, 19
total_publisher
visual_odom.robot.VisualRobotSample, 89
visual_odom.visual_odom_node.VisualOdom, 74
trust
visual_odom.landmark.Landmark, 56
type
metadata, 19
type_description_hash
metadata, 19

u
visual_odom.constants.PixelCoordinate, 64
update
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 46
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 50

v
visual_odom.constants.PixelCoordinate, 64
valid_des
visual_odom.visual_odom_node.VisualOdom, 74
valid_des_last
visual_odom.visual_odom_node.VisualOdom, 75
valid_kp
visual_odom.visual_odom_node.VisualOdom, 75
valid_kp_depth
visual_odom.visual_odom_node.VisualOdom, 75
valid_kp_depth_last
visual_odom.visual_odom_node.VisualOdom, 75
valid_kp_last
visual_odom.visual_odom_node.VisualOdom, 75
version
metadata, 19
setup, 23
visible_landmarks
visual_odom.robot.VisualRobotSample, 89
visual_odom.visual_odom_node.VisualOdom, 75
VISION_CONE_TOPIC
visual_odom.constants, 32
Visual Odometry SLAM Package, 1
visual_odom, 23
visual_odom.config, 23
visual_odom.constants, 23
ACCUMULATED_POINTCLOUD_FRAME_TOPIC, 27
BASE_LINK_FRAME_ID, 27
CAMERA_ANGLE_HOR_RAD, 27
CAMERA_ANGLE_VER_RAD, 27
CAMERA_POS_IN_BASELINK, 27
CU, 27
CV, 27
DECREASE_TRUST_FACTOR, 27
DEPTH_IMAGE_TOPIC, 27
ERROR_MIN_DEPTH, 28
ERROR_QUADRATIC_DEPTH, 28
F, 28
FILTERED_ODOMETRY_TOPIC, 28
GAUSS_NOISE_THETA_SIGMA, 28
GAUSS_NOISE_X_SIGMA, 28
GAUSS_NOISE_Y_SIGMA, 28
IMU_ODOMETRY_TOPIC, 28
INCREASE_TRUST_VALUE, 28
INITIAL_LANDMARK_COUNT, 28
KEYPOINT_POINTCLOUD_FRAME_TOPIC, 28
KINECT_FRAME_ID, 28
KP_IMAGE_TOPIC, 29
MATCHES_FOR_NEW_LANDMARKS, 29
MAX_ADDITIONAL_NOISE_RANSAC_FAILURE, 29
MAX_ALTITUDE, 29
MAX_AZIMUTH, 29
MAX_DEPTH, 29
MAX_ROTATION_ANGLE_DEG, 29
MAX_U, 29
MAX_V, 29
MIN_DEPTH, 29
MIN_DET_VALUE, 29
MIN_LANDMARK_TRUST, 29
MIN_MATCHES_FOR_RANSAC, 30
N_ROBOT_SAMPLES, 30
NOISE_INCREMENT_RANSAC_FAILURE, 30
NOISE_ROTATION_THRESHOLD, 30
NOISE_TRANSLATION_THRESHOLD, 30
normalize_angle, 27
parameters, 30
PERCENTAGE_OF_PLAY_SPEED, 30
PIXEL_TOLERANCE, 30
POINTCLOUD_FRAME_TOPIC, 30
RANSAC_EVALUATION_TOLERANCE, 30
RANSAC_ITERATION, 30
RANSAC_MIN_INLIER_RATIO, 30
RANSAC_SAMPLE_SIZE, 31
RESAMPLE_POS_TOLERANCE, 31
RESAMPLE_THETA_TOLERANCE, 31
RESAMPLE_TIME_TOLERANCE, 31
RESOLUTION_CONE_H, 31
RESOLUTION_CONE_V, 31
RGB_DEPTH_SYNC_TOLERANCE_SEC, 31
RGB_IMAGE_TOPIC, 31
ROT_BK, 31
SIGMA_PIXEL, 31
SIGMA_THETA_ODOM_WHEEL_Q, 31
SIGMA_X_ODOM_WHEEL_Q, 32
SIGMA_Y_ODOM_WHEEL_Q, 32
TF_STATIC_TOPIC, 32
VISION_CONE_TOPIC, 32
VISUAL_ODOM_FRAME_ID, 32
VISUAL_ODOM_MSG_TOPIC, 32
VISUAL_ODOM_PATH_TOPIC, 32
WHEEL_ODOMETRY_TOPIC, 32

visual_odom.constants.Coordinate, 39
__add__, 39
__mul__, 40
__sub__, 40
__truediv__, 40
x, 40
y, 40
z, 41
visual_odom.constants.Parameters, 62
matches_for_new_landmarks, 62
max_depth, 62
max_rotation_angle_deg, 62
min_landmark_trust, 63
min_matches_for_ransac, 63
n_robot_samples, 63
pixel_tolerance, 63
ransac_evaluation_tolerance, 63
ransac_iteration, 63
ransac_min_inlier_ratio, 63
ransac_sample_size, 63
rgb_depth_sync_tolerance_sec, 63
topic_visual_odometry_msg, 63
visual_odom.constants.PixelCoordinate, 63
u, 64
v, 64
z, 64
visual_odom.constants.State, 64
__add__, 65
__mul__, 65
__sub__, 65
__truediv__, 65
theta, 66
x, 66
y, 66
visual_odom.csv_creator_node, 32
main, 33
visual_odom.csv_creator_node.OdomImuCsvWriter, 58
__init__, 59
csv_file, 61
csv_path, 61
destroy_node, 59
imu_callback, 59, 61
imu_theta, 61
normalize_angle, 59
odom_callback, 60, 61
odom_visual_theta, 61
odom_visual_x, 61
odom_visual_y, 61
quaternion_to_yaw, 60
visual_odom_callback, 60, 61
wheel_x, 61
wheel_y, 61
visual_odom.ekf_landmark, 33
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 41
__init__, 42
calculate_jacobian_F, 42
calculate_jacobian_H, 42
calculate_Q_matrix, 43
calculate_R_sigma_matrix, 43
computeKalmanGain, 43
get_R, 43
JF, 47
JH, 47
landmark_kalman_iteration, 44
meas_func, 44
P, 47
prediction, 44
predictMeasurement, 45
Q, 47
R, 47
set_jacobian_F, 45
set_jacobian_H, 45
set_Q, 45
set_R, 45
sigma_R_approximation, 46
state_func, 46
update, 46
x, 47
visual_odom.ekf_robot, 33
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 47
__init__, 48
add_noise_to_R, 48
additional_noise, 50
computeKalmanGain, 48
F, 50
get_covariance_p, 49
get_state, 49
H, 50
P, 50
prediction, 49
Q, 50
R, 50
set_Q, 49
set_R, 49
set_state, 49
update, 50
x, 50
visual_odom.landmark, 33
kabsch, 34
visual_odom.landmark.Landmark, 50
__init__, 52
calculate_likelihood, 52
color, 55
decrease_trust, 53
des, 55
get_color, 53
get_depth, 53
get_descriptor, 53
get_kinect_coordinates, 53
get_odom_coordinates, 53
get_P, 53
get_trust, 54
get_u, 54
get_v, 54
increase_trust, 54

is_visible, 54
kinect_coordinates, 55
landmark_ekf, 55
landmark_kalman_iteration, 55
odom_coordinates, 55
P, 56
pixel_coor, 56
reset_trust, 55
set_odom_coordinates, 55
trust, 56
visual_odom.odom_buffer, 34
visual_odom.odom_buffer.odom_message, 56
msg, 56
pose, 56
stamp, 56
visual_odom.odom_buffer.OdomBuffer, 57
__init__, 57
add_odom_message, 57
get_interpolated, 57
get_latest, 57
Odom_Buffer, 57
visual_odom.robot, 34
visual_odom.robot.VisualRobotSample, 82
__init__, 83
acc_ransac_noise_delta, 88
accumulate_pixels, 84
accumulated_pixels, 88
bf, 88
cone_publisher, 88
covariance_P, 88
delta_ransac_state, 88
delta_wheel_odom_ransac, 88
ekf, 88
first_iteration, 88
frame_depth, 88
frame_rgb, 88
get_covariance_P, 84
get_drawn_keypoints, 84
get_log_weight, 84
get_position, 84
get_theta, 84
get_weight, 84
index, 88
init_camera_cone, 85
last_wheel_odom, 88
log_weight, 89
map_publisher, 89
noise_Q, 89
odometry_msg_publisher, 89
path, 89
path_color, 89
pos_baselink, 89
pos_visual_odom, 89
predict_with_wheel_odom, 85
publish_accumulated_pixels, 85
publish_odometry_msg, 85
publish_pixels, 86
publish_pointcloud_map, 86
publish_vision_cone, 86
publish_yourself, 86
ransac_delta_p, 89
ransac_delta_theta, 89
ransac_draw_keypoints, 89
ransac_failed, 87
ransac_inlier_ratio, 89
robot_iteration, 87
set_log_weight, 87
set_state, 87
set_weight, 87
theta, 89
total_publisher, 89
visible_landmarks, 89
visual_odom_map, 90
visual_odom.tf_methods, 34
calculate_tf, 35
kinect_depth_to_baselink, 35
kinect_depth_to_odom, 35
odom_to_baselink, 36
pixel_to_kinect, 36
pixel_to_kinect_, 36
visual_odom.visual_odom_map, 37
visual_odom.visual_odom_map.VisualOdomMap, 75
__init__, 78
add_landmark, 78
add_landmarks_from_kps, 78
calculate_log_weight, 78
cleanup_old_landmarks, 79
descriptors, 82
get_all_landmarks, 79
get_colors, 79
get_depth, 79
get_descriptors, 80
get_kinect_coordinates, 80
get_landmark, 80
get_odom_coordinates, 80
get_size, 80
get_u_coordinates, 81
get_v_coordinates, 81
get_visible_landmarks, 81
landmark_kalman_iteration, 81
publish_pointcloud_map, 82
visual_odom.visual_odom_node, 37
create_particle_path_markers, 37
main, 37
visual_odom.visual_odom_node.VisualOdom, 66
__init__, 68
_stamp_to_sec, 69
best_robot, 72
best_robot_idx, 72
best_weight, 72
bf, 72
bridge, 72
covariance_P, 72
first_des, 73
first_frame_stamp, 73
first_iteration, 73

first_kp, 73
first_kp_depth, 73
first_pos, 73
first_theta, 73
frame_depth, 73
frame_depth_stamp, 73
frame_rgb, 73
frame_stamp, 73
img_to_kp_des_filtered, 69
iteration, 69
listener_depth_callback, 70
listener_rgb_callback, 70
listener_wheel_odom_callback, 70
orb, 73
parameter_callback, 70, 73
parameter_initialization, 70
pos_baselink, 73
position_initialized, 73
possible_resample, 71
publisher_accumulated_pixels, 74
publisher_cone, 74
publisher_image, 74
publisher_keypoints_3d, 74
publisher_visual_odometry_msg, 74
ransac_calculation, 71
ransac_frame_to_frame, 71
ransac_frame_to_map, 72
robots, 74
subscription_depth, 74
subscription_rgb, 74
subscription_wheel_odom, 74
tf_broadcaster, 74
tf_buffer, 74
tf_listener, 74
theta, 74
total_publisher, 74
valid_des, 74
valid_des_last, 75
valid_kp, 75
valid_kp_depth, 75
valid_kp_depth_last, 75
valid_kp_last, 75
visible_landmarks, 75
visual_odom_path, 75
visual_odom_callback
visual_odom.csv_creator_node.OdomImuCsvWriter, 60, 61
VISUAL_ODOM_FRAME_ID
visual_odom.constants, 32
visual_odom_launch, 38
generate_launch_description, 38
visual_odom_map
visual_odom.robot.VisualRobotSample, 90
VISUAL_ODOM_MSG_TOPIC
visual_odom.constants, 32
visual_odom_node
param, 21
visual_odom_path
visual_odom.visual_odom_node.VisualOdom, 75
VISUAL_ODOM_PATH_TOPIC
visual_odom.constants, 32

WHEEL_ODOMETRY_TOPIC
visual_odom.constants, 32
wheel_x
visual_odom.csv_creator_node.OdomImuCsvWriter, 61
wheel_y
visual_odom.csv_creator_node.OdomImuCsvWriter, 61

x
visual_odom.constants.Coordinate, 40
visual_odom.constants.State, 66
visual_odom.ekf_landmark.ExtendedKalmanFilterLandmark, 47
visual_odom.ekf_robot.ExtendedKalmanFilterRobot, 50

y
visual_odom.constants.Coordinate, 40
visual_odom.constants.State, 66

z
visual_odom.constants.Coordinate, 41
visual_odom.constants.PixelCoordinate, 64
zip_safe
setup, 23